

35. Ethernet CPU Agent (GWCA)

35.1 Overview

Ethernet CPU Agent (GWCA) is a CPU interface that consists of an agent interface module allowing communication within ESWM. GWCA handles data exchange between ESWM and the GWCPU subsystem. Software is required for the configuration of Gateway AXI bus interface.

Table 35.1 lists the GWCA specifications and Figure 35.1 shows a block diagram.

Table 35.1 GWCA Specifications

Function		Description
Interfaces	Switch mode	Interface to receive the switch mode [MFWD]
	Pause interface	Interface to pause TX queues [COMA]
	AXI Master interface	Master interface conforming to AXI 3 AMBA protocol. It is used for data transfer between GWCPU and GWCA. • Data bit width: 128 bits • Number of outstanding read: 8 • Number of outstanding write: 8 • Number of ID read: 8 (from 0 to 7) • Number of ID write: 8 (from 0 to 8-1) • Incremental Access only • Supports unaligned transactions • Supports out of order • Supports interleave on read bus
	SFR interface	Interface to access GWCA SFRs [COMA]
	Interrupts	GWCA interrupt to CPU
	Fabric interface	Interface to communicate with the data, TAG and pointer RAM [MFAB]
	Descriptor bus	Interface to receive frame information to send them to CPU [MFWD]
	L2/L3 update bus	Interface to fetch the frame routing information [MFWD]
	BP Request	Interface to receive BPs to store frames in the data, TAG and pointer RAM [COMA]
	BP Release	Interface to release the BPs that has been used [COMA]
	TX Timestamp Capture interface	Interface to receive TX timestamps and send it to CPU [RMAC]
	RAM interfaces	Interface to communication with RAMs [TOP]
Data provision	Ethernet Frames	Corresponds to the IEEE 802.3, 802.1Q and 802.1CB standards [802.3] [802.1Q] [802.1CB]
	Descriptors	AXI TX Descriptor (CPU to GWCA) AXI RX Descriptor (GWCA to CPU) Local Ethernet Descriptor, Local Direct Descriptor (GWCA to Forwarding engine) Forwarding descriptor (Forwarding engine to GWCA)
	Timestamp	TX timestamp for transmission to CPU RAM [RMAC]

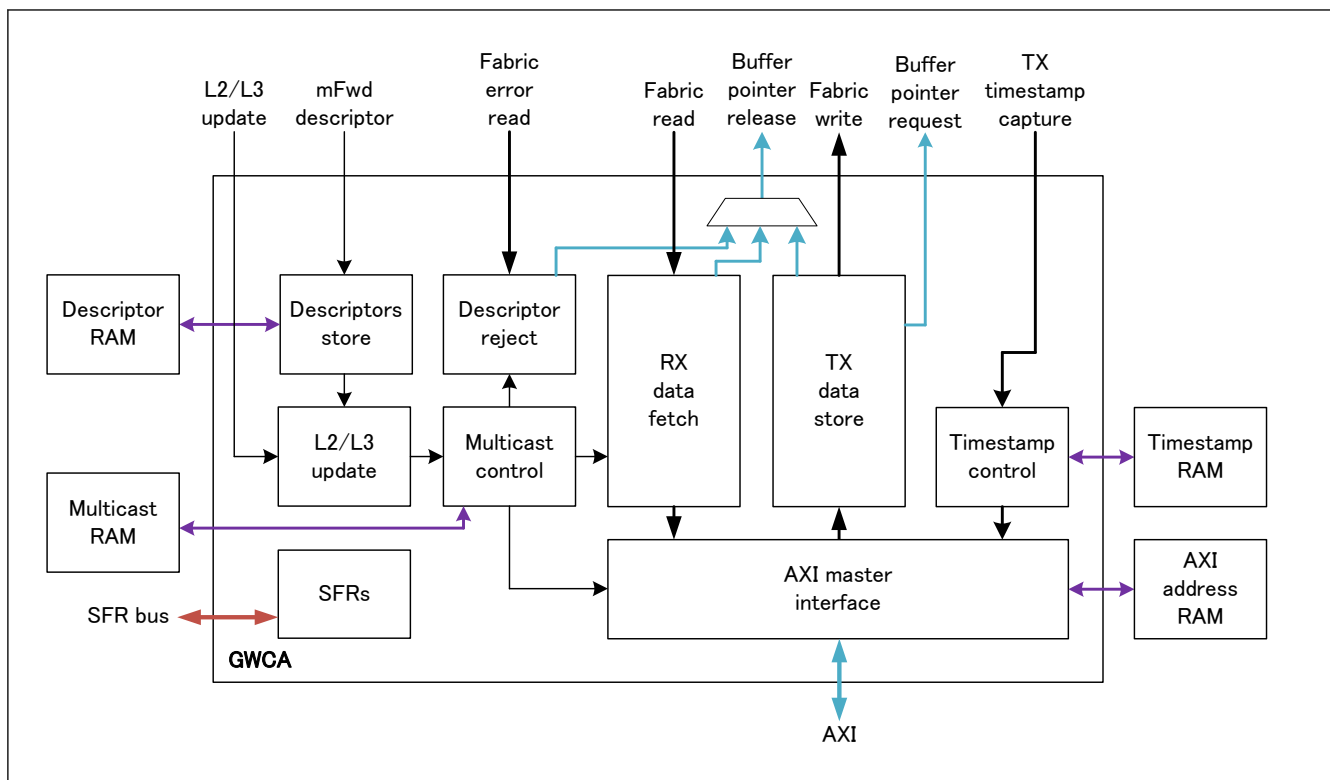


Figure 35.1 GWCA block diagram

Table 35.2 GWCA functional blocks

Block name	Function
TX data store	Transmits frames from AXI master to the local RAM.
RX data fetch	Transmits frame from local RAM to AXI master.
Descriptor store	Receives the descriptors from forwarding engine and controls the priority between descriptors.
L2/L3 update	Fetches L2/L3 update information.
Descriptor reject	Reject descriptors.
Multicast control	Controls multicast transactions.
Timestamp control	Receives timestamps for Ethernet Agents, saves them and transmits them to the AXI master.
AXI master interface	AXI master for data exchange with the CPU
SFRs	GWCA SFRs

35.2 GWCA Register List

Table 35.3 GWCA registers (1 of 4)

Offset address	Register symbol	Register name	Write access mode
0x0000	GWMC	Mode Configuration Register	Any
0x0004	GWMS	Mode Status Register	—
0x0010	GWIRC	IPV Remapping Configuration Register [802.1Q]	C
0x0014	GWRDQSC	RX Descriptor Queue Security Configuration Register	C, O
0x0018	GWRDQC	RX Descriptor Queue Control Register	O
0x001C	GWRDQAC	RX Descriptor Queue Arbitration Control Register	C
0x0020	GWRGC	RX General Configuration Register	C

Table 35.3 GWCA registers (2 of 4)

Offset address	Register symbol	Register name	Write access mode
0x0040 + 0x04 × q	GWRMFSCq	Reception Maximum Frame Size Configuration Register q (q = 0 to 7)	C, O
0x0060 + 0x04 × q	GWRDQDCq	Reception Descriptor Queue q Depth Configuration Register (q = 0 to 7)	C
0x0080 + 0x04 × q	GWRDQMq	RX Descriptor Queue q Monitoring Register (q = 0 to 7)	—
0x00A0 + 0x04 × q E: 0x00C0 + 0x04 × q	GWRDQMLMq	RX Descriptor Queue q Max Level Monitoring Register (q = 0 to 7)	—
0x0100	GWMTIRM	Multicast Table Initialization Register Monitoring Register	C, O
0x0104	GWMSTLS	Multicast Table Learning Setting Register	C, O
0x0108	GWMSTLR	Multicast Table Learning Result Register	—
0x010C	GWMSTSS	Multicast Table Searching Setting Register	C, O
0x0110	GWMSTSR	Multicast Table Searching Result Register	—
0x0120	GWMAC0	MAC Address Configuration Register 0	C
0x0124	GWMAC1	MAC Address Configuration Register 1	C
0x0130	GWVCC	VLAN Control Configuration Register	C
0x0134	GWVTC	VLAN TAG Configuration Register	C
0x0138	GWTTCF	Transmission TAG Filtering Configuration Register	C
0x0140 + 0x08 × s	GWTDCAcs0	Timestamp Descriptor Chain s Address Configuration Register 0 (s = 0, 1)	C, O
0x0144 + 0x08 × s	GWTDCAcs1	Timestamp Descriptor Chain s Address Configuration Register 1 (s = 0, 1)	C, O
0x0160 + 0x04 × s	GWTSDCCs	Timestamp Descriptor Chain s Configuration Register (s = 0, 1)	C
0x0180	GWTSNM	Timestamp Number Monitoring Register	C, O
0x0184 E: 0x0188	GWTSNMNM	Timestamp Maximum Number Monitoring Register	C, O
0x0190	GWAC	AXI Control Register	O
0x0194	GWDCBAC0	Descriptor Chain Base Address Configuration Register 0	C
0x0198	GWDCBAC1	Descriptor Chain Base Address Configuration Register 1	C
0x01A0	GWMDNC	Maximum Descriptor Number Configuration Register	C
0x0200 + 0x4 × i	GWTRCi	Transmission Request Configuration Register i (i = 0, 1)	O
0x0300 + 0x4 × p	GWTPCp	Transmission Pause Configuration Register p (p = 0, 1)	C
0x0380	GWARIRM	AXI RAM Initialization Register Monitoring Register	C, O
0x0400 + 0x4 × i	GWDCCi	Descriptor Chain i Configuration Register (i = 0 to 63)	C, O
0x0800	GWAARSS	AXI Address RAM Searching Setting Register	C, O
0x0804	GWAARSR0	AXI Address RAM Searching Result Register 0	—
0x0808	GWAARSR1	AXI Address RAM Searching Result Register 1	—
0x0840 + 0x4 × i	GWIDAUASi	Incremental Data Area i Used Area Size Register (i = 0 to 3)	C, O, D
0x0880 + 0x4 × i	GWIDASMi	Incremental Data Area i Size Monitoring Register (i = 0 to 3)	—
0x0900 + 0x8 × i	GWIDASAMi0	Incremental Data Area i Start Address Monitoring Register 0 (i = 0 to 3)	—

Table 35.3 GWCA registers (3 of 4)

Offset address	Register symbol	Register name	Write access mode
0x0904 + 0x8 × i	GWIDASAMi1	Incremental Data Area i Start Address Monitoring Register 1 (i = 0 to 3)	—
0x0980 + 0x8 × i	GWIDACAMi0	Incremental Data Area i Current Address Monitoring Register 0 (i = 0 to 3)	—
0x0984 + 0x8 × i	GWIDACAMi1	Incremental Data Area i Current Address Monitoring Register 1 (i = 0 to 3)	—
0x0A00	GWGRLC	Global Rate Limiter Configuration Register	C, O
0x0A04	GWGRLULC	Global Rate Limiter Upper Limit Configuration Register	C, O
0x0A80 + 0x8 × i	GWRLCi	Rate Limiter i Configuration Register (i = 0 to 7)	C, O
0x0A84 + 0x8 × i	GWRLULCi	Rate Limiter i Upper Limit Configuration Register (i = 0 to 7)	C, O
0x0B80	GWIDPC	Interrupt Delay Prescaler Configuration Register	C
0x0C00 + 0x4 × i	GWIDCi	Interrupt Delay Configuration Register i (i = 0 to 63)	C, O
0x1000 E: 0x1080	GWRDCN	Received Data Counter Register	—
0x1004 E: 0x1084	GWTDCN	Transmitted Data Counter Register	—
0x1008 E: 0x1088	GWTSCN	Timestamp Counter Register	—
0x100C E: 0x108C	GWTSOVFECDN	Timestamp Overflow Error Counter Register	—
0x1010 E: 0x1090	GWUSMFSECDN	Under Minimum Frame Size Error Counter Register	—
0x1014 E: 0x1094	GWTFECN	TAG Filtering Error Counter Register	—
0x1018 E: 0x1098	GWSEQECN	Sequence Error Counter Register	—
0x1020 E: 0x10A0	GWTXDNECN	TX Descriptor Number Error Counter Register	—
0x1024 E: 0x10A4	GWFFECN	Frame Size Error Counter Register	—
0x1028 E: 0x10A8	GWTDFFECN	Timestamp Descriptor Full Error Counter Register	—
0x102C E: 0x10AC	GWTSDFECN	Timestamp Descriptor Number Error Counter Register	—
0x1030 E: 0x10B0	GWDDQOECN	Descriptor Queue Overflow Error Counter Register	—
0x1034 E: 0x10B4	GWDDQSECN	Descriptor Queue Security Error Counter Register	—
0x1038 E: 0x10B8	GWDFECN	Descriptor Full Error Counter Register	—
0x103C E: 0x10BC	GWDFSECN	Descriptor Security Error Counter Register	—
0x1040 E: 0x10C0	GWDSZECN	Data Size Error Counter Register	—
0x1044 E: 0x10C4	GWDCTECN	Descriptor Chain Type Error Counter Register	—
0x1048 E: 0x10C8	GWRXDNECN	RX Descriptor Number Error Counter Register	—
0x1100 + 0x10 × i	GWDISi	Data Interrupt Status Register i (i = 0, 1)	C, O, D

Table 35.3 GWCA registers (4 of 4)

Offset address	Register symbol	Register name	Write access mode
0x1104 + 0x10 × i	GWDIEi	Data Interrupt Enable Register i (i = 0, 1)	C, O, D
0x1108 + 0x10 × i	GWDIDi	Data Interrupt Disable Register i (i = 0, 1)	C, O, D
0x110C + 0x10 × i	GWDIDSi	Data Interrupt Delayed Status Register i (i = 0, 1)	—
0x1180	GWTSDIS	Timestamp Data Interrupt Status Register	C, O, D
0x1184	GWTSDIE	Timestamp Data Interrupt Enable Register	C, O, D
0x1188	GWTSDID	Timestamp Data Interrupt Disable Register	C, O, D
0x1190	GWEIS0	Error Interrupt Status Register 0	C, O, D
0x1194	GWEIE0	Error Interrupt Enable Register 0	C, O, D
0x1198	GWEID0	Error Interrupt Disable Register 0	C, O, D
0x11A0	GWEIS1	Error Interrupt Status Register 1	C, O, D
0x11A4	GWEIE1	Error Interrupt Enable Register 1	C, O, D
0x11A8	GWEID1	Error Interrupt Disable Register 1	C, O, D
0x1200 + 0x10 × i	GWEIS2i	Error Interrupt Status Register 2i (i = 0, 1)	C, O, D
0x1204 + 0x10 × i	GWEIE2i	Error Interrupt Enable Register 2i (i = 0, 1)	C, O, D
0x1208 + 0x10 × i	GWEID2i	Error Interrupt Disable Register 2i (i = 0, 1)	C, O, D
0x1280	GWEIS3	Error Interrupt Status Register 3	C, O, D
0x1284	GWEIE3	Error Interrupt Enable Register 3	C, O, D
0x1288	GWEID3	Error Interrupt Disable Register 3	C, O, D
0x1290	GWEIS4	Error Interrupt Status Register 4	C, O, D
0x1294	GWEIE4	Error Interrupt Enable Register 4	C, O, D
0x1298	GWEID4	Error Interrupt Disable Register 4	C, O, D
0x12A0	GWEIS5	Error Interrupt Status Register 5	C, O, D
0x12A4	GWEIE5	Error Interrupt Enable Register 5	C, O, D
0x12A8	GWEID5	Error Interrupt Disable Register 5	C, O, D

- Note:
- Base address: GWCA0 = 0x403C_E000, GWCA0_NS = 0x503C_E000
 - All registers can be read in any mode.
 - The address preceded by "E:" correspond to an emulation address which allows a register to be read without modifying its content.
 - Write Access Mode:
 - Any: Register can be written in any mode.
 - R: Register can be written in RESET mode
 - D : Register can be written in DISABLE mode
 - C: Register can be written in CONFIG mode
 - O: Register can be written in OPERATION mode

35.3 GWCA Register Descriptions

35.3.1 GWCA Mode Function Registers

35.3.1.1 GWMC : Mode Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0000

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	OPC[1:0]
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Symbol	Function	R/W
1:0	OPC[1:0]	Operating Mode Command*1 Used to set GWCA mode. For more details, see section 35.4.1. Operation Modes . 0 0: Enter RESET mode 0 1: Enter DISABLE mode 1 0: Enter CONFIG mode 1 1: Enter OPERATION mode	R/W
31:2	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: This register is not writable if its value is different than GWMS.OPS.

35.3.1.2 GWMS : Mode Status Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0004

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	OPS[1:0]
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Symbol	Function	R/W
1:0	OPS[1:0]	Operating Mode Status Indicates the current GWCA mode. For more details, see section 35.4.1. Operation Modes . 0 0: RESET mode 0 1: DISABLE mode 1 0: CONFIG mode 1 1: OPERATION mode	R
31:2	—	These bits are read as 0.	R

OPS[1:0] bits (Operating Mode Status)

[Update conditions]

HW: Mode change completed.

This register is change at “Maximum frame communication time (exp: 64 KB / 100 Mbps = 5.12 ms)” from OPERATION to DISABLE.

35.3.2 Reception Function Registers

35.3.2.1 GWIRC : IPV Remapping Configuration Register [802.1Q]

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0010

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	IPVR7[2:0]			—	IPVR6[2:0]			—	IPVR5[2:0]			—	IPVR4[2:0]		
Value after reset:	0	1	1	1	0	1	1	0	0	1	0	1	0	1	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	IPVR3[2:0]			—	IPVR2[2:0]			—	IPVR1[2:0]			—	IPVR0[2:0]		
Value after reset:	0	0	1	1	0	0	1	0	0	0	0	1	0	0	0	0

Bit	Symbol	Function	R/W
2:0	IPVR0[2:0]	IPV remapping 0	R/W
3	—	These bits are read as 0. The write value should be 0.	R/W
6:4	IPVR1[2:0]	IPV remapping 1	R/W
7	—	These bits are read as 0. The write value should be 0.	R/W
10:8	IPVR2[2:0]	IPV remapping 2	R/W
11	—	These bits are read as 0. The write value should be 0.	R/W
14:12	IPVR3[2:0]	IPV remapping 3	R/W
15	—	These bits are read as 0. The write value should be 0.	R/W
18:16	IPVR4[2:0]	IPV remapping 4	R/W
19	—	These bits are read as 0. The write value should be 0.	R/W
22:20	IPVR5[2:0]	IPV remapping 5	R/W
23	—	These bits are read as 0. The write value should be 0.	R/W
26:24	IPVR6[2:0]	IPV remapping 6	R/W
27	—	These bits are read as 0. The write value should be 0.	R/W
30:28	IPVR7[2:0]	IPV remapping 7	R/W
31	—	These bits are read as 0. The write value should be 0.	R/W

IPVRi (i = 0 to 7) bits (IPV remapping i)

Configure to which descriptor queue descriptor received with IPV i will be stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to i).

35.3.2.2 GWRDQSC : RX Descriptor Queue Security Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0014

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	RDQS L7	RDQS L6	RDQS L5	RDQS L4	RDQS L3	RDQS L2	RDQS L1	RDQS L0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	RDQSL7 to RDQSL0	RX Descriptor Queue i Security Level (The variable i corresponds to the bit number.) When a queue is secured, an unsecured descriptor cannot enter it (when a descriptor is from MFWD with FDESCR.SEC equal to 0). 0: Queue i unsecure 1: Queue i secure	R/W
31:8	—	These bits are read as 0. The write value should be 0.	R/W

35.3.2.3 GWRDQC : RX Descriptor Queue Control Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0018

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	RDQP 7	RDQP 6	RDQP 5	RDQP 4	RDQP 3	RDQP 2	RDQP 1	RDQP 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	RDQD 7	RDQD 6	RDQD 5	RDQD 4	RDQD 3	RDQD 2	RDQD 1	RDQD 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	RDQD7 to RDQD0	RX Descriptor Queue i Disable (The variable i corresponds to the bit position number.) Avoid descriptor from MFWD to enter the corresponding queue. The Forwarding Engine will reject the corresponding descriptor. If disabled queue data is not needed anymore, corresponding GWRMFSCq.MFS register can be set to 0 in order to avoid more frames to go to the CPU RAM. 0: Queue i enabled 1: Queue i disabled	R/W
15:8	—	These bits are read as 0. The write value should be 0.	R/W
23:16	RDQP7 to RDQP0	RX Descriptor Queue i Pause* ¹ (The variable i corresponds to the bit position number.) Avoid frames to be sent to the CPU by stopping descriptor fetching from the descriptor RAM. 0: Queue i active 1: Queue i paused	R/W* ²
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: Pausing a queue during a long time could result in switch overflow. In case of switch overflow [COMA] this register should be set to 0.

Note 2. Read value differs from written value.

RDQDi bits (RX Descriptor Queue i Disable)

[Setting condition]

SW: Writing 1 to this bit will set it.

[Clearing condition]

SW: Writing 0 to this bit will clear it.

HW: Going out of OPERATION mode will clear this register (GWMC.OPC != 11b).

RDQPi bits (RX Descriptor Queue i Pause)

[Setting condition]

SW: Writing 1 to this bit will set it.

[Clearing condition]

SW: Writing 0 to this bit will clear it.

HW: Going out of OPERATION mode will clear this register (GWMC.OPC != 11b).

[Cautions]

This register is used to stop a queue but MFWD does not stop sending descriptor to the corresponding queue. In this case the queue might overflow.

35.3.2.4 GWRDQAC : RX Descriptor Queue Arbitration Control Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x001C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	RDQA7[3:0]				RDQA6[3:0]				RDQA5[3:0]				RDQA4[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	RDQA3[3:0]				RDQA2[3:0]				RDQA1[3:0]				RDQA0[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	RDQA0[3:0]	RX Descriptor Queue 0 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
7:4	RDQA1[3:0]	RX Descriptor Queue 1 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
11:8	RDQA2[3:0]	RX Descriptor Queue 2 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
15:12	RDQA3[3:0]	RX Descriptor Queue 3 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
19:16	RDQA4[3:0]	RX Descriptor Queue 4 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W

Bit	Symbol	Function	R/W
23:20	RDQA5[3:0]	RX Descriptor Queue 5 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
27:24	RDQA6[3:0]	RX Descriptor Queue 6 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W
31:28	RDQA7[3:0]	RX Descriptor Queue 7 Arbitration* ¹ For more details, see section 35.5.3.1.2. Descriptor Arbitration . 0: Queue i strict arbitration Others: Queue i WRR arbitration	R/W

- Note 1.
- SW: In hybrid arbitration mode, all the queues with a GWRDQAC.RDQAI value other than 0 should have a consecutive priority (priority refers to the queue number i).
 - SW: In all arbitration mode except strict priority arbitration mode, at least two queues should have a GWRDQAC.RDQAI other than 0.

35.3.2.5 GWRGC : RX General Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0020

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	RCPT
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	RCPT	Receive CRC Pass Through [802.3]* ¹ This bit selects to pass FCS field on the reception frame. 0: FCS is not passed to GWCPU. If a descriptor is received from forwarding engine with FDESCR.FI equal to 1, GWCA remove the last 4 bytes of the Frame Data (FCS). 1: FCS is passed to GWCPU GWCA don't remove FCS from the Frame which has one (a descriptor is received from forwarding engine with FDESCR.FI equal to 1) if the frame has not been modified by the switch (No update due to VLAN tagging/untagging and no update requested by L3 routing [FWD]).	R/W
31:1	—	These bits are read as 0. The write value should be 0.	R/W

- Note 1. HW: GWCA does not check FCS integrity.

35.3.2.6 GWRMFSCq : Reception Maximum Frame Size Configuration Register q (q = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0040 + 0x04 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	MFS[15:0]															
Value after reset:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
15:0	MFS[15:0]	Maximum Frame Size ^{*1} Maximum frame size for descriptor queue q. All bigger frames will be rejected.	R/W
31:16	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: The maximum frame size comparison takes in account the frame size which will actually be written by the AXI master (after VLAN untagging, R-TAG insertion and L2L3 update) but always counts in this size FCS even if not received by CPU.

35.3.2.7 GWRDQDCq : Reception Descriptor Queue q Depth Configuration Register (q = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0060 + 0x04 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DQD[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DQD[9:0]	Descriptor Queue Depth ^{*1} Number of descriptors that can contain descriptor queue q. For details see section 35.5.3.1.1. Descriptor Storage .	R/W
31:10	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: The sum of DQD fields should always be smaller or equal to 512.

35.3.2.8 GWRDQMq : RX Descriptor Queue q Monitoring Register (q = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0080 + 0x04 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DNQ[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DNQ[9:0]	Descriptor Number in Queue Indicate the current number of descriptors stored in RX Descriptor Queue q.	R
31:10	—	These bits are read as 0.	R

DNQ[9:0] bits (Descriptor Number in Queue)

[Increment conditions]

HW: Incremented by 1 when a descriptor is received from MFWD for the corresponding queue and, the queue is not full, there is no descriptor security error and the queue is not disabled.

[Decrement conditions]

HW: Decrement by 1 when a descriptor is read by GWCA to send data to AXI Master Interface or to reject data.

35.3.2.9 GWRDQMLMq : RX Descriptor Queue q Max Level Monitoring Register (q = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x00A0 + 0x04 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DMLQ[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DMLQ[9:0]	Descriptor Max Level in Queue Indicates the maximum number of descriptors that has been stored in RX Descriptor Queue q.	R
31:10	—	These bits are read as 0.	R

DMLQ[9:0] bits (Descriptor Max Level in Queue)

[Clearing condition]

HW: Being in RESET mode will clear this register (GWMC.OPS == 00).

SW: By reading this register, it is cleared to GWRDQMq.DNQ.

[Increment conditions]

HW: Increment to GWRDQMq.DNQ value when smaller than GWRDQMq.DNQ.

35.3.3 Reception Multicast Function Registers

35.3.3.1 GWMTIRM : Multicast Table Initialization Register Monitoring Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0100

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	MTR	MTIO G
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	MTIOG	Multicast Table Initialization Ongoing [Setting condition] SW: By writing 1 to this register. It starts Multicast Table initialization. [Clearing condition] HW: This bit is cleared when Multicast Table initialization is finished. Min clk_period * 64 = [exp] (1000/320) * 128 = 400 ns	R/W ^{*1}
1	MTR	Multicast Table Ready [Setting condition] When GWMTIRM.MTIOG is getting cleared. [Clearing condition] By writing 1 to GWMTIRM.MTIOG.	R
31:2	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.3.2 GWMSTLS : Multicast Table Learning Setting Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0104

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	MSEN[5:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	MNL[2:0]			—	—	MNRCNL[5:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
5:0	MNRCNL[5:0]	Multicast Next Descriptor Chain Number Learn This register is used to set the next address in the multicast table to be learnt in GWMSTLS.MSEN[5:0] multicast table entry, see section 35.5.3.3.3. Multicast Learning .	R/W
7:6	—	These bits are read as 0. The write value should be 0.	R/W

Bit	Symbol	Function	R/W
10:8	MNL[2:0]	Multicast Number Learn This register is used to set the multicast number that should be learnt in GWMSTLS.MSENLMulticast table entry. 0 0 0: Multicast 0 (1 target, no multicast) 0 0 1: Multicast 1 (2 targets) ⋮ 1 1 1: Multicast 7 (8 targets)	R/W
15:11	—	These bits are read as 0. The write value should be 0.	R/W
21:16	MSENLM[5:0]	Multicast Setting Entry Number Learn This register is used to set in which address the current learning should happen.	R/W
31:22	—	These bits are read as 0. The write value should be 0.	R/W

35.3.3.3 GWMSTLR : Multicast Table Learning Result Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0108

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	MTL	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	MTLF
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	MTLF	Multicast Table Learning Fail [Update Conditions] HW: GWMSTLR.MTL clear event. 0: Entry learning did not fail because the Multicast table is not ready. 1: Entry learning failed the Multicast table is not ready.	R
30:1	—	These bits are read as 0.	R
31	MTL	Multicast Table Learning [Setting condition] SW: Writing GWMSTLS register will set this bit. [Clearing condition] HW: This bit will be deasserted when the Multicast RAM write is completed.	R

35.3.3.4 GWMSTSS : Multicast Table Searching Setting Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x010C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	MSENS[5:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
5:0	MSENS[5:0]	Multicast Setting Entry Number Search This register is used to read multicast table.	R/W
31:6	—	These bits are read as 0. The write value should be 0.	R/W

35.3.3.5 GWMSTSR : Multicast Table Searching Result Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0110

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	MTS	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	MNR[2:0]			—	—	MNRCNR[5:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
5:0	MNRCNR[5:0]	Multicast Next RX Descriptor Chain Number Result This register is used to read multicast table. [Update conditions] HW: GWMSTSR.MTS clear event.	R
7:6	—	These bits are read as 0.	R
10:8	MNR[2:0]	Multicast Number Result This register is used to read multicast table. [Update conditions] HW: GWMSTSR.MTS clear event.	R
30:11	—	These bits are read as 0.	R
31	MTS	Multicast Table Searching This register is used to read multicast table. [Setting condition] SW: Writing GWMSTSS register will set this bit. [Clearing condition] HW: This bit will be deasserted when the Multicast RAM read is completed.	R

35.3.4 MAC Function Registers

35.3.4.1 GWMAC0 : MAC Address Configuration Register 0

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0120

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	MAU[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	MAU[15:0]	MAC Address Upper Part These bits are used to set the 16 higher-order bits of the MAC address. For example, if the MAC address is 01-23-45-67-89-AB (hexadecimal), set 0x0123 in this register.	R/W
31:16	—	These bits are read as 0. The write value should be 0.	R/W

35.3.4.2 GWMAC1 : MAC Address Configuration Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0124

Bit position: 31 0

Bit field: MAL[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	MAL[31:0]	MAC Address Lower Part These bits are used to set the 32 lower-order bits of the MAC address. For example, if the MAC address is 01-23-45-67-89-AB (hexadecimal), set 0x456789AB in this register.	R/W

35.3.5 TAG Function Registers

35.3.5.1 GWVCC : VLAN Control Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0130

Bit position: 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

Bit field: — — — — — — — — — — — — — VEM[2:0]

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit position: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Bit field: — — — — — — — — — — — — — VIM

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit	Symbol	Function	R/W
0	VIM	VLAN Ingress Mode*1 0: Incoming VLAN mode 1: Port based VLAN mode	R/W
15:1	—	These bits are read as 0. The write value should be 0.	R/W
18:16	VEM[2:0]	VLAN Egress Mode 000: No VLAN mode, frames are transmitted with no VLAN. 001: C-TAG VLAN mode, frames are transmitted with ingress C-TAG if there is one stored in the Local RAM. 010: HW C-TAG VLAN mode, frames are transmitted with the C-TAG stored in local RAM. 011: SC-TAG VLAN mode, frames are transmitted with ingress C-TAG and S-TAG if they were stored in the Local RAM. 100: HW SC-TAG VLAN mode, frames are transmitted with the C-TAG and S-TAG stored in local RAM.	R/W
31:19	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: This register should not be set to 1 is the switch is in no VLAN mode (In forwarding engine, FWGC.SVM is set to 00 [FWD])

35.3.5.2 GWVTC : VLAN TAG Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0134

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	STD	STP[2:0]			STV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CTD	CTP[2:0]			CTV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
11:0	CTV[11:0]	C-TAG VLAN	R/W
14:12	CTP[2:0]	C-TAG PCP	R/W
15	CTD	C-TAG DEI	R/W
27:16	STV[11:0]	S-TAG VLAN	R/W
30:28	STP[2:0]	S-TAG PCP	R/W
31	STD	S-TAG DEI	R/W

35.3.5.3 GWTTFC : Transmission TAG Filtering Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0138

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	UT	SCRT	SCT	CRT	CT	CSRT	CST	RT	NT
Value after reset:	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	NT	No Tag 0: No Tag frame passed 1: No Tag frame rejected	R/W
1	RT	R-TAG 0: R-TAG frame passed 1: R-TAG frame rejected	R/W
2	CST	CoS-TAG 0: CoS-TAG frame passed 1: CoS-TAG frame rejected	R/W
3	CSRT	CoSR-TAG 0: CoSR-TAG frame passed 1: CoSR-TAG frame rejected	R/W

Bit	Symbol	Function	R/W
4	CT	C-TAG 0: C-TAG frame passed 1: C-TAG frame rejected	R/W
5	CRT	CR-TAG 0: CR-TAG frame passed 1: CR-TAG frame rejected	R/W
6	SCT	SC-TAG 0: SC-TAG frame passed 1: SC-TAG frame rejected	R/W
7	SCRT	SCR-TAG 0: SCR-TAG frame passed 1: SCR-TAG frame rejected	R/W
8	UT	Unknown TAG 0: Unknown Tag frame passed 1: Unknown Tag frame rejected	R/W
31:9	—	These bits are read as 0. The write value should be 0.	R/W

35.3.6 Timestamp Function Registers

35.3.6.1 GWTDCAcs0 : Timestamp Descriptor Chain s Address Configuration Register 0 (s = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0140 + 0x08 × s

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	TSCCAU[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TSCCAU[7:0]	TS Descriptor Chain s Current Address Upper Part* ¹	R/W * ²
31:8	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: To overwrite the current address, GWTDCAcs0 and GWTDCAcs1 should always both be written stating by GWTDCAcs1 and finishing by GWTDCAcs0.

Note 2. Read value differs from written value.

TSCCAU[7:0] bits (TS Descriptor Chain s Current Address Upper Part)

[Update conditions]

SW: Writing to this field with update it to the write value.

HW: When an FEMPTY_ND descriptor is read for descriptor queue s, this field value is automatically updated to the next descriptor address upper part.

HW: When a LINK or LINKFIX descriptor is read, this register value is updated to DESCR.PTR[39:32].

35.3.6.2 GWTDCACs1 : Timestamp Descriptor Chain s Address Configuration Register 1 (s = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0144 + 0x08 × s

Bit position: 31

0

Bit field:

TSCCAL[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TSCCAL[31:0]	TS Descriptor Chain s Current Address Lower Part*1	R/W*2

Note 1. SW: To overwrite the current address, GWTDCACs0 and GWTDCACs1 should always both be written stating by GWTDCACs1 and finishing by GWTDCACs0.

Note 2. Read value differs from written value.

TSCCAL[31:0] bits (TS Descriptor Chain s Current Address Lower Part)

[Update conditions]

SW: Writing GWTDCACs0 register with update this field to the previous value written on it by the same APB.

HW: When an FEMPTY_ND descriptor is read for descriptor queue s, this field value is automatically updated to the next descriptor address downer part.

HW: When a LINK or LINKFIX descriptor is read, this register value is updated to DESCR.PTR[31:0].

35.3.6.3 GWTSDCCs : Timestamp Descriptor Chain s Configuration Register (s = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0160 + 0x04 × s

Bit position: 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

Bit field:

—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit position: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Bit field:

—	—	—	—	—	OSID[2:0]					—	—	—	—	—	DCS	TE
---	---	---	---	---	-----------	--	--	--	--	---	---	---	---	---	-----	----

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit	Symbol	Function	R/W
0	TE	Timer Enable*1 Enable timestamp reception for timestamps taken with timer s.	R/W
1	DCS	Descriptor Chain Select Select to which chain timestamps taken with timer s will be received.	R/W
7:2	—	These bits are read as 0. The write value should be 0.	R/W
10:8	OSID[2:0]	OS ID When a memory access is done for timer number s, the OSID set in this register will be used.	R/W
31:11	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: If a timestamp is received from RMAC for a timer which is disabled for timestamp reception, the timestamp will be ignored and lost. This is a valid setting (Another GWCA could have taken the timestamp) so there is no flag to monitor this event.

35.3.6.4 GWTSNM : Timestamp Number Monitoring Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0180

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	TNTR[4:0]				
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	TNTR[4:0]	Timestamp Number in Timestamp RAM [Increment conditions] HW: Incremented by 1 when a timestamp is received from Ethernet Agents. [Decrement conditions] HW: Decrement by 1 when a timestamp is read from timestamp RAM.	R
31:5	—	These bits are read as 0.	R

35.3.6.5 GWTSNMN : Timestamp Maximum Number Monitoring Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0184

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	TMNTR[4:0]				
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	TMNTR[4:0]	Timestamp Maximum Number in Timestamp RAM [Update conditions] HW: Set to GWTSNM.TNTR when GWTSNM.TNTR > GWTSNM.TMNTR. [Clearing condition] HW: Being in RESET mode will clear this register. SW: Reading this register clears it to GWTSNM.TNTR value.	R
31:5	—	These bits are read as 0.	R

35.3.7 AXI Master Registers

35.3.7.1 GWAC : AXI Control Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0190

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	AMP	AMPR
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	AMPR	AXI Master Pause Request This register blocks the AXI master from making new accesses (Because the AXI master has register slices for timing purpose few AXI accesses can be emitted before stopping). 0: AXI master pause not requested 1: AXI master pause requested	R/W
1	AMP	AXI Master Paused “AXI Master Pause” stop only the final processing of the AXI interface. (Internal processing cannot be stopped.) For example, the AXI address for chain [i] is loaded immediately before “AXI Master Paused”. This address will be used after “AXI Master not paused” regardless of AXI address re-configuration (by GWDCCi) during “AXI Master Pause”. 0: AXI master not paused (transaction ongoing) 1: AXI master paused (no transaction ongoing)	R/W
31:2	—	These bits are read as 0. The write value should be 0.	R/W

AMP bit (AXI Master Paused)

[Setting condition]

HW: When GWAC.AMPR is set to 1'b1 and all ongoing AXI transactions are completed, this bit will be set.

[Clearing condition]

HW: When GWAC.AMPR is set to 0, this bit will be set to 0 (it can take time because of asynchronous modules).

35.3.7.2 GWDCBAC0 : Descriptor Chain Base Address Configuration Register 0

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0194

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	DCBAU[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DCBAU[7:0]	Descriptor Chain Base Address Upper Part For further setting information, see section 35.5.1.3.1. LINKFIX Table Initialization .	R/W
31:8	—	These bits are read as 0. The write value should be 0.	R/W

35.3.7.3 GWDCBAC1 : Descriptor Chain Base Address Configuration Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0198

Bit position: 31

0

Bit field:

DCBAL[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	DCBAL[31:0]	Descriptor Chain Base Address Lower Part For further setting information, see section 35.5.1.3.1. LINKFIX Table Initialization .	R/W

35.3.7.4 GWMDNC : Maximum Descriptor Number Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x01A0

Bit position:

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

Bit field:

— — — — — — — — — — — — — — TSDMN[1:0]

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

Bit position:

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Bit field:

— — — TXDMN[4:0] — — — RXDMN[4:0]

Value after reset: 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1

Bit	Symbol	Function	R/W
4:0	RXDMN[4:0]	RX Descriptor Maximum Number Configures the maximum number of descriptors that can be processed at once for an RX queue (It limits to GWMDNC.RXDMN+1 the number of descriptors read that can be done for an RX frame). See GWEIS0.RXDNEs register explanation.	R/W
7:5	—	These bits are read as 0. The write value should be 0.	R/W
12:8	TXDMN[4:0]	TX Descriptor Maximum Number*1 Configures the maximum number of descriptors that can be processed at once for a TX queue (It limits to GWMDNC.TXDMN+1 the number of descriptors read that can be done for a TX frame). See GWEIS0.TXDNEs.	R/W
15:13	—	These bits are read as 0. The write value should be 0.	R/W
17:16	TSDMN[1:0]	Timestamp Descriptor Maximum Number Configures the maximum number of descriptors that can be processed at once for a TS queue (It limits to GWMDNC.TSDMN+1 the number of descriptors read that can be done for a timestamp). See GWEIS0.TSDNEs.	R/W
31:18	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: The switch maximum input frame size is 60 KB so this register should be set to a value smaller or equal to 30. To avoid any unexpected frame loss due to this register, any software sending frames bigger than 58 KB should avoid having more than one consecutive LINK/LINKFIX descriptor in a descriptor chain.

RXDMN[4:0] bits (RX Descriptor Maximum Number)

[Cautions]

SW: This maximum descriptor number takes in account all descriptors including LINK, LINKFIX descriptors and descriptors remaining from a previous error frame like FMID_EMPTY and FEND_EMPTY descriptors. It should be taken in account in account while setting this register.

TXDMN[4:0] bits (TX Descriptor Maximum Number)

[Cautions]

SW: This maximum descriptor number takes in account all descriptors including LINK, LINKFIX descriptors and descriptors remaining from a previous error frame like FMID and FEND descriptors. It should be taken in account while setting this register.

35.3.7.5 GWTRCi : Transmission Request Configuration Register i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0200 + 0x04 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj	TSRj
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	TSRj (j = 32 × i + b)	Transmission Start Request j (j = 0 to 63) ^{*1} This register is used to start transfer from CPU to switch for its corresponding TX queue. 0: TX Descriptor Chain is empty. 1: Transmission request is ongoing.	R/W ^{*2}

Note 1. HW: For RX queues, this bit cannot be set.

Note 2. Read value differs from written value.

TSRj bits (Transmission Start Request j (j = 0 to 63))

[Setting condition (only for TX queues)]

SW: Writing 1 to this bit will this bit set if queue t is a transmission queue (GWDCCi.DQT is set).

HW: a transmission request received through INTER_TX[t] interface will set this bit if queue t is a transmission queue (GWDCCi.DQT is set).

[Clearing condition]

HW: When HW read a descriptor other than FSINGLE, FSTART, FMID, FEND, LINK and LINKFIX. *

HW: When an AXI error occurs while reading a descriptor in the corresponding AXI descriptor queue. *

HW: When a descriptor number error occurs while reading the corresponding AXI descriptor queue. *

HW: Going out of OPERATION mode will clear this register.

35.3.7.6 GWTPCp : Transmission Pause Configuration Register p (p = 0 to 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0300 + 0x04 × p

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	PPPL7	PPPL6	PPPL5	PPPL4	PPPL3	PPPL2	PPPL1	PPPL0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	PPPL7 to PPPL0	Per Priority Pause Level i (The variable i corresponds to the bit position number.) 0: Transmission queues with their priority GWDCCi.DCP set to i are not paused when pause level p is set (PAUSE[2][p] is set [COMA]). 1: Transmission queues with the priority GWDCCi.DCP set to i are paused when pause level p is set (PAUSE[2][p] is set [COMA]).	R/W
31:8	—	These bits are read as 0. The write value should be 0.	R/W

35.3.7.7 GVARIRM : AXI RAM Initialization Register Monitoring Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0380

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	ARR	ARIOG
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	ARIOG	AXI RAM Initialization Ongoing [Setting condition] SW: By writing 1 to this register. It starts AXI RAM initialization. [Clearing condition] HW: This bit is cleared when AXI RAM initialization is finished. Min clk_period * 64 = [exp] (1000/320) * 128 = 400 ns	R/W ¹
1	ARR	AXI RAM Ready [Setting condition] When GVARIRM.ARIOG is getting cleared. [Clearing condition] By writing 1 to GVARIRM.ARIOG.	R
31:2	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.7.8 GWDCCi : Descriptor Chain i Configuration Register (i = 0 to 63)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0400 + 0x04 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	OSID[2:0]			—	—	—	BALR	—	—	—	—	—	DCP[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	DQT	SL	ETS	EDE	—	—	—	—	—	—	SM[1:0]	
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
1:0	SM[1:0]	Synchronization Mode 0 0: Normal mode (full descriptor write back) 0 1: No-write-back mode (no descriptor write back) 1 0: Keep-DT mode (no update of DT field at descriptor write back) 1 1: Reserved	R/W
7:2	—	These bits are read as 0. The write value should be 0.	R/W
8	EDE	Extended Descriptor Enable 0: Basic Descriptor 1: Extended Descriptor	R/W
9	ETS	Enable Timestamp Storage*1 0: Timestamp is not added in the Descriptor. 1: Timestamp is added in the Descriptor.	R/W
10	SL	Security Level*1 When a chain is secured, an unsecure descriptor cannot enter it (when a descriptor is received from MFWD with FDESCR.SEC equal to 0). 0: Chain i unsecure 1: Chain i secure	R/W
11	DQT	Descriptor Queue Type 0: Descriptor queue i is a reception queue. 1: Descriptor queue i is a transmission queue.	R/W
15:12	—	These bits are read as 0. The write value should be 0.	R/W
18:16	DCP[2:0]	Descriptor Chain Priority*2 0 0 0: Lowest priority 1 1 1: Highest priority	R/W*3
23:19	—	These bits are read as 0. The write value should be 0.	R/W
24	BALR	Base Address Load Request This bit is used to reset current_address field in the AXI address RAM to {{GWDCBAC.DCBAU, GWDCBAC.DCBAL} + I × 8} [Setting condition] SW: Writing 1 to this bit will set it. [Clearing condition] HW: This bit will be cleared when the current_address field in the AXI address RAM is reset.	R/W*3
27:25	—	These bits are read as 0. The write value should be 0.	R/W
30:28	OSID[2:0]	OS ID When a memory access is done for descriptor queue number i the OSID set in this register will be used.	R/W
31	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: This register is only valid for RX queues.

Note 2. • HW: This register is only valid for TX queues.

- SW: This register value should not be changed for a TX queue when GWTRCj.TSRj is set. (When transmission is ongoing for the corresponding queue)

Note 3. Read value differs from written value.

35.3.7.9 GWAARSS : AXI Address RAM Searching Setting Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0800

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	AARA[5:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
5:0	AARA[5:0]	AXI Address RAM Address* ¹ This register is used to read the current address used in AXI master for GWAARSS.AARA descriptor queue.	R/W
31:6	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: Due to the pipeline architecture of the AXI master, the AXI read can just give an idea of which address is processing the AXI master and is not precise enough to be used for HW/SW synchronization. This register is only here for debug purpose.

35.3.7.10 GWAARSR0 : AXI Address RAM Searching Result Register 0

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0804

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	AARS	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	ACARU[7:0]					
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	ACARU[7:0]	AXI Current Address Result Upper Part* ¹ Displays AXI address read value. [Update conditions] HW: GWAARSR0.AARS clear event.	R
30:8	—	These bits are read as 0.	R
31	AARS	AXI Address RAM Searching This register is used to read an in AXI address RAM. [Setting condition] SW: Writing GWAARSS will set this bit. [Clearing condition] HW: This bit will be deasserted when the AXI address RAM search is completed.	R

Note 1. AXI address read will not restart from the base address of chain after RESET mode (Because being in RESET mode will not clear this register).

35.3.7.11 GWAARSR1 : AXI Address RAM Searching Result Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0808

Bit position: 31

0

Bit field:

ACARL[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	ACARL[31:0]	AXI Current Address Result Lower Part*1 Displays AXI address read value. [Update conditions] HW: GWAARSR0.AARS clear event.	R

Note 1. AXI address read will not restart from the base address of chain after RESET mode (Because being in RESET mode will not clear this register).

35.3.7.12 GWIDAUASi : Incremental Data Area i Used Area Size Register (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0840 + 0x04 × i

Bit position: 31

23

0

Bit field:

IDAUS[23:0]

Value after reset: 0

Bit	Symbol	Function	R/W
23:0	IDAUS[23:0]	Incremental Data Area Used Size of RX Descriptor Chain i*1 Represents the byte numbers that have been used in the corresponding incremental area. [Clearing condition] HW: Being in RESET mode will clear this register. HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register gets cleared. [Increment conditions] HW: When a frame has been stored in the corresponding incremental area, this register is incremented by the frame size. [Decrement conditions] SW: By writing a value, SW will decrement this register by this value.	R/W*2
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: should always write a value smaller than the current register value.

Note 2. Read value differs from written value.

35.3.7.13 GWIDASMi : Incremental Data Area i Size Monitoring Register (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0880 + 0x04 × i

Bit position: 31

23

0

Bit field:

IDAS[23:0]

Value after reset: 0

Bit	Symbol	Function	R/W
23:0	IDAS[23:0]	Incremental Data Area Size of RX Descriptor Chain i This register indicates the size of RX descriptor chain i incremental area in bytes. [Clearing condition] HW: Being in RESET mode will clear this register. [Update conditions] HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register is updated to DESC.R.DS × 4096.	R
31:24	—	These bits are read as 0.	R

35.3.7.14 GWIDASAMi0 : Incremental Data Area i Start Address Monitoring Register 0 (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0900 + 0x08 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	IDASAU[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	IDASAU[7:0]	Incremental Data Area Start Address Upper Part of RX Descriptor Chain i [Clearing condition] HW: Being in RESET mode will clear this register. [Update conditions] HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register is updated to DESC.R.PTR[39:32].	R
31:8	—	These bits are read as 0.	R

35.3.7.15 GWIDASAMi1 : Incremental Data Area i Start Address Monitoring Register 1 (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0904 + 0x08 × i

Bit position:	31																																	0		
Bit field:	IDASAL[31:0]																																			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	IDASAL[31:0]	Incremental Data Area Start Address Lower Part of RX Descriptor Chain i [Clearing condition] HW: Being in RESET mode will clear this register. [Update conditions] HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register is updated to DESC.R.PTR[31:0].	R

35.3.7.16 GWIDACAMi0 : Incremental Data Area i Current Address Monitoring Register 0 (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0980 + 0x08 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	IDACAU[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	IDACAU[7:0]	Incremental Data Area Current Address Upper Part of RX Descriptor Chain i [Clear conditions] HW: Being in RESET mode will clear this register. [Update conditions] HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register is updated to DESC.PTR[39:32]. HW: When a burst is decided to be sent in the incremental area, {GWIDACAMi0.IDACAU, GWIDACAMi1.IDACAL} is incremented by the burst size. HW: {GWIDACAMi0.IDACAU, GWIDACAMi1.IDACAL} overflows at {GWIDASAMi0.IDASAU, GWIDASAMi1.IDASAL} + GWIDASMi.IDAS-1.	R
31:8	—	These bits are read as 0.	R

35.3.7.17 GWIDACAMi1 : Incremental Data Area i Current Address Monitoring Register 1 (i = 0 to 3)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0984 + 0x08 × i

Bit position:	31	0																																
Bit field:	IDACAL[31:0]																																	
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	IDACAL[31:0]	Incremental Data Area Current Address Lower Part of RX Descriptor Chain i [Clear conditions] HW: Being in RESET mode will clear this register. [Update conditions] HW: When a FEMPTY_IS descriptor is read in the corresponding descriptor queue, this register is updated to DESC.PTR[31:0]. HW: When a burst is decided to be sent in the incremental area, {GWIDACAMi0.IDACAU, GWIDACAMi1.IDACAL} is incremented by the burst size. HW: {GWIDACAMi0.IDACAU, GWIDACAMi1.IDACAL} overflows at {GWIDASAMi0.IDASAU, GWIDASAMi1.IDASAL} + GWIDASMi.IDAS-1.	R

35.3.8 Rate Limiter Function Registers

35.3.8.1 GWGRLC : Global Rate Limiter Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0A00

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	GRLULRS	GRLE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	GRLIV[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	GRLIV[15:0]	Global Rate Limiter Incremental Value For further setting information see section 35.5.2.1.2. Rate Limiters . [Cautions] SW: Setting this register to 0 or a value close to 0 would set GWCA to a very low throughput or could stuck GWCA in transmission.	R/W
16	GRLE	Global Rate Limiter Enable The global rate limiter limits the throughput through all TX queues to avoid the switch to take in too much data at once. 0: Global Rate limiter disabled 1: Global Rate limiter enabled	R/W
17	GRLULRS	Global Rate Limiter Upper Limit Reached Status Flag If this bit gets set when only the global rate limiter is enabled (per port rate limiters disabled), it means that the AXI master is too slow to provide the throughput set in the global rate limiter or that the AXI latency has been underestimated. This bit can get set while using per port rate limiters. In this case it should be ignored. [Setting condition] HW: When the global rate limit reaches its upper limit GWGRLULC.GRLUL, this bit gets set. [Clearing condition] HW: Being in RESET mode will clear this register. SW: Writing one to this bit will clear it.	R/W ¹
31:18	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.8.2 GWGRLULC : Global Rate Limiter Upper Limit Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0A04

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	GRLUL[23:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	GRLUL[23:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
23:0	GRLUL[23:0]	Global Rate Limiter Upper Limit* ¹ Set the maximum number of credits in bit rate limiter that can accumulate before stopping. For further setting information see section 35.5.2.1.2. Rate Limiters .	R/W
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: This register should not be set to a value bigger than 0x800000.

35.3.8.3 GWRLCi : Rate Limiter i Configuration Register (i = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0A80 + 0x8 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	RLE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	RLIV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
11:0	RLIV[11:0]	Rate Limiter Incremental Value For further setting information see section 35.5.2.1.2. Rate Limiters . [Cautions] SW: Setting this register to 0 or a value close to 0 would set corresponding queue to a very low throughput or could stuck the corresponding queue.	R/W
15:12	—	These bits are read as 0. The write value should be 0.	R/W
16	RLE	Rate Limiter Enable Rate limiter i limit the through put for AXI descriptor queue number 63-I. For details, see section 35.5.1.1. AXI Descriptor Queue Mapping . 0: Rate limiter i disabled 1: Rate limiter i enabled	R/W
31:17	—	These bits are read as 0. The write value should be 0.	R/W

35.3.8.4 GWRLULCi : Rate Limiter i Upper Limit Configuration Register (i = 0 to 7)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0A84 + 0x8 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	RLUL[23:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	RLUL[23:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
23:0	RLUL[23:0]	Rate Limiter Upper Limit*1 Set the maximum number of credits in bit rate limiter i can accumulate before stopping. This register can be used if a lot of traffic happens on the AXI bus so burst on queues i make the switch overflow. If not required, set this register to 0x80_0000. For further setting information see section 35.5.2.1.2. Rate Limiters .	R/W
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. SW: This register should not be set to a value bigger than 0x80_0000.

35.3.9 Interrupt Function Registers

35.3.9.1 GWIDPC : Interrupt Delay Prescaler Configuration Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0B80

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	IDPV[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	IDPV[9:0]	Interrupt Delay Prescaler Value This register is used to create an internal clock. This internal clock period will be $\text{clk_delay_period} = \text{clk_period} * (\text{GWIDPC.IDPV} + 1)$ where clk_period is the period of clk . For further information, see section 35.5.6. Interrupt Delay Function .	R/W
31:10	—	These bits are read as 0. The write value should be 0.	R/W

35.3.9.2 GWIDCi : Interrupt Delay i Configuration Register (i = 0 to 63)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x0C000 + 0x4 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	IDV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
11:0	IDV[11:0]	Interrupt Delay Value Configures the delay between the interrupt status flag GWDISI.DiSt setting and the actual interrupt event in us. If this register is set to 0, data interrupt will occur without delay. If value X is set to this register, the interrupt will be delayed by a time between $X-1 \times \text{clk_delay_period}$ us and $X \times \text{clk_delay_period}$ us where clk_delay_period is the internal clock created using GWIDPC register period. For further information, see section 35.5.6. Interrupt Delay Function .	R/W
31:12	—	These bits are read as 0. The write value should be 0.	R/W

35.3.10 GWCA Counter Registers

35.3.10.1 GWRDCN : Received Data Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1000

Bit position: 31

0

Bit field:

RDN[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	RDN[31:0]	Received Data Number This register counts data that has been received by GWCA (data aborted by AXI error are accounted here).	R

RDN[31:0] bits (Received Data Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a frame is passed from multicast control to AXI master interface and if this register has a value other than 32.

35.3.10.2 GWTDCN : Transmitted Data Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1004

Bit position: 31

0

Bit field:

TDN[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TDN[31:0]	Transmitted Data Number This register counts data that has been transmit by GWCA (this includes data transmitted with an error and frames smaller than 32 bytes).	R

TDN[31:0] bits (Transmitted Data Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a frame is passed from AXI master interface to TX data store and if this register has a value different than 32.

35.3.10.3 GWTSCN : Timestamp Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1008

Bit position: 31

0

Bit field:

TN[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TN[31:0]	Timestamp Number This register counts timestamp processed by GWCA (TS lost because of AXI errors are accounted here).	R

TN[31:0] bits (Timestamp Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Timestamp is saved in timestamp RAM and if this register has a value other than 32.

35.3.10.4 GWTSOVFECN : Timestamp Overflow Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x100C

Bit position:

31

30

29

28

27

26

25

24

23

22

21

20

19

18

17

16

Bit field:

—

—

—

—

—

—

—

—

—

—

—

—

—

—

—

—

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit position:

15

14

13

12

11

10

9

8

7

6

5

4

3

2

1

0

Bit field:

TSOVFEN[15:0]

Value after reset: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Bit	Symbol	Function	R/W
15:0	TSOVFEN[15:0]	Timestamp Overflow Error Number This register counts the number of timestamps lost because of timestamp RAM overflow.	R
31:16	—	These bits are read as 0.	R

TSOVFEN[15:0] bits (Timestamp Overflow Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a timestamp overflow error happens and if this register has a value other than 16.

35.3.10.5 GWUSMFSECN : Under Minimum Frame Size Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1010

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	USMFSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	USMFSEN[15:0]	Under Switch Minimum Frame Size Error Number This register counts the number of transmit data lost because of under switch minimum size error.	R
31:16	—	These bits are read as 0.	R

USMFSEN[15:0] bits (Under Switch Minimum Frame Size Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when an Under Switch Minimum Frame Size Error happens and if this register has a value other than 16.

35.3.10.6 GWTFECN : TAG Filtering Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1014

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TFEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TFEN[15:0]	TAG Filtering Error Number This register counts the number of transmit data lost because of TAG filtering.	R
31:16	—	These bits are read as 0.	R

TFEN[15:0] bits (TAG Filtering Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a TAG filter error happens and if this register has a value different than 16.

35.3.10.7 GWSEQECN : Sequence Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1018

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	SEQEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	SEQEN[15:0]	Sequence Error Number This register counts the number of transmit data lost because of a sequence error.	R
31:16	—	These bits are read as 0.	R

SEQEN[15:0] bits (Sequence Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a sequence error happens and if this register has a value different than 16.

35.3.10.8 GWTXDNECN : TX Descriptor Number Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1020

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TXDNEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TXDNEN[15:0]	TX Descriptor Number Error Number This register counts the number of transmit data lost because of a TX Descriptor Number Error.	R
31:16	—	These bits are read as 0.	R

TXDNEN[15:0] bits (TX Descriptor Number Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a TX Descriptor Number Error happens and if this register has a value other than 16.

35.3.10.9 GWFSECN : Frame Size Error Counter Register

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x1024

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	FSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	FSEN[15:0]	Frame Size Error Number This register counts the number of receive data lost because of a Frame Size Error.	R
31:16	—	These bits are read as 0.	R

FSEN[15:0] bits (Frame Size Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Frame Size Error happens and if this register has a value different than 16.

35.3.10.10 GWTD FECN : Timestamp Descriptor Full Error Counter Register

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x1028

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TDFEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TDFEN[15:0]	Timestamp Descriptor Full Error Number This register counts the number of timestamps lost because of a Timestamp Descriptor Full Error.	R
31:16	—	These bits are read as 0.	R

TDFEN[15:0] bits (Timestamp Descriptor Full Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Timestamp Descriptor Full Error happens and if this register has a value other than 16.

35.3.10.11 GWTSDNECN : Timestamp Descriptor Number Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x102C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TSDNEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TSDNEN[15:0]	Timestamp Descriptor Number Error Number This register counts the number of timestamps loss because of a Timestamp Descriptor Number Error.	R
31:16	—	These bits are read as 0.	R

TSDNEN[15:0] bits (Timestamp Descriptor Number Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Timestamp Descriptor Number Error happens and if this register has a value other than 16.

35.3.10.12 GWDQOECN : Descriptor Queue Overflow Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1030

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DQOEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DQOEN[15:0]	Descriptor Queue Overflow Error Number This register counts the number of receive data lost because of a Descriptor Queue Overflow Error.	R
31:16	—	These bits are read as 0.	R

DQOEN[15:0] bits (Descriptor Queue Overflow Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Descriptor Queue Overflow Error happens and if this register has a value other than 16.

35.3.10.13 GWDQSECN : Descriptor Queue Security Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1034

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DQSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DQSEN[15:0]	Descriptor Queue Security Error Number This register counts the number of receive data lost because of a Descriptor Queue Security Error.	R
31:16	—	These bits are read as 0.	R

DQSEN[15:0] bits (Descriptor Queue Security Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Descriptor Queue Security Error happens and if this register has a value other than 16.

35.3.10.14 GWDFECN : Descriptor Full Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1038

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DFEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DFEN[15:0]	Descriptor Full Error Number This register counts the number of receive data lost because of a Descriptor Full Error.	R
31:16	—	These bits are read as 0.	R

DFEN[15:0] bits (Descriptor Full Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Descriptor Full Error happens and if this register has a value other than 16.

35.3.10.15 GWDSECN : Descriptor Security Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x103C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DSEN[15:0]	Descriptor Security Error Number This register counts the number of receive data lost because of a Descriptor Security Error.	R
31:16	—	These bits are read as 0.	R

DSEN[15:0] bits (Descriptor Security Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Descriptor Security Error happens and if this register has a value other than 16.

35.3.10.16 GWDSZECN : Data Size Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1040

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DSZEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DSZEN[15:0]	Data Size Error Number This register counts the number of receive data lost because of a Data Size Error.	R
31:16	—	These bits are read as 0.	R

DSZEN[15:0] bits (Data Size Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Data Size Error happens and if this register has a value other than 16.

35.3.10.17 GWDCTECN : Descriptor Chain Type Error Counter Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1044

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DCTEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DCTEN[15:0]	Descriptor Chain Type Error Number This register counts the number of receive data lost because of a Descriptor Chain Type Error.	R
31:16	—	These bits are read as 0.	R

DCTEN[15:0] bits (Descriptor Chain Type Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a Descriptor Chain Type Error happens and if this register has a value different than 16.

35.3.10.18 GWRXDNECN : RX Descriptor Number Error Counter Register

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x1048

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	RXDNEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	RXDNEN[15:0]	RX Descriptor Number Error Number This register counts the number of receive data lost because of a RX Descriptor Number Error.	R
31:16	—	These bits are read as 0.	R

RXDNEN[15:0] bits (RX Descriptor Number Error Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Reading this register clears it.

[Increment conditions]

HW: Incremented by 1 when a RX Descriptor Number Error happens and if this register has a value different than 16.

35.3.11 GWCA Interrupt Registers**35.3.11.1 GWDISi : Data Interrupt Status Register i (i = 0, 1)**

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x1100 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi	DISi
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DlSt (t = 32 × i + b)	Data t Interrupt Status Flag (t = 0 to 63)	R/W ^{*1}

Note 1. Read value differs from written value.

DlSt bits (Data t Interrupt Status Flag (t = 0 to 63))

[Setting condition TX]

HW: A descriptor has been processed in the corresponding queue and DESCR.DIE was set in the corresponding read descriptor. [About set timing information for normal TX] A descriptor is considered as processed when written back. All descriptor which are not written back are considered as processed at the timing it should have been written back so that SW can know that all the previous at the actual descriptor has been fully processed.

[Setting condition RX]

HW: A frame has been processed in the corresponding queue and DESCR.DIE was set in at least one of the corresponding read descriptors. [About set timing information for normal RX] A frame is considered as processed when frame FSTART or FSINGLE descriptor is processed. A descriptor is considered as processed when written back. All descriptor which are not written back are considered as processed at the timing it should have been written back so that SW can know that all the previous at the actual descriptor has been fully processed. All unknown descriptors are never considered as processed (See GWEIS2i.DFEST).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

35.3.11.2 GWDIEi : Data Interrupt Enable Register i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1104 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt	DIEt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DIEt (t = 32 × i + b)	Data t Interrupt Enable (t = 0 to 63) [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWDIDi.DIDt register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}

Note 1. Read value differs from written value.

35.3.11.3 GWDIDi : Data Interrupt Disable Register i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1108 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt	DIDt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DIDt (t = 32 × i + b)	Data t Interrupt Disable (t = 0 to 63) Writing 1 to this bit will clear GWDIEi.DIEt register.	R

35.3.11.4 GWDIDSi : Data Interrupt Delayed Status Register i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x110C + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt	DIDSt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DIDSt (t = 32 × i + b)	Data t Interrupt Delayed Status Flag (t = 0 to 63) Displays the interrupt GWDISi.DISt delayed by interrupt delay function.	R

35.3.11.5 GWTSDIS : Timestamp Data Interrupt Status Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1180

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	TSDIS	TSDIS
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Bit	Symbol	Function	R/W
1:0	TSDIS1 to TSDIS0	Timestamp Data i Interrupt Status Flag (The variable i corresponds to the bit position number.)	R/W ^{*1}
31:2	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

TSDISi bits (Timestamp Data i Interrupt Status Flag)

[Setting condition]

HW: A descriptor is processed in the corresponding queue and DESCR.DIE was set in the corresponding read descriptor. A descriptor is considered as processed when written back. All descriptor which are not written back are considered as processed at the timing it should have been written back so that SW can know that all the previous at the actual descriptor has been fully processed. All unknown descriptors are never considered as processed (See GWEIS0.TDFES).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

35.3.11.6 GWTSDIE : Timestamp Data Interrupt Enable Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1184

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	TSDIE 1	TSDIE 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
1:0	TSDIE1 to TSDIE0	Timestamp Data i Interrupt Enable (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWTSDID.TSDIDs register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
31:2	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.11.7 GWTSDID : Timestamp Data Interrupt Disable Register

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1188

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	TSDID 1	TSDID 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
1:0	TSDID1 to TSDID0	Timestamp Data i Interrupt Disable (The variable i corresponds to the bit position number.) Writing 1 to this bit will clear GWTSDIE.TSDIEs register.	R
31:2	—	These bits are read as 0. The write value should be 0.	R

35.3.11.8 GWEIS0 : Error Interrupt Status Register 0

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1190

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	TSDN ES1	TSDN ES0	—	—	TDFE S1	TDFE S0	FSES7	FSES6	FSES5	FSES4	FSES3	FSES2	FSES1	FSES0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TSHE S	TXDN ES	—	SEQE S	TFES	USMF SES	TSOV FES	—	—	—	—	—	—	—	—	AES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	AES	AXI Error Status Flag	R/W ^{*1}
8:1	—	These bits are read as 0. The write value should be 0.	R/W
9	TSOVFES	Timestamp Overflow Error Status Flag	R/W ^{*1}
10	USMFSES	Under Switch Minimum Frame Size Error Status Flag	R/W ^{*1}
11	TFES	TAG Filtering Error Status Flag	R/W ^{*1}
12	SEQES	Sequence Error Status Flag ^{*2}	R/W ^{*1}
13	—	This bit is read as 0. The write value should be 0.	R/W
14	TXDNES	TX Descriptor Number Error Status Flag ^{*2}	R/W ^{*1}
15	TSHES	Timestamp Hardware Error Status Flag	R/W ^{*1}
23:16	FSES7 to FSES0	Frame Size Error Status Flag i (The variable i corresponds to the bit position number.)	R/W ^{*1}
25:24	TDFES1 to TDFES0	Timestamp Descriptor i Full Error Status Flag (The variable i corresponds to the bit position number.)	R/W ^{*1}
27:26	—	These bits are read as 0. The write value should be 0.	R/W

Bit	Symbol	Function	R/W
29:28	TSDNES1 to TSDNES0	Timestamp Descriptor i Number Error Status Flag (The variable i corresponds to the bit position number.)	R/W ^{*1}
31:30	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

Note 2. HW: This flag is only available for TX descriptor queues.

AES bit (AXI Error Status Flag)

[Setting condition]

HW: When an error is detected on an AXI access (xRESP != 00)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: For reception, if an AXI error happens during descriptor read before a frame/timestamp transfer started, the frame/timestamp is rejected.

HW: For reception, if an AXI error happens during descriptor read after a frame transfer started, the remaining part of the frame is rejected and DESC.RXIE is set in the frame corresponding first written back data descriptor, which is FSTART in normal synchronization mode (for GWDCCi.SM == 00).

HW: For reception, if an AXI error happens during data write, DESC.RXIE is set in the frame corresponding first written back data descriptor, which is FSTART or a FSINGLE descriptor in normal synchronization mode (for GWDCCi.SM == 00).

HW: For reception, if an AXI error happens during RX/TS descriptor write, nothing happens.

HW: For transmission, if an AXI error happens during descriptor read and no frame is ongoing, corresponding GWTRCi.TSRj bit is cleared and no frame is transmitted.

HW: For transmission, if an AXI error happens during descriptor read and a frame is ongoing, the ongoing frame is padded with 0, sent to MFWD with AXE error set ((3) SAEF format) and corresponding GWTRCi.TSRj bit is cleared.

HW: For transmission, if an AXI error happens during data read, the ongoing frame is sent to MFWD with AXE error set ((3) SAEF format).

HW: For transmission, if an AXI error happens during descriptor write, nothing happens.

SW: System reset.

TSOVFES bit (Timestamp Overflow Error Status Flag)

[Setting condition]

HW: When a timestamp is received when the timestamp RAM is full.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: A timestamp is lost.

SW: Nothing if it is acceptable to lose a timestamp else switch reset.

USMFSES bit (Under Switch Minimum Frame Size Error Status Flag)

[Setting condition]

HW: A frame smaller than 32 bytes has been received from CPU for transmission.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: The frame is discarded.

SW: Software error, software should be reviewed.

TFES bit (TAG Filtering Error Status Flag)

[Setting condition]

HW: An unauthorized TAG format has been detected. (See [section 35.5.2.3.2. VLAN Tagging](#)).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: Frame is sent to MFWD with local descriptor DESCR.TFE bit set (See [section 35.5.2.3.3. Frame Saving in Local RAM](#)).

SW: Software error, software should be reviewed.

SEQES bit (Sequence Error Status Flag)

[Setting condition]

HW: A FMID, FEND has been received before a frame transfer started.

HW: A descriptor different than FMID, FEND, LINK and LINKFIX has been received after a frame transfer started.

HW: An FSTART, FMID, FEND or FSINGLE descriptor has been received with DESCR.DS set to 0.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: If a SEQUENCE Error happens before a frame transfer started, nothing happens.

HW: If a SEQUENCE Error happens after a frame transfer started, The ongoing frame is padded with 0 and sent to MFWD with SEQE error set ((3) SAEF format).

SW: Software error, software should be reviewed.

TXDNES bit (TX Descriptor Number Error Status Flag)

[Setting condition]

HW: GWMDNC.TXDMN+1 descriptor has already been used to process one frame and the frame processing is not finished.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: If a TX Descriptor Number Error happens before a frame transfer started, GWTRCi.TSRj bit is cleared.

HW: If an TX Descriptor Number Error happens after a frame transfer started, the ongoing frame is padded with 0, sent to MFWD with DNE error set ((3) SAEF format) and corresponding GWTRCi.TSRj bit is cleared.

SW: Software error, software should be reviewed.

TSHES bit (Timestamp Hardware Error Status Flag)

[Setting condition]

HW: This error happens when timestamps are coming too fast to GWCA to be stored in time, so timestamp are lost even if the timestamp RAM is not full. This error should never happen and tells that the number of captured timestamps is too high or the bus clock clk has a too low frequency.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: timestamp lost.

SW: No recovery, this error should not happen.

FSESi bits (Frame Size Error Status Flag i)

[Setting condition]

HW: Bit q of this register is set when a frame bigger than GWRMFSC.MFS has been received for descriptor queue q.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: The current frame will be lost. Following data will be processed normally.

SW: System dependent, cannot be defined here.

TDFESi bits (Timestamp Descriptor i Full Error Status Flag)

[Setting condition]

HW: Bit i sets when a timestamp has been received for descriptor chain i which is full (read any other descriptor than a FEMPTY_ND, LINKFIX or LINK descriptor).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: Timestamp lost.

SW: Nothing if it is acceptable to lose a timestamp else switch reset.

SW: AXI is too slow to save all timestamps in the CPU RAM. Less timestamps should be captured or more bandwidth on AXI should be reserved for ESWM.

TSDNESi bits (Timestamp Descriptor i Number Error Status Flag)

[Setting condition]

HW: Bit i sets when GWMDNC.TSDMN+1 descriptor has already been used to process one timestamp for descriptor chain i and the timestamp processing is not finished.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: Timestamp lost.

SW: Software error, software should be reviewed.

35.3.11.9 GWEIE0 : Error Interrupt Enable Register 0

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x1194

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	TSDN EE1	TSDN EE0	—	—	TDFE E1	TDFE E0	FSEE7	FSEE6	FSEE5	FSEE4	FSEE3	FSEE2	FSEE1	FSEE0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TSHE E	TXDN EE	—	SEQE E	TFEE	USMF SEE	TSOV FEE	—	—	—	—	—	—	—	—	AEE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	AEE	AXI Error Enable [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWEID0.AED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
8:1	—	These bits are read as 0. The write value should be 0.	R/W
9	TSOVFEE	Timestamp Overflow Error Enable [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWEID0.TSOVFED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
10	USMFSEE	Under Minimum Frame Size Error Enable [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWEID0.USMFSED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
11	TFEE	TAG Filtering Error Enable [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWEID0.TFED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
12	SEQEE	Sequence Error Enable [Setting condition] Writing 1 to this bit will set it. [Clearing condition] Writing 1 to GWEID0.SEQED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
13	—	This bit is read as 0. The write value should be 0.	R/W
14	TXDNEE	TX Descriptor Number Error Enable [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to GWEID0.TXDNEED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}

Bit	Symbol	Function	R/W
15	TSHEE	Timestamp Hardware Error Enable [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to GWEID0.TSHED register will clear this bit. 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
23:16	FSEE7 to FSEE0	Frame Size Error Enable i (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to one of GWEID0.FSED bits will clear the corresponding bit in this register. 0: Interrupt disabled for error q 1: Interrupt enabled for error q	R/W ^{*1}
25:24	TDFEE1 to TDFEE0	Timestamp Descriptor i Full Error Enable (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to one of GWEID0.TDFEID bits will clear the corresponding bit in this register. 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
27:26	—	These bits are read as 0. The write value should be 0.	R/W
29:28	TSDNEE1 to TSDNEE0	Timestamp Descriptor i Number Error Enable (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to one of GWEID0.TSDNED bits will clear the corresponding bit in this register. 0: Interrupt disabled 1: Interrupt enabled	R/W ^{*1}
31:30	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.11.10 GWEID0 : Error Interrupt Disable Register 0

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1198

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	TSDN ED1	TSDN ED0	—	—	TDFE D1	TDFE D0	FSED 7	FSED 6	FSED 5	FSED 4	FSED 3	FSED 2	FSED 1	FSED 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TSHE D	TXDN ED	—	SEQE D	TFED	USMF SED	TSOV FED	—	—	—	—	—	—	—	—	AED
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	AED	AXI Error Disable Writing 1 to this bit will clear GWEIE0.AEE register.	R
8:1	—	These bits are read as 0.	R
9	TSOVFED	Timestamp Overflow Error Disable Writing 1 to this bit will clear GWEIE0.TSOVFEE register.	R
10	USMFSED	Under Minimum Frame Size Error Disable Writing 1 to this bit will clear GWEIE0.USMFSEE register.	R

Bit	Symbol	Function	R/W
11	TFED	TAG Filtering Error Disable Writing 1 to this bit will clear GWEIE0.TFEE register.	R
12	SEQED	Sequence Error Disable Writing 1 to this bit will clear GWEIE0.SEQEE register.	R
13	—	This bit is read as 0.	R
14	TXDNED	TX Descriptor Number Error Disable Writing 1 to this bit will clear GWEIE0.TXDNEE register.	R
15	TSHED	Timestamp Hardware Full Error Disable Writing 1 to this bit will clear GWEIE0.TSHEE register.	R
23:16	FSED7 to FSED0	Frame Size Error Disable i (The variable i corresponds to the bit position number.) Writing 1 to one of these bits will clear the corresponding bit in GWEIE0.FSEE register.	R
25:24	TDFED1 to TDFED0	Timestamp Descriptor i Full Error Disable (The variable i corresponds to the bit position number.) Writing 1 to one of these bits will clear the corresponding bit in GWEIE0.TDFEIE register.	R
27:26	—	These bits are read as 0.	R
29:28	TSDNED1 to TSDNED0	Timestamp Descriptor i Number Error Disable (The variable i corresponds to the bit position number.) Writing 1 to one of these bits will clear the corresponding bit in GWEIE0.TSDNED register.	R
31:30	—	These bits are read as 0.	R

35.3.11.11 GWEIS1 : Error Interrupt Status Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x11A0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSE S7	DQSE S6	DQSE S5	DQSE S4	DQSE S3	DQSE S2	DQSE S1	DQSE S0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	DQOE S7	DQOE S6	DQOE S5	DQOE S4	DQOE S3	DQOE S2	DQOE S1	DQOE S0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOES7 to DQOES0	Descriptor Queue i Overflow Error Status Flag (The variable i corresponds to the bit position number.)	R/W ^{*1}
15:8	—	These bits are read as 0. The write value should be 0.	R/W
23:16	DQSES7 to DQSES0	Descriptor Queue i Security Error Status Flag (The variable i corresponds to the bit position number.)	R/W ^{*1}
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

DQOESi bits (Descriptor Queue i Overflow Error Status Flag)

[Setting condition]

HW: Bit q of this register is set when a descriptor is received for queue q when it is full (GWRDQDCq.DQD == GWRDQMq.DNQ), not disabled (GWRDQC.RDQDi is not set) and GWCA is not going out or out of OPERATION mode (GWMC.OPC == 11).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: Any descriptor received for a full Descriptor queue will not be accepted by GWCA and will not be forwarded to CPU.

SW: Too many descriptors are received for the corresponding queue. GWRDQDCq.DQD setting should be reviewed.

DQSESi bits (Descriptor Queue i Security Error Status Flag)

[Setting condition]

HW: Bit q of this register is set when a non-secure descriptor is received (FDESCR.SEC is not set [FWD]) and queue q is a secure queue (GWRDQSC.RDQSLn is set)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: Any non-secure descriptor received for secure Descriptor queue will not be accepted by GWCA and will not be forwarded to CPU.

SW: System dependent, cannot be defined here.

35.3.11.12 GWEIE1 : Error Interrupt Enable Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x11A4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSE E7	DQSE E6	DQSE E5	DQSE E4	DQSE E3	DQSE E2	DQSE E1	DQSE E0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	DQOE E7	DQOE E6	DQOE E5	DQOE E4	DQOE E3	DQOE E2	DQOE E1	DQOE E0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOEE7 to DQOEE0	Descriptor Queue i Overflow Error Enable (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to bit q in GWEID1.DQOED register will clear the bit q in this register. 0: Interrupt disabled for error q 1: Interrupt enabled for error q	R/W ¹
15:8	—	These bits are read as 0. The write value should be 0.	R/W
23:16	DQSEE7 to DQSEE0	Descriptor Queue i Security Error Enable (The variable i corresponds to the bit position number.) [Setting condition] Writing 1 to one of these bits will set it. [Clearing condition] Writing 1 to bit q in GWEID1.DQSED register will clear the bit q in this register. 0: Interrupt disabled for error q 1: Interrupt enabled for error q	R/W ¹
31:24	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

35.3.11.13 GWEID1 : Error Interrupt Disable Register 1

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x11A8

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSE D7	DQSE D6	DQSE D5	DQSE D4	DQSE D3	DQSE D2	DQSE D1	DQSE D0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	DQOE D7	DQOE D6	DQOE D5	DQOE D4	DQOE D3	DQOE D2	DQOE D1	DQOE D0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOED7 to DQOED0	Descriptor Queue i Overflow Error Disable (The variable i corresponds to the bit position number.) Writing 1 to bit q in this register will clear bit q in GWEIE1.DQOEE register.	R
15:8	—	These bits are read as 0.	R
23:16	DQSED7 to DQSED0	Descriptor Queue i Security Error Disable (The variable i corresponds to the bit position number.) Writing 1 to bit q in this register will clear bit q in GWEIE1.DQSEE register.	R
31:24	—	These bits are read as 0.	R

35.3.11.14 GWEIS2i : Error Interrupt Status Register 2i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1200 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST	DFEST
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DFEST (t = 32 × i + b)	Descriptor t Full Error Status Flag (t = 0 to 63) ^{*1}	R/W ^{*2}

Note 1. HW: This register is only valid for RX queues.

Note 2. Read value differs from written value.

DFEST bits (Descriptor t Full Error Status Flag (t = 0 to 63))

[Setting condition]

HW: When a frame has been received for descriptor queue t which is full (read a descriptor other than FEMPTY, FEMPTY_IS, FEMPTY_IC, FEMPTY_ND, FEMPTY_START, FEMPTY_MID, FEMPTY_END, LINKFIX and LINK). This flag is set at the timing the frame last descriptor write-back should have been completed.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: set of the DESC.RERR bit in RX descriptor if a part of the frame has been already sent.

HW: Current frame or remaining part of the frame will be discarded, and next frames will be processed normally (as long as software does not insert new descriptors for reception the queue will continue overflowing).

SW: Discard data which have DESC.RERR bit set.

35.3.11.15 GWEIE2i : Error Interrupt Enable Register 2i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1204 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt	DFEEt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DFEEt (t = 32 × i + b)	Descriptor t Full Error Enable (t = 0 to 63) 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}

Note 1. Read value differs from written value.

DFEEt bits (Descriptor t Full Error Enable (t = 0 to 63))

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID2i.DFEDt register will clear this bit.

35.3.11.16 GWEID2i : Error Interrupt Disable Register 2i (i = 0, 1)

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1208 + 0x10 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt	DFEDt
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
b (b = 0 to 31)	DFEDt (t = 32 × i + b)	Descriptor t Full Error Disable (t = 0 to 63) Writing 1 to this bit will clear GWEIE2i.DFEET register.	R

35.3.11.17 GWEIS3 : Error Interrupt Status Register 3

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1280

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	IAOES 3	IAOES 2	IAOES 1	IAOES 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	IAOES3 to IAOES0	Incremental Area i Overflow Error Status Flag* ¹ (The variable i corresponds to the bit position number.)	R/W* ²
31:4	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. HW: This register is only valid for RX incremental queues.

Note 2. Read value differs from written value.

IAOESi bits (Incremental Area i Overflow Error Status Flag)

[Setting condition]

HW: When incremental area i overflows, for further information, see [section 35.5.3.5.3. Single Page Incremental Data Reception](#). This flag is only set when GWIDAUASi.IDAUAS crosses GWIDASMi.IDAS value (if incremental area is already overflowing, this flag will not be set again except if GWIDAUASi.IDAUAS overflows).

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: The data coming for this incremental area will still be written.

SW: Set a new incremental area.

35.3.11.18 GWEIE3 : Error Interrupt Enable Register 3

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1284

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	IAOEE 3	IAOEE 2	IAOEE 1	IAOEE 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	IAOEE3 to IAOEE0	Incremental Area i Overflow Error Enable (The variable i corresponds to the bit position number.) 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
31:4	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

IAOEEi bits (Incremental Area i Overflow Error Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID3.IAOEDi register will clear this bit.

35.3.11.19 GWEID3 : Error Interrupt Disable Register 3

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1288

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	IAOED 3	IAOED 2	IAOED 1	IAOED 0
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	IAOED3 to IAOED0	Incremental Area i Overflow Error Disable (The variable i corresponds to the bit position number.) Writing 1 to this bit will clear GWEIE3.IAOEDi register.	R
31:4	—	These bits are read as 0.	R

35.3.11.20 GWEIS4 : Error Interrupt Status Register 4

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1290

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	DSECN[5:0]						—	—	—	—	—	—	DSEIOS	DSES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	DSSECN[5:0]						—	—	—	—	—	—	DSSEIOS	DSSES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DSES	Descriptor Security Error Status Flag	R/W ^{*1}
1	DSSEIOS	Descriptor Security Error Interrupt Overflow Status Flag ^{*2}	R/W ^{*1}
7:2	—	These bits are read as 0. The write value should be 0.	R/W
13:8	DSSECN[5:0]	Descriptor Security Error Chain Number ^{*2}	R
15:14	—	These bits are read as 0. The write value should be 0.	R/W
16	DSES	Data Size Error Status Flag ^{*2}	R/W ^{*1}
17	DSEIOS	Data Size Error Interrupt Overflow Status Flag ^{*2}	R/W ^{*1}
23:18	—	These bits are read as 0. The write value should be 0.	R/W
29:24	DSECN[5:0]	Data Size Error Chain Number ^{*2}	R
31:30	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

Note 2. HW: This register is only valid for RX queues.

DSES bit (Descriptor Security Error Status Flag)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Setting condition]

HW: During reception, when an unsecure descriptor has been received for a secure queue (GWDCCi.SL is set).

[Error recovery]

HW: During reception, the current frame will be lost. Following data will be processed normally.

SW: Secure CPU should read the unsecure forwarding engine registers to find which entry is trying to use a secure queue for unsecure traffic and suppress it.

SW: Secure CPU could block access to switch from unsecure CPU.

DSSEIOS bit (Descriptor Security Error Interrupt Overflow Status Flag)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Setting condition]

When a Descriptor Security Error occurs and GWEIS4.DSES is already set to one. In this case, the descriptor chain number in which the new error occurred is lost.

[Error recovery]

SW: Switch reset.

DSSECN[5:0] bits (Descriptor Security Error Chain Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

[Update conditions]

This field is update to the descriptor chain number in which a Descriptor Security Error occurred when GWEIS4.DSSES is being set.

DSES bit (Data Size Error Status Flag)

[Setting condition]

HW: When a frame has been received with an unexpected size, for more information see [section 35.5.3.5.2. Size-controlled Data Reception](#). This flag is set after the frame last descriptor write back is completed.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

HW: set of DESC.DSE bit in FSTART RX descriptor.

HW: If the frame was an undersized frame the frame is written with the normal descriptor sequence (FSINGLE or FSTART/ FEND or FSTART/n*FMID/ FEND)

HW: If the frame was an undersized frame, when a new frame comes for the corresponding queue, the AXI master will skip all following descriptors until an empty descriptor which is not FEMPTY_MID or FEMPTY_END (at start of a frame, FEMPTY_MID and FEMPTY_END descriptors are always ignored).

If the frame is an oversized frame, the frame will be truncated.

SW: Discard data which have DESC.DSE bit set.

SW: Because DESC.DSE is set in RX descriptors with a size error, it is not mandatory to read this flag for error recovery.

DSEIOS bit (Data Size Error Interrupt Overflow Status Flag)

[Setting condition]

When a Data Size Error occurs and GWEIS4.DSES is already set to one. In this case, the descriptor chain number in which the new error occurred is lost.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Error recovery]

SW: Switch reset.

DSECN[5:0] bits (Data Size Error Chain Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

[Update conditions]

This field is update to the descriptor chain number in which a Data Size Error occurred when GWEIS4.DSEIS is being set.

35.3.11.21 GWEIE4 : Error Interrupt Enable Register 4

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1294

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DSEIO E	DSEE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DSSEI OE	DSSE E
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DSSEE	Descriptor Security Error Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
1	DSSEIOE	Descriptor Security Error Interrupt Overflow Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
15:2	—	These bits are read as 0. The write value should be 0.	R/W
16	DSEE	Data Size Error Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
17	DSEIOE	Data Size Error Interrupt Overflow Interrupt Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
31:18	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

DSSEE bit (Descriptor Security Error Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID4.DSSED register will clear this bit.

DSSEIOE bit (Descriptor Security Error Interrupt Overflow Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID4.DSSEIOD register will clear this bit.

DSEE bit (Data Size Error Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID4.DSED register will clear this bit.

DSEIOE bit (Data Size Error Interrupt Overflow Interrupt Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID4.DSEIOD register will clear this bit.

35.3.11.22 GWEID4 : Error Interrupt Disable Register 4

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x1298

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DSEIOD	DSED
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DSSEIOD	DSSED
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DSED	Descriptor Security Error Disable Writing 1 to this bit will clear GWEIE4.DSSEE register.	R
1	DSSEIOD	Descriptor Security Error Interrupt Overflow Disable Writing 1 to this bit will clear GWEIE4.DSSEIOE register.	R
15:2	—	These bits are read as 0.	R
16	DSED	Data Size Error Disable Writing 1 to this bit will clear GWEIE4.DSEE register.	R
17	DSEIOD	Data Size Error Interrupt Overflow Disable Writing 1 to this bit will clear GWEIE4.DSEIOE register.	R
31:18	—	These bits are read as 0.	R

35.3.11.23 GWEIS5 : Error Interrupt Status Register 5

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x12A0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	RXDNECN[5:0]					—	—	—	—	—	—	—	RXDN EIOS	RXDN ES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	DCTECN[5:0]					—	—	—	—	—	—	—	DCTEIOS	DCTES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DCTES	Descriptor Chain Type Error Status Flag	R/W ¹
1	DCTEIOS	Descriptor Chain Type Error Interrupt Overflow Status Flag	R/W ¹
7:2	—	These bits are read as 0. The write value should be 0.	R/W
13:8	DCTECN[5:0]	Descriptor Chain Type Error Chain Number	R
15:14	—	These bits are read as 0. The write value should be 0.	R/W

Bit	Symbol	Function	R/W
16	RXDNES	RX Descriptor Number Error Status Flag	R/W ^{*1}
17	RXDNEIOS	RX Descriptor Number Error Interrupt Overflow Status Flag	R/W ^{*1}
23:18	—	These bits are read as 0. The write value should be 0.	R/W
29:24	RXDNECN[5:0]	RX Descriptor Number Error Chain Number	R
31:30	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

DUCTES bit (Descriptor Chain Type Error Status Flag)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Setting condition]

HW: When a descriptor is received for a transmission descriptor queue (GWDCCi.DQT is set).

[Error recovery]

HW: When a descriptor is received for a transmission descriptor queue, the corresponding frame is discarded.

SW: It is a software error. Software should be reviewed.

DUCTEIOS bit (Descriptor Chain Type Error Interrupt Overflow Status Flag)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Setting condition]

Descriptor Chain Type Error occurs and GWEIS5.DCTES is already set to one. In this case, the descriptor chain number in which the new error occurred is lost.

[Error recovery]

SW: Switch reset.

DUCTECN[5:0] bits (Descriptor Chain Type Error Chain Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

[Update conditions]

This field is update to the descriptor chain number in which a Descriptor Chain Type Error occurred when GWEIS5.DCTES is being set.

RXDNES bit (RX Descriptor Number Error Status Flag)

[Setting condition]

HW: GWMDNC.RXDMN+1 descriptor has already been used to process one frame and the frame processing is not finished.

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to one of these bits will clear it.

[Error recovery]

HW: set of the DESCR.ERR bit in RX descriptor if a part of the frame has been already sent.

HW: Current frame or remaining part of the frame will be discarded, and next frames will be processed normally.

SW: Discard data which have DESCR.ERR bit set.

SW: Software error, software should be reviewed.

RXDNEIOS bit (RX Descriptor Number Error Interrupt Overflow Status Flag)

[Clearing condition]

HW: Being in RESET mode will clear this register.

SW: Writing 1 to this bit will clear it.

[Setting condition]

RX Descriptor Number Error occurs and GWEIS5.RXDNES is already set to one. In this case, the descriptor chain number in which the new error occurred is lost.

[Error recovery]

SW: Switch reset.

RXDNECN[5:0] bits (RX Descriptor Number Error Chain Number)

[Clearing condition]

HW: Being in RESET mode will clear this register.

[Update conditions]

This field is update to the descriptor chain number in which a RX Descriptor Number Error occurred when GWEIS5.RXDNES is being set.

35.3.11.24 GWEIE5 : Error Interrupt Enable Register 5

Base address: GWCA0 = 0x403C_E000
GWCA0_NS = 0x503C_E000

Offset address: 0x12A4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	RXDN EIOE	RXDN EE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DCTE OE	DCTE E
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DCTEE	Descriptor Chain Type Error Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
1	DCTEIOE	Descriptor Chain Type Error Interrupt Overflow Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
15:2	—	These bits are read as 0. The write value should be 0.	R/W
16	RXDNEE	RX Descriptor Number Error Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
17	RXDNEIOE	RX Descriptor Number Error Interrupt Overflow Enable 0: Interrupt disabled 1: Interrupt Enabled	R/W ^{*1}
31:18	—	These bits are read as 0. The write value should be 0.	R/W

Note 1. Read value differs from written value.

DCTEE bit (Descriptor Chain Type Error Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID5.DCTED register will clear this bit.

DCTEIOE bit (Descriptor Chain Type Error Interrupt Overflow Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID5.DCTEIOD register will clear this bit.

RXDNEE bit (RX Descriptor Number Error Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID5.RXDNEED register will clear this bit.

RXDNEIOE bit (RX Descriptor Number Error Interrupt Overflow Enable)

[Setting condition]

Writing 1 to this bit will set it.

[Clearing condition]

Writing 1 to GWEID5.RXDNEIOD register will clear this bit.

35.3.11.25 GWEID5 : Error Interrupt Disable Register 5

Base address: GWCA0 = 0x403C_E000
 GWCA0_NS = 0x503C_E000

Offset address: 0x12A8

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	RXDN EIOD	RXDN ED
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	DCTEI OD	DCTE D
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	DCTED	Descriptor Chain Type Error Disable Writing 1 to this bit will clear GWEIE5.DCTEE register.	R
1	DCTEIOD	Descriptor Chain Type Error Interrupt Overflow Disable Writing 1 to this bit will clear GWEIE5.DCTEIOE register.	R
15:2	—	These bits are read as 0.	R
16	RXDNEED	RX Descriptor Number Error Disable Writing 1 to this bit will clear GWEIE5.RXDNEE register.	R
17	RXDNEIOD	RX Descriptor Number Error Interrupt Overflow Disable Writing 1 to this bit will clear GWEIE5.RXDNEIOE register.	R
31:18	—	These bits are read as 0.	R

35.4 Register Utilization

35.4.1 Operation Modes

Table 35.4 describes GWCA operation modes.

Table 35.4 GWCA operation modes

Operation mode	GWMS.OPS value	Description
DISABLE	1	- No transaction is ongoing. But there can be tolerance (left behind) of transactions*1 from GWCA when the agent has moved from OPERATION to DISABLE. - Only status registers are accessible for writing when the agent clock is enabled.
RESET	0	- No transaction is ongoing. - An internal Reset is asserted to reset GWCA logic with status registers (When a register is reset in RESET mode, it is mentioned in its description). - No register is accessible for writing. - RAM values are held.
CONFIG	2	- No transaction is ongoing. - Static and some dynamic registers are accessible for writing.
OPERATION	3	- Transactions are ongoing. - Dynamic registers are accessible for writing.

Note 1. max 8 read transactions or max 8 write transactions.

35.4.1.1 Operation Mode Transitions

Figure 35.2 shows the operating mode transitions.

A mode transition can be trigger by:

- Hardware reset.
- Software reset.
- Configuration of **GWMC.OPC**. In that case, mode transition will be confirmed by **GWMS.OPS**.

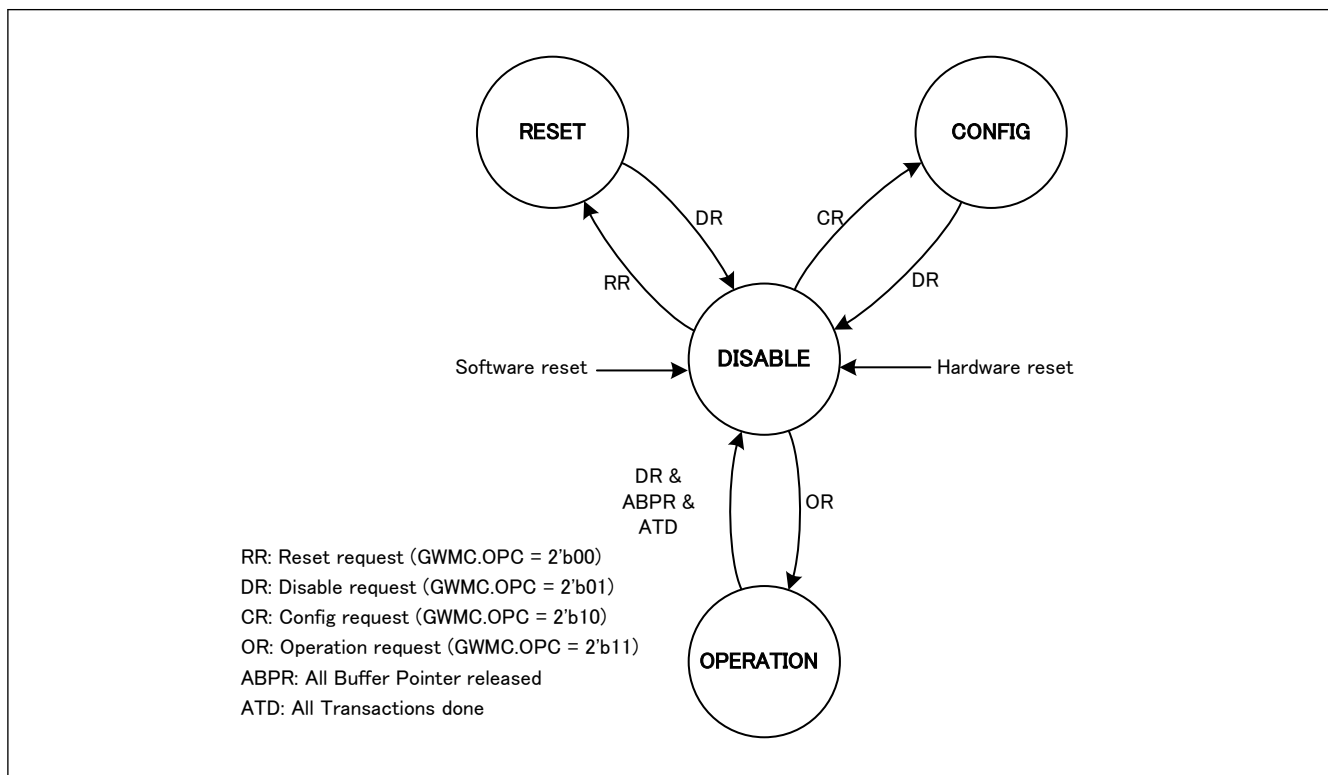


Figure 35.2 Operating mode transitions

Complementary information on transition conditions

ABPR:

- All the buffer pointers contained in the GWCA are release to the forwarding engine [FWD].

ATD:

- All the descriptors already received are processed and corresponding frames are sent to CPU and AXI descriptors are written back.

All ongoing transmit frames are fully sent to MFWD and AXI descriptors are written back.

[Restrictions]

- Software shall only trigger transitions shown to [Figure 35.2](#).
Exp : Do not directory transition from OPERATION to CONFIG.

35.4.2 Software Flows

Restrictions:

- SW: Please follow to the flow in this section.

35.4.2.1 Software Flows Legend

Software flow legend is described in [Figure 35.3](#).

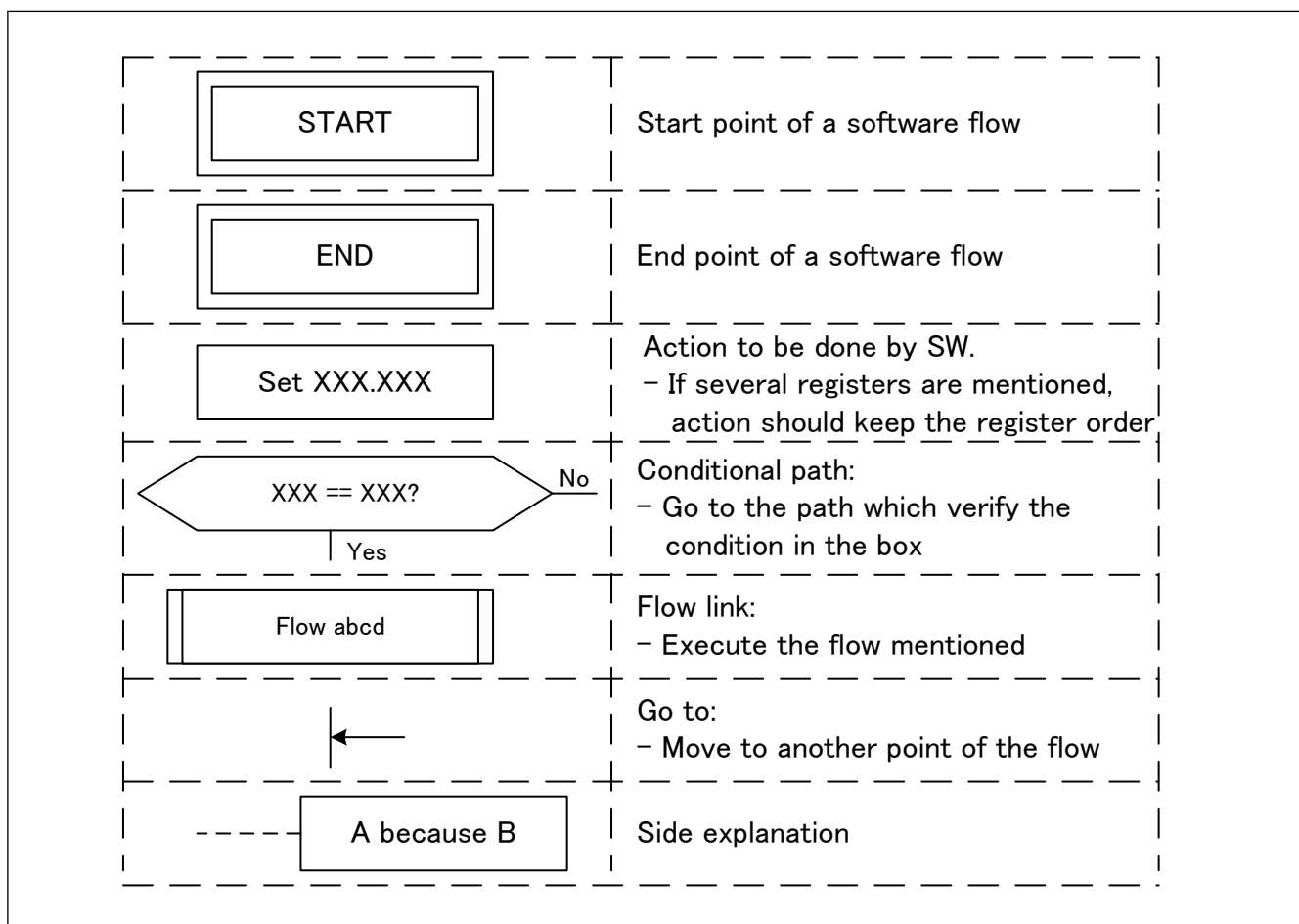


Figure 35.3 Software flow legend

35.4.2.2 Mode Transition Flow

The mode transition flow is described in [Figure 35.4](#).

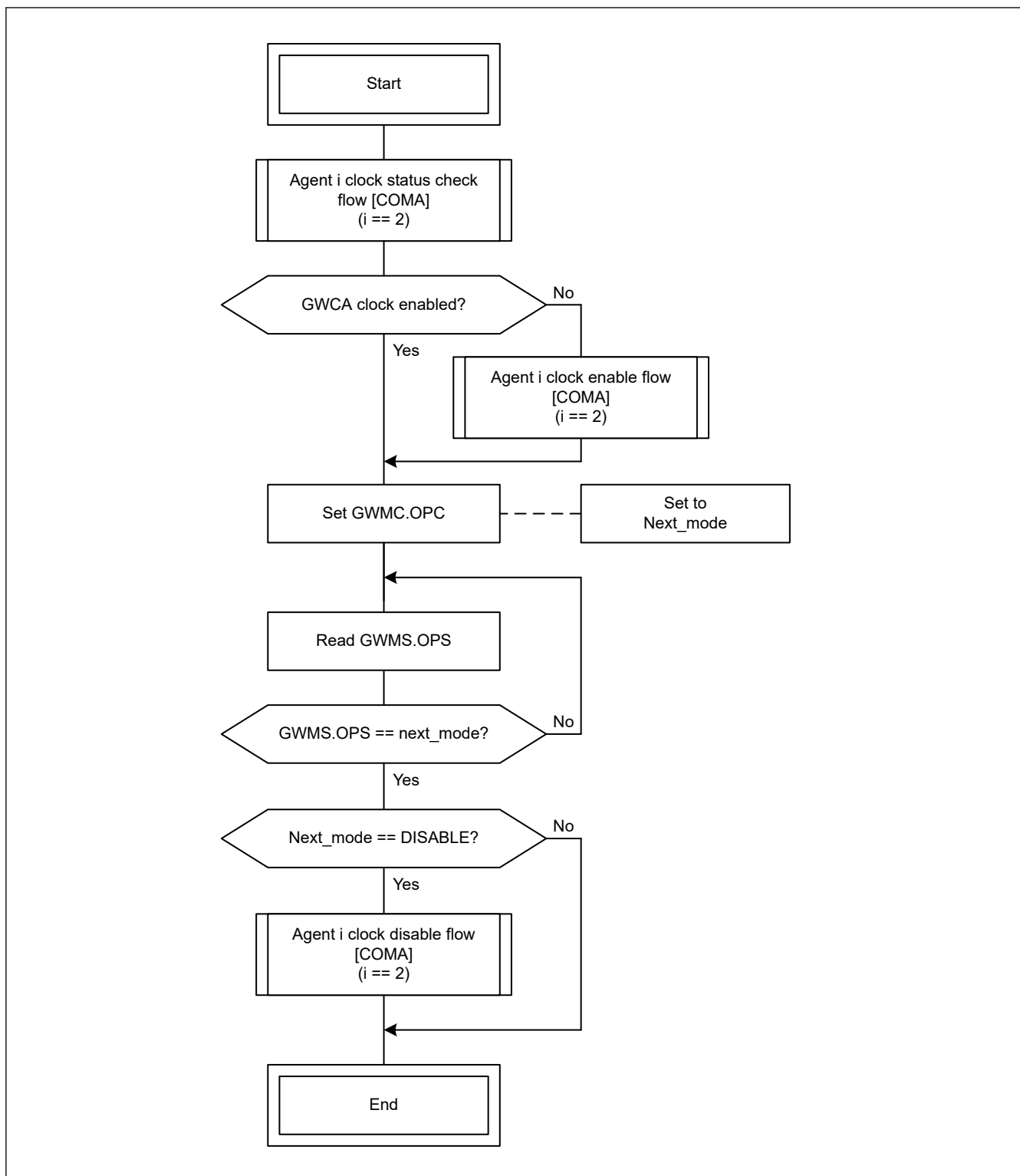


Figure 35.4 Mode transition flow

- Note:
- If SW already knows that the clock is enabled, clock status check step can be skipped.
 - If SW does not need to disable the agent clock in DISABLE mode (because it will go directly to another mode or because the clock never needs to be disabled), clock disable step can be skipped.

35.4.2.3 Reset Flow

The reset flow is described in [Figure 35.5](#).

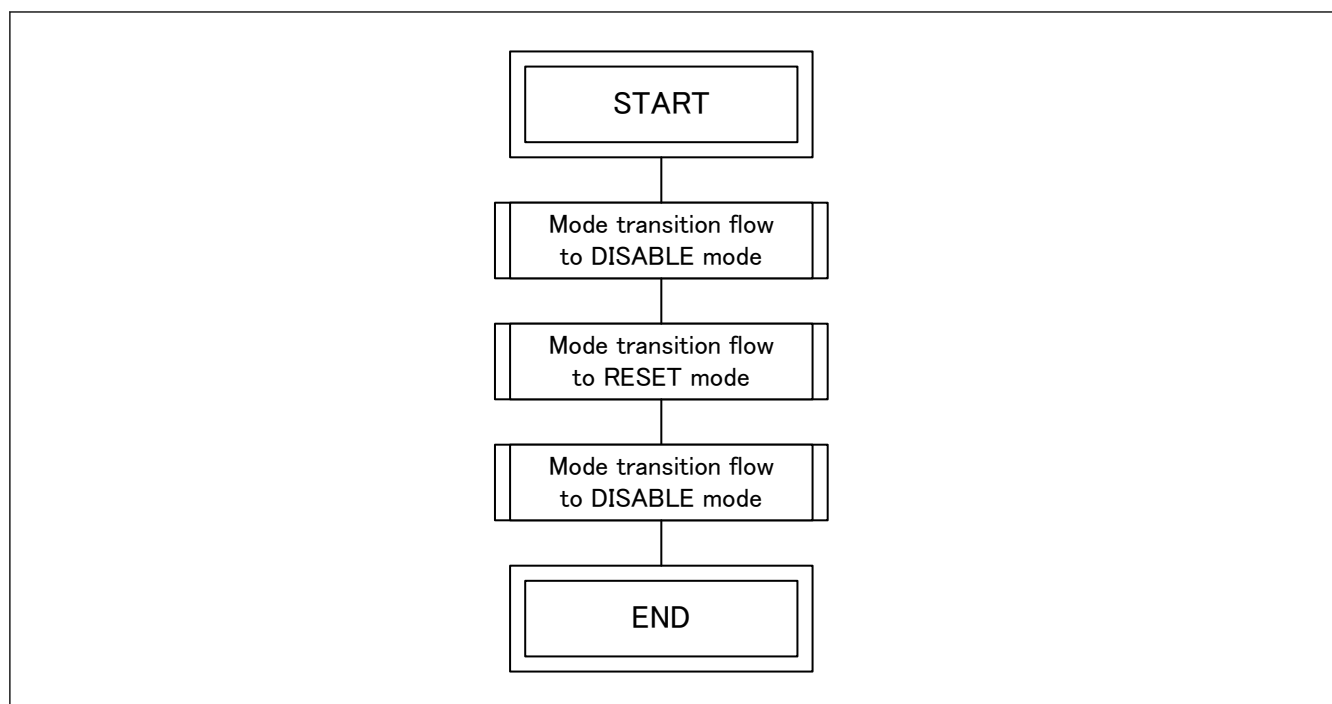


Figure 35.5 Reset flow

35.4.2.4 Initialization Flow

The initialization flow is described in [Figure 35.6](#).

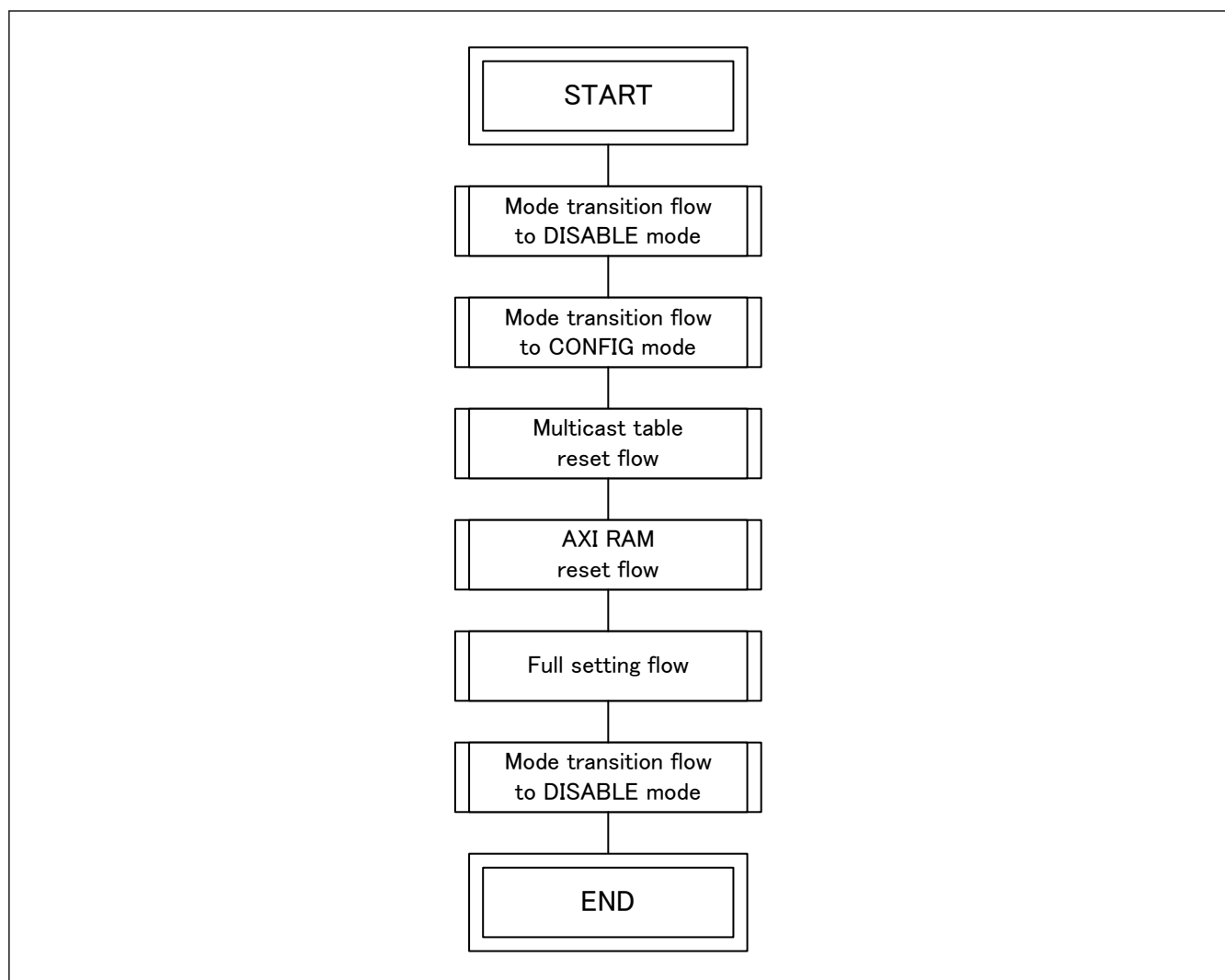


Figure 35.6 Initialization flow

35.4.2.5 Reinitialization Flow

The reinitialization flow is described in [Figure 35.7](#).

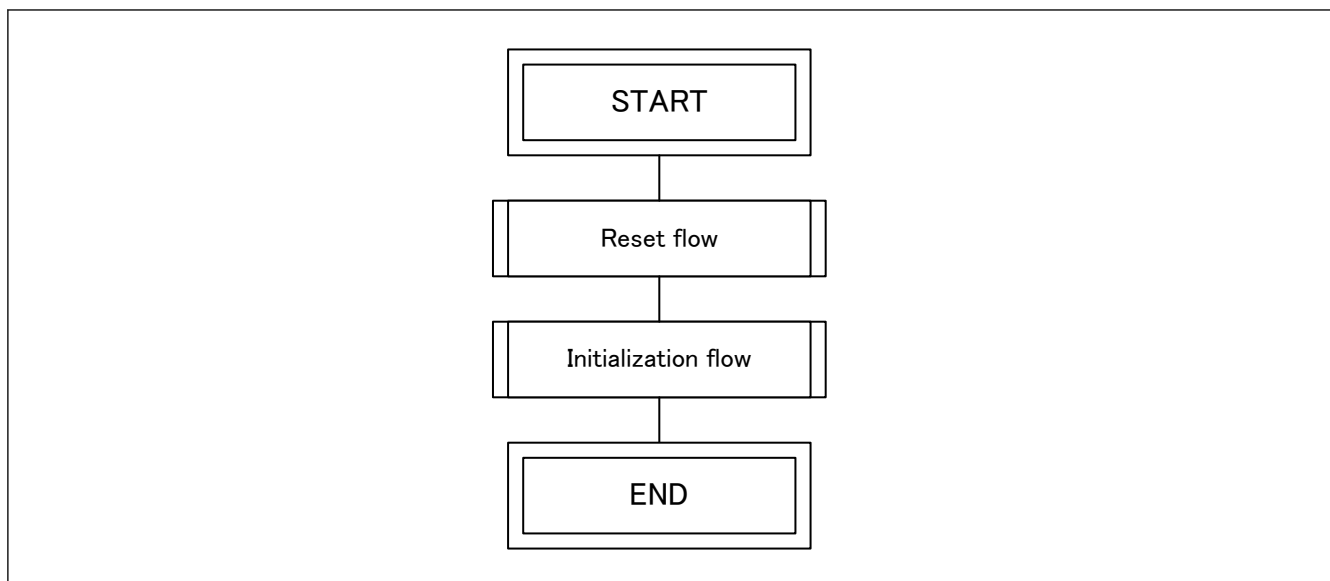


Figure 35.7 Reinitialization flow

35.4.2.6 Multicast Table Reset Flow

The multicast table reset flow is described in [Figure 35.8](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

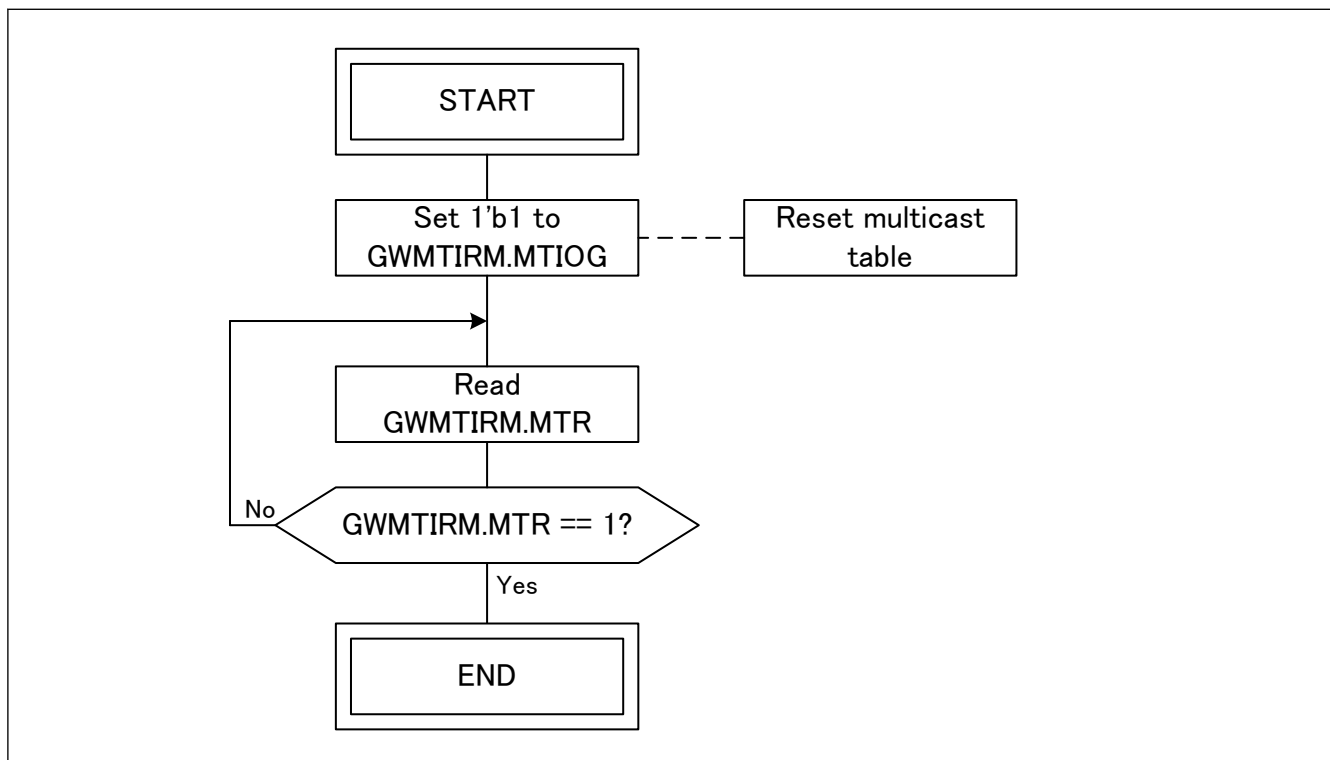


Figure 35.8 Multicast table reset flow

35.4.2.7 Multicast Table Setting Flow

The multicast table setting flow is described in [Figure 35.9](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

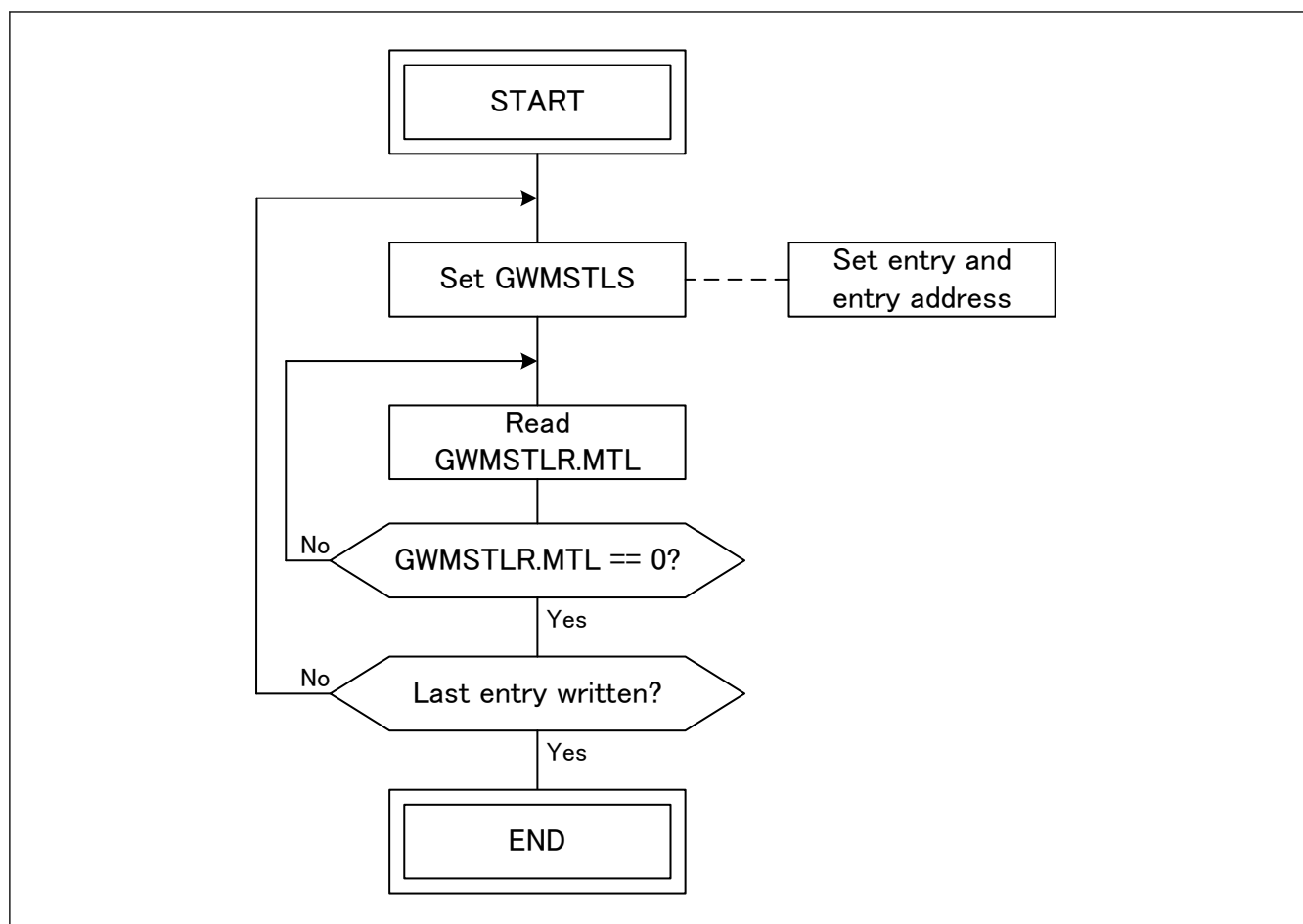


Figure 35.9 Multicast table setting flow

35.4.2.8 Multicast Table Read Flow

The multicast table read flow is described in [Figure 35.10](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

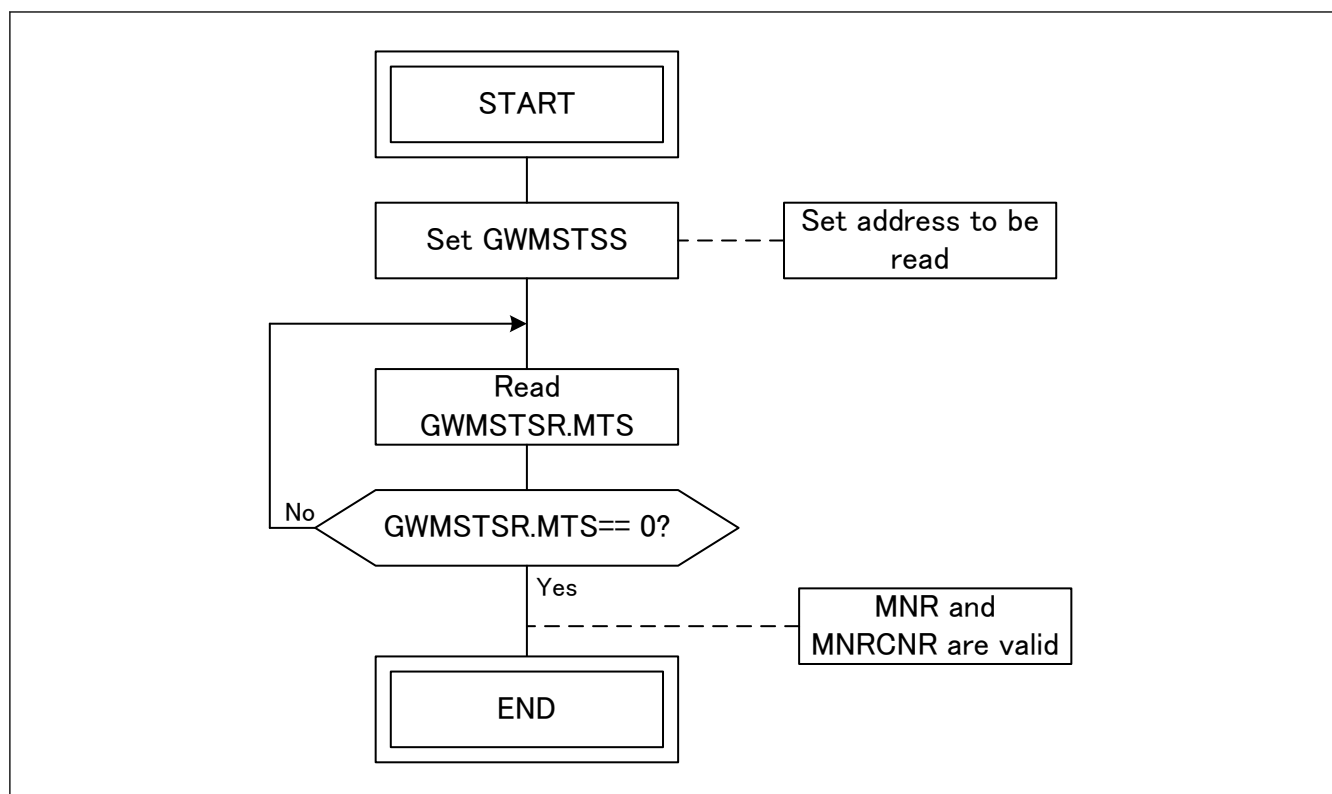


Figure 35.10 Multicast table read flow

35.4.2.9 AXI RAM Reset Flow

The AXI RAM reset flow is described in [Figure 35.11](#).

Restrictions:

- This flow is not usable in RESET, DISABLE and OPERATION modes.

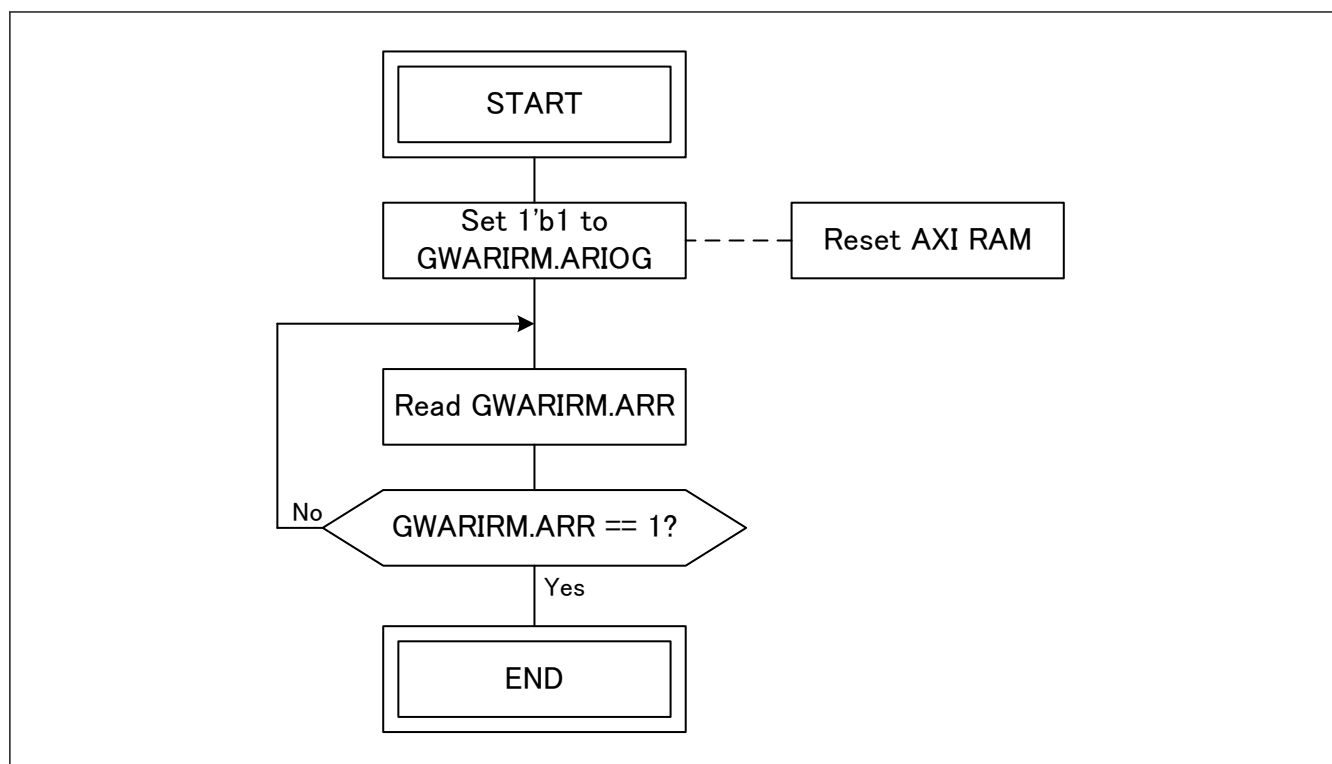


Figure 35.11 AXI RAM reset flow

35.4.2.10 AXI RAM Read Flow

The AXI RAM read flow is described in [Figure 35.12](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

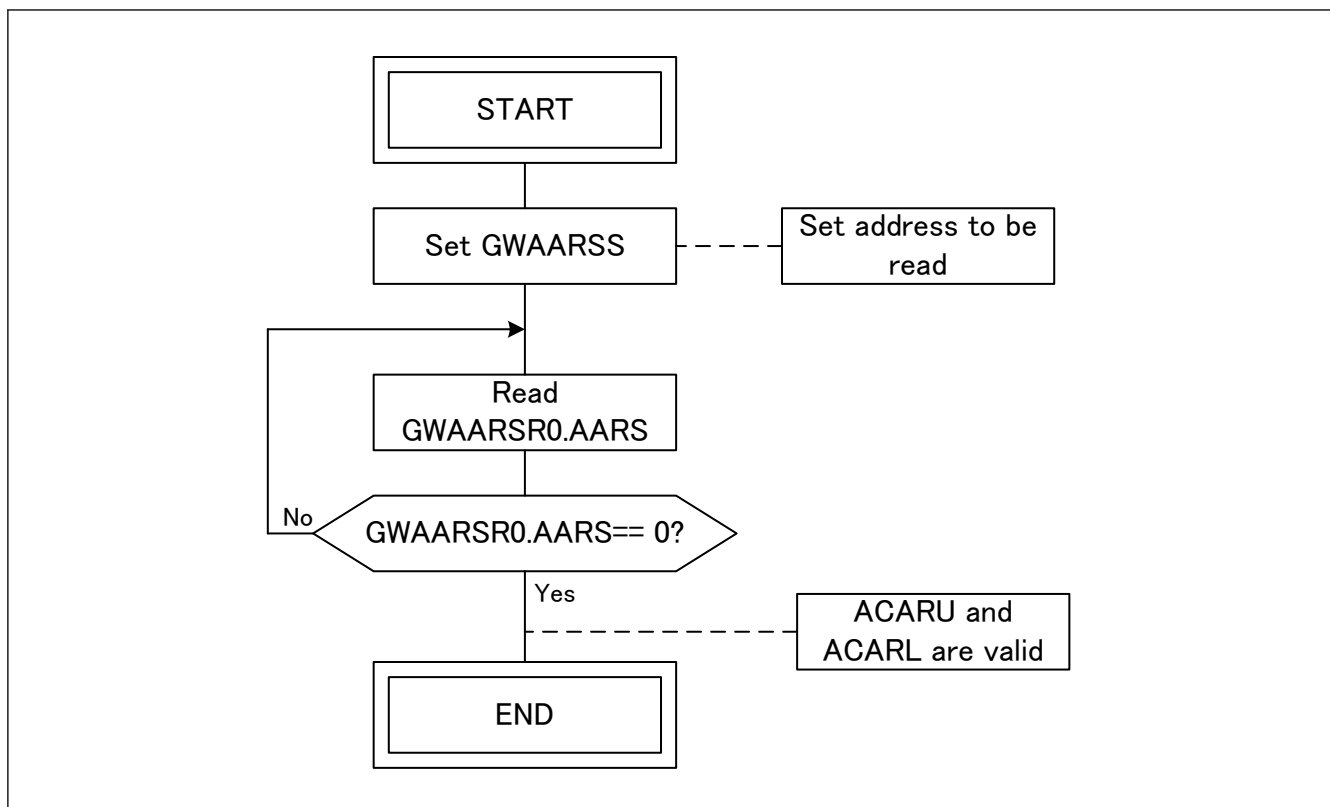


Figure 35.12 AXI RAM read flow

35.4.2.11 Global Rate Limiter Setting Flow

The global rate limiter setting flow is described in [Figure 35.13](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

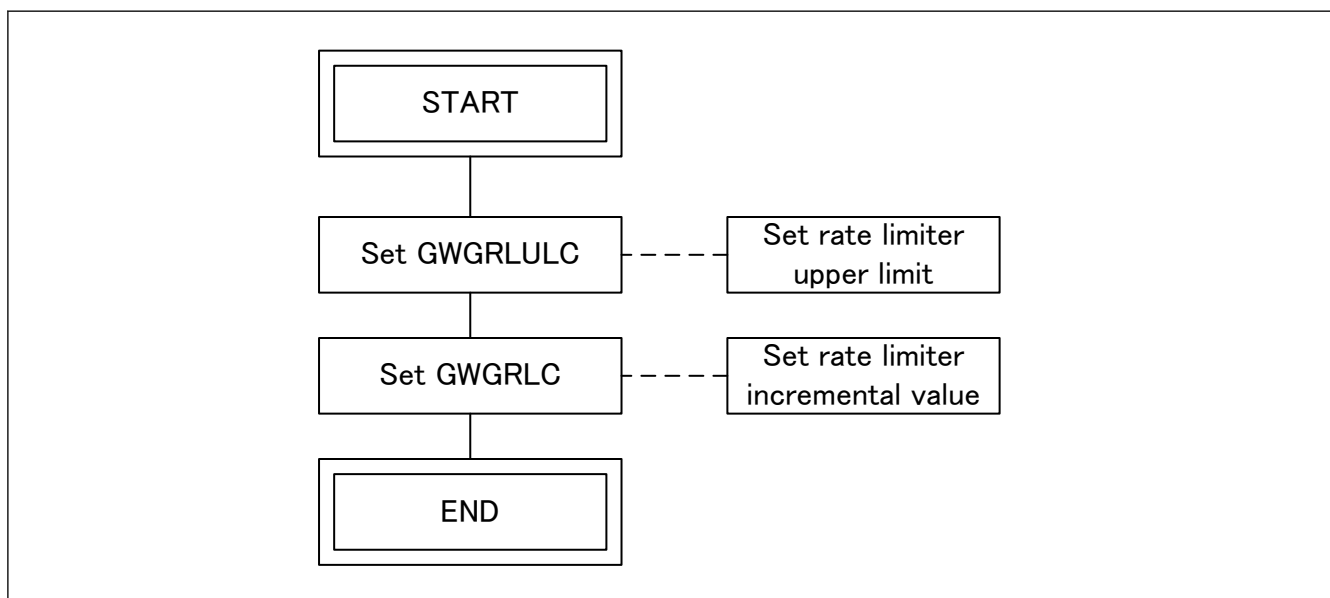


Figure 35.13 Global Rate limiter setting flow

35.4.2.12 Global Rate Limiter Disabling Flow

The global rate limiter disabling flow is described in [Figure 35.14](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

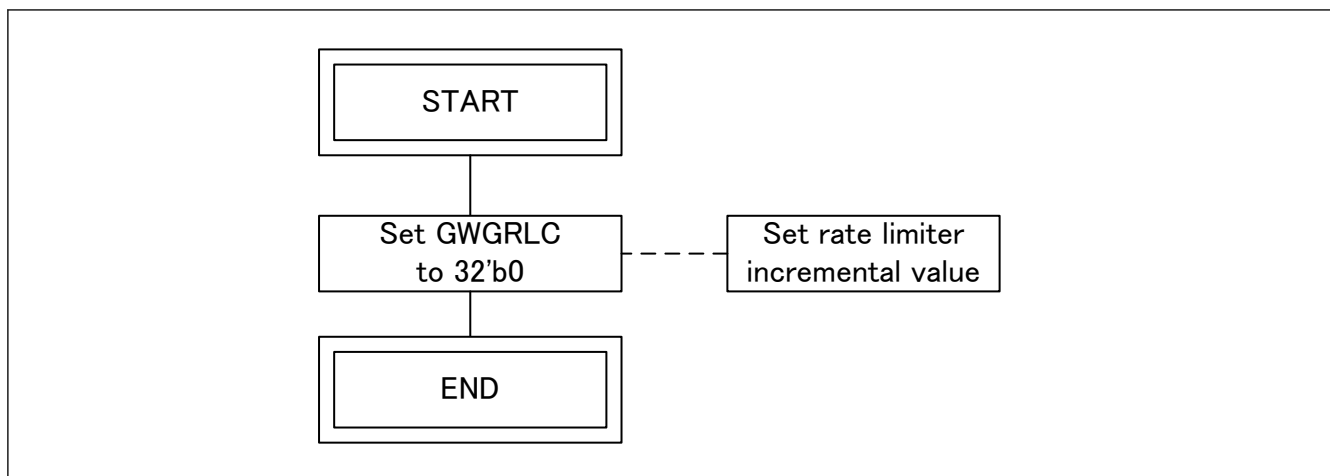


Figure 35.14 Global Rate limiter disabling flow

35.4.2.13 Rate Limiter i Setting Flow

The rate limiter i setting flow is described in [Figure 35.15](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

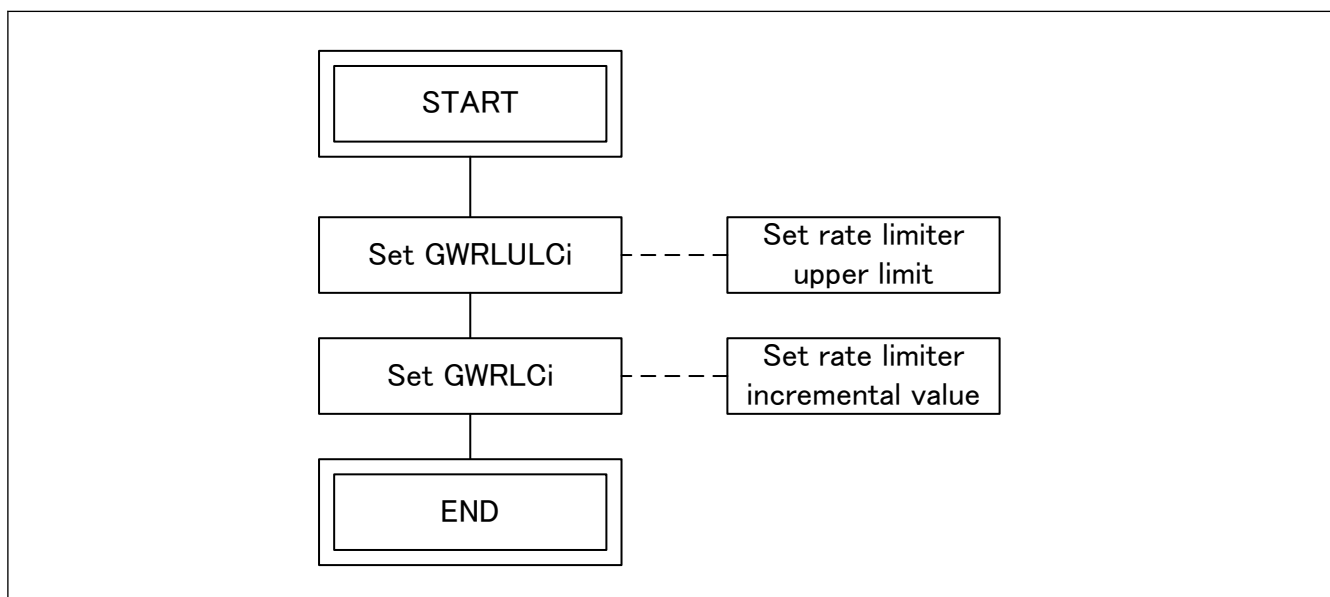


Figure 35.15 Rate limiter i setting flow

35.4.2.14 Rate Limiter i Disabling Flow

The rate limiter i disabling flow is described in [Figure 35.16](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

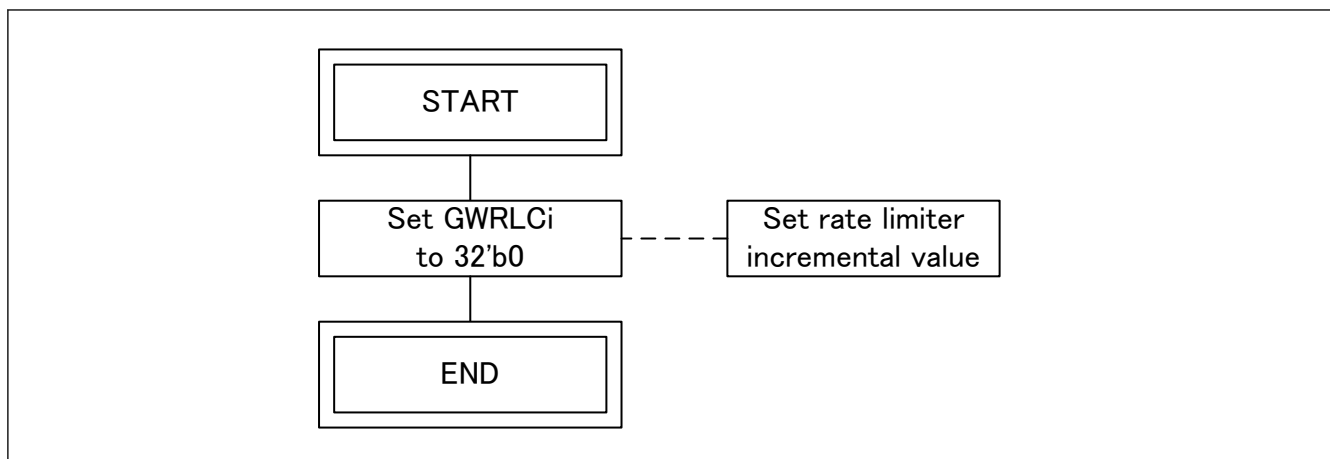


Figure 35.16 Rate limiter i disabling flow

35.4.2.15 AXI Table Read Flow

The AXI table read flow is described in [Figure 35.17](#).

Restrictions:

- This flow is not usable in RESET and DISABLE modes.

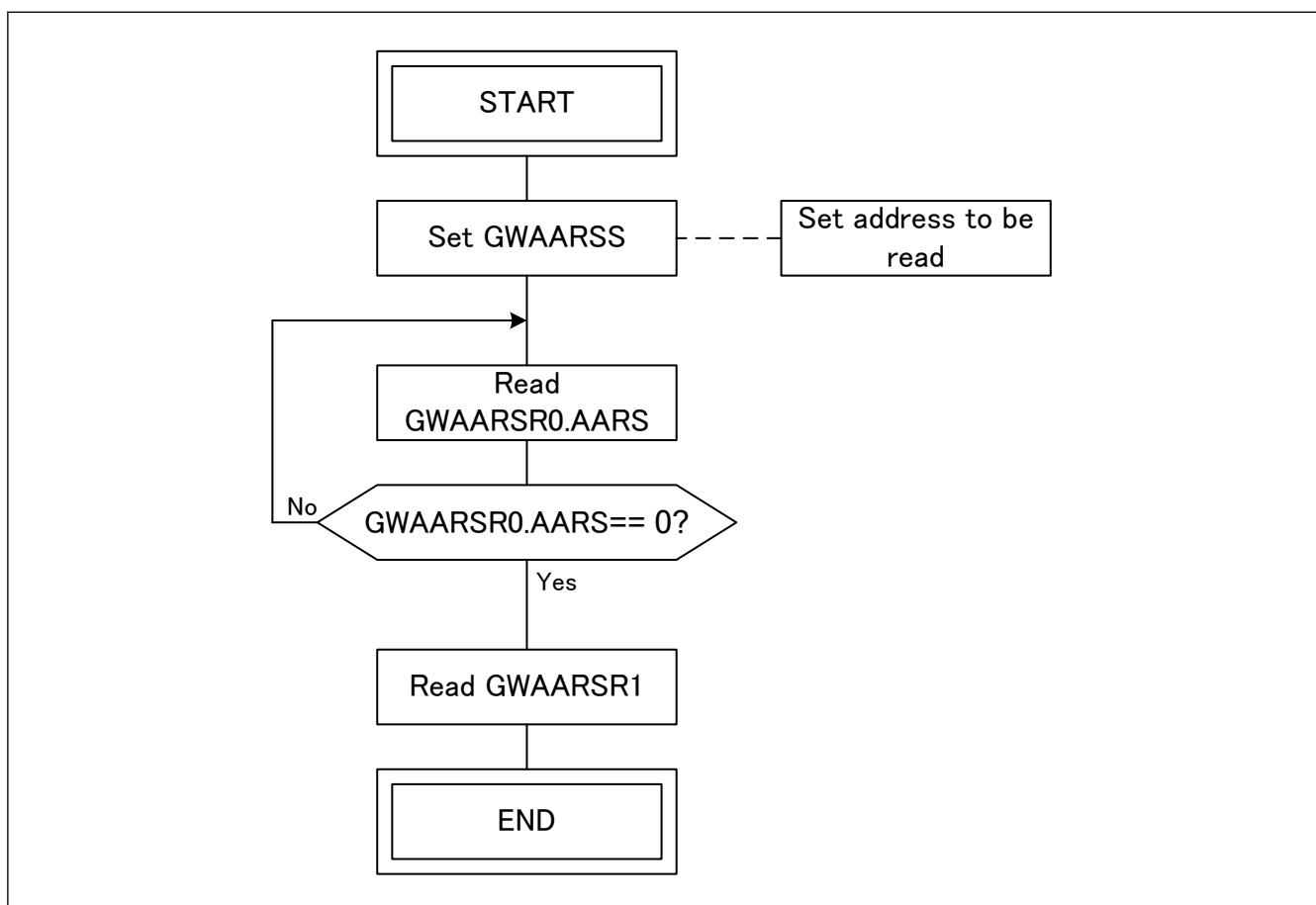


Figure 35.17 AXI table read flow

35.4.2.16 Interrupt Handling Flow

The interrupt handling flow is described in [Figure 35.18](#).

Restrictions:

- This flow is not usable in RESET mode.

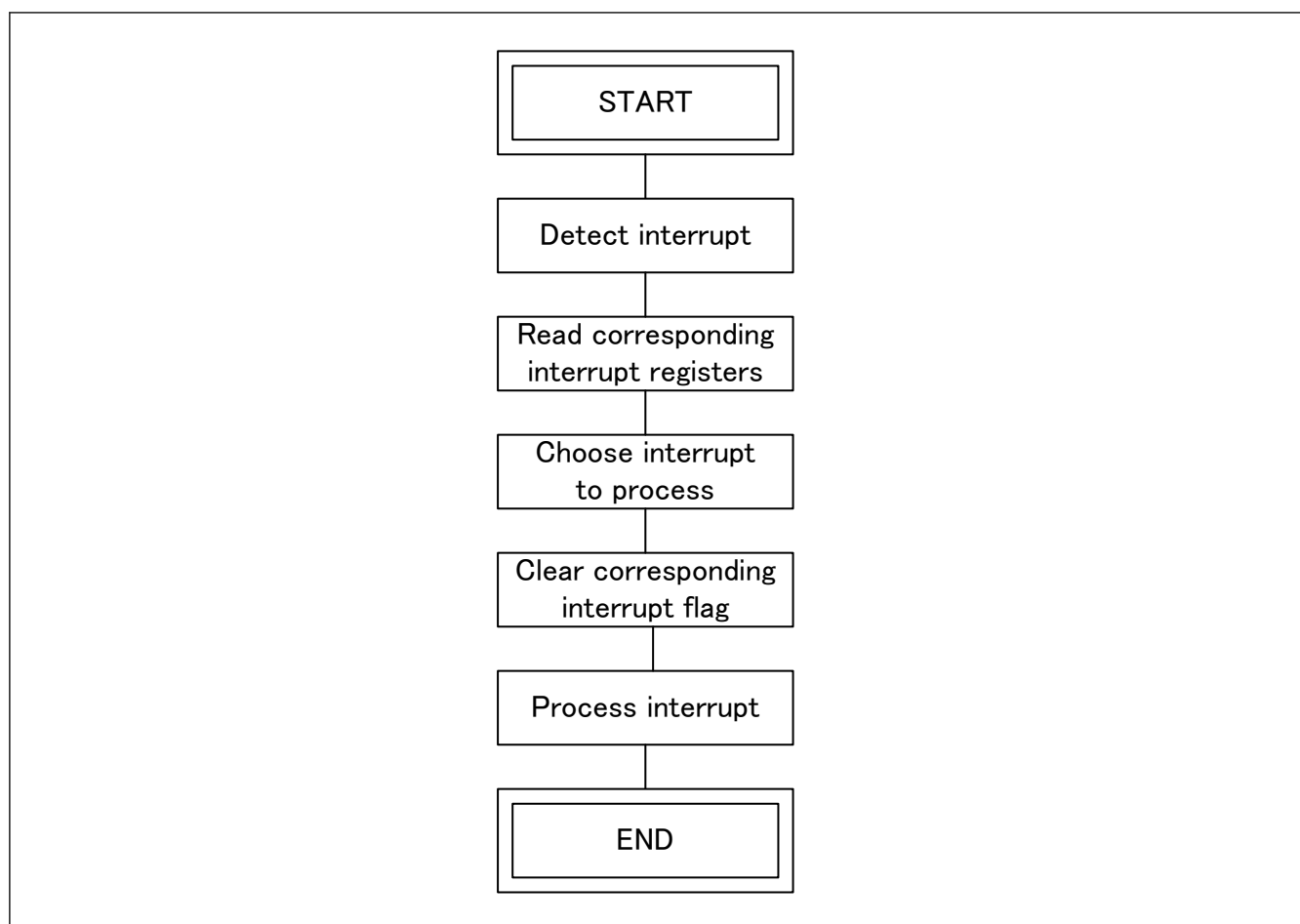


Figure 35.18 Interrupt handling flow

35.4.2.17 Called Software Flows

The flows described in this section can only be called from other flows thanks to a “flow link” box ([Figure 35.3](#)) and cannot be used alone.

35.4.2.17.1 Full Setting Flow

The full setting flow is described in [Figure 35.19](#).

For LINKFIX table setting see [section 35.5.1.3.1. LINKFIX Table Initialization](#).

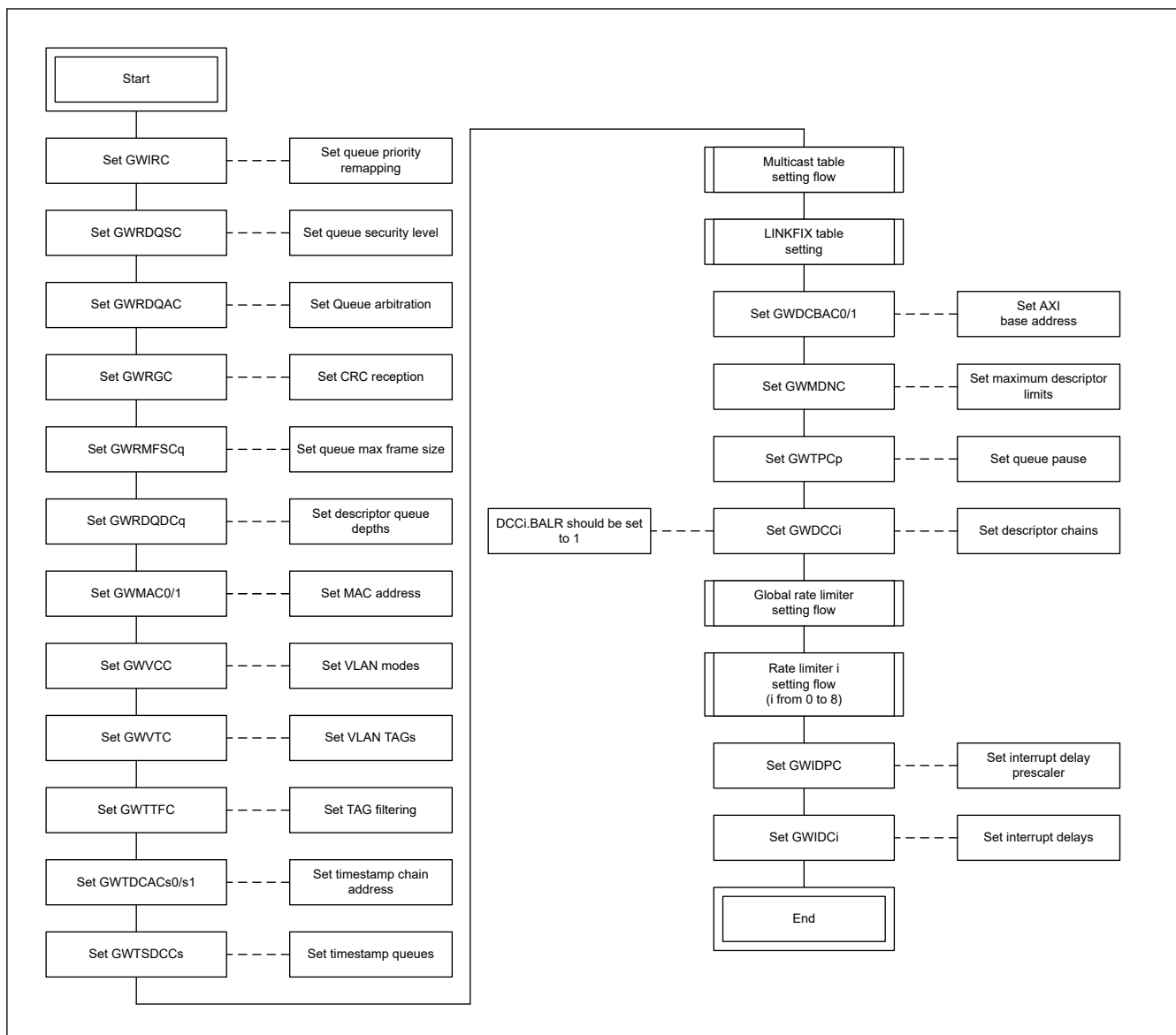


Figure 35.19 Full setting flow

35.4.2.17.2 AXI Emergency Stop Flow

The AXI emergency stop flow is described in [Figure 35.20](#).

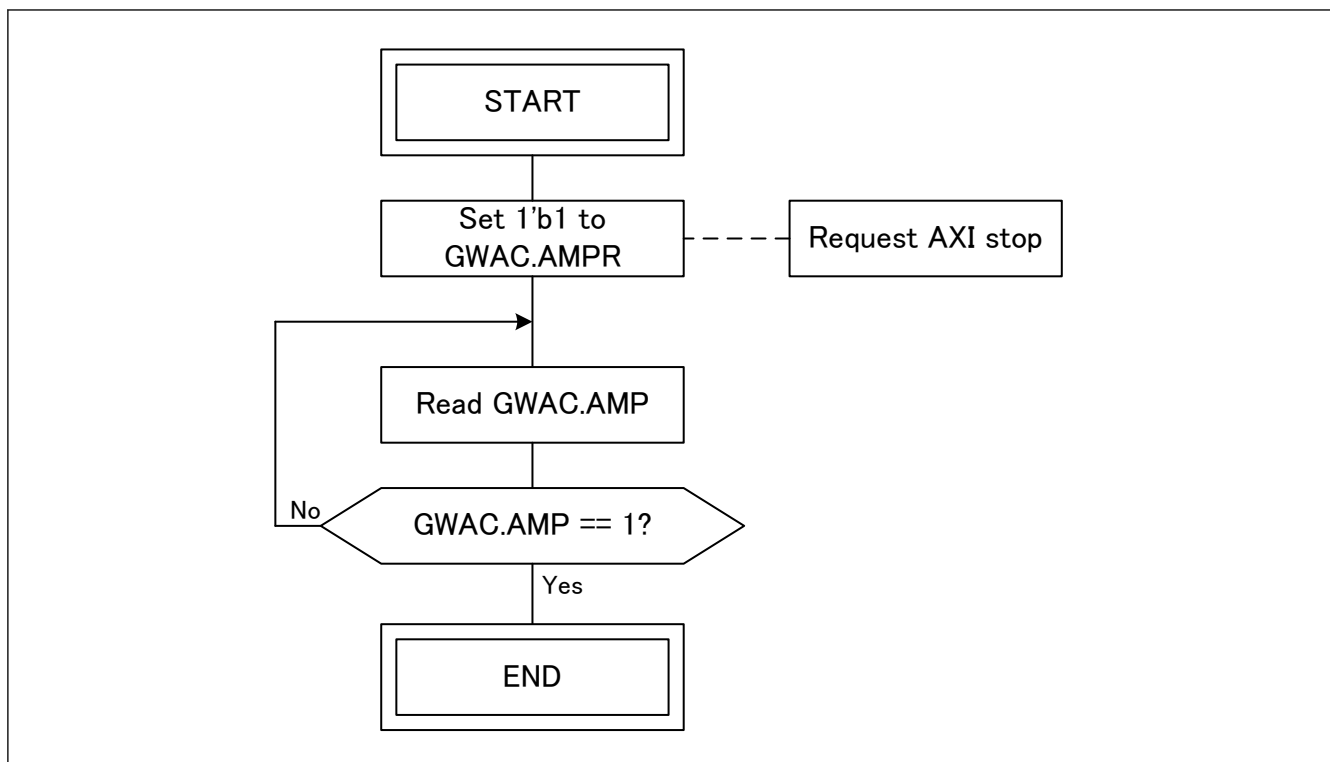


Figure 35.20 AXI emergency stop flow

35.4.2.18 Register Writable Without Software Flow

This section describes registers that have not been described so far. These registers can be changed dynamically. (However, it is necessary that the initial settings such as the clock enabling have been completed.)

Note: When a frame has reached GWCA and before the frame has been written into URAM by AXI BMI if GWDCC configurations of target chain is reconfigured by the user then the frame will be handled using the GWDCC register settings before reconfiguration and not using the reconfigured GWDCC settings.

35.5 Functional Details

35.5.1 AXI Master General Information

In this section is described the knowledge required to utilize the GWCA AXI master. It contains the AXI address table handling, the AXI descriptor general description, and the AXI IDs utilization description.

35.5.1.1 AXI Descriptor Queue Mapping

The AXI descriptor queue mapping is described along with its mapping to the **INTER_TX** interface, the incremental queues and the rate limiters in [Figure 35.21](#). The global rate limiter is not represented because it limits throughput for all the TX queues.

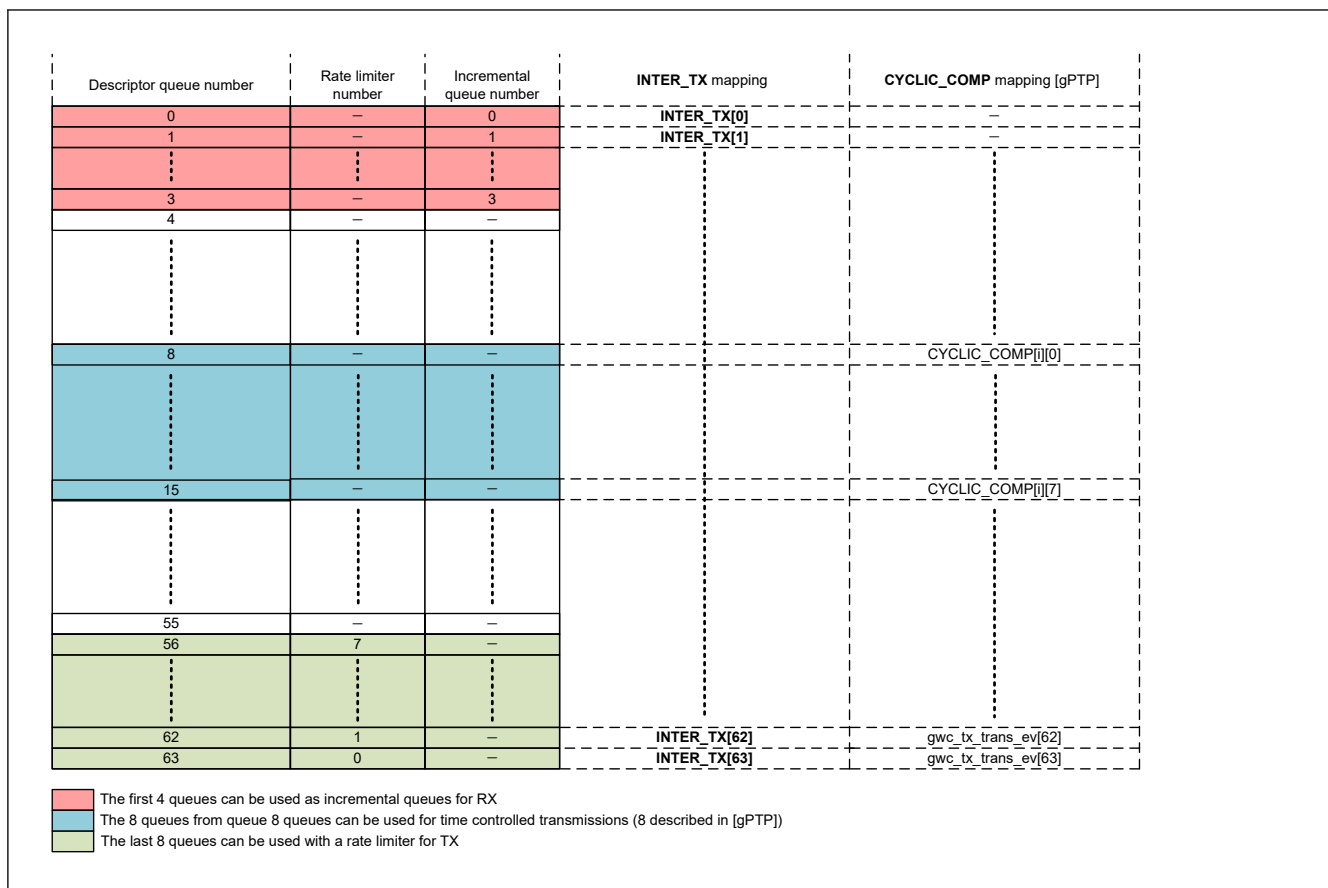


Figure 35.21 AXI descriptor queue mapping

35.5.1.2 AXI Descriptor General Description

35.5.1.2.1 General Formats

AXI descriptor general formats are described in [Figure 35.22](#), [Figure 35.23](#), [Figure 35.24](#) and [Figure 35.25](#). The fields are explained in [Table 35.5](#). The fields utilization will be explained per transaction type (Data reception, Data transmission and Timestamp reception).

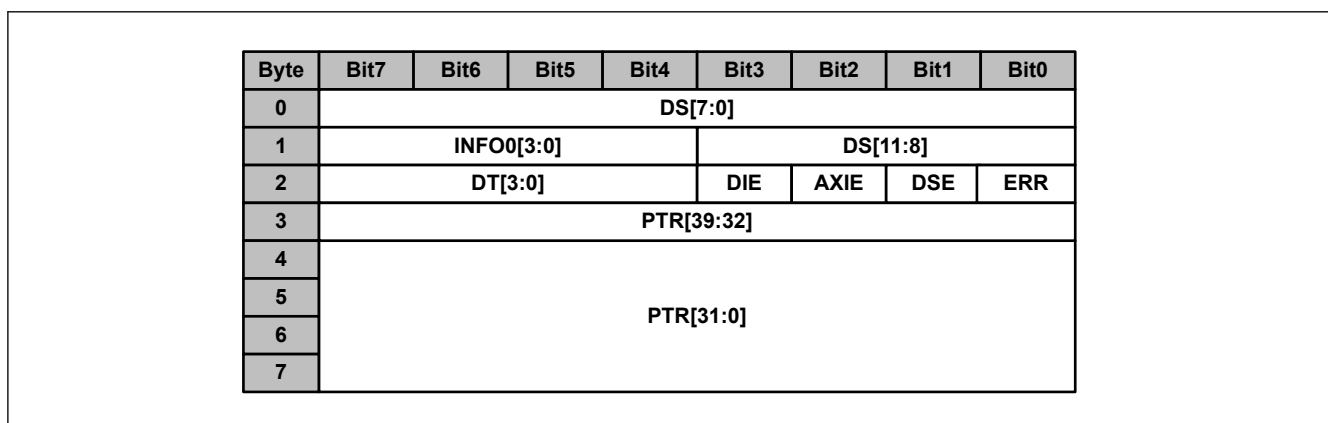


Figure 35.22 Basic descriptor (GWDCCi.ETS == 0 and GWDCCi.EDE == 0)

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	DS[7:0]							
1	INFO0[3:0]				DS[11:8]			
2	DT[3:0]				DIE	AXIE	DSE	ERR
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								
8	TS[63:0]							
9								
10								
11								
12								
13								
14								
15								

Figure 35.23 Timestamp descriptor (GWDCCi.ETS == 1 and GWDCCi.EDE == 0)

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	DS[7:0]							
1	INFO0[3:0]				DS[11:8]			
2	DT[3:0]				DIE	AXIE	DSE	ERR
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								
8	INFO1[63:0]							
9								
10								
11								
12								
13								
14								
15								

Figure 35.24 Extended descriptor (GWDCCi.ETS == 0 and GWDCCi.EDE == 1)

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	DS[7:0]							
1	INFO0[3:0]				DS[11:8]			
2	DT[3:0]				DIE	AXIE	DSE	ERR
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								
8	INFO1[63:0]							
9								
10								
11								
12								
13								
14								
15								
16	TS[63:0]							
17								
18								
19								
20								
21								
22								
23								

Figure 35.25 Extended timestamp descriptor (GWDCCi.ETS == 1 and GWDCCi.EDE == 1)

Table 35.5 General format descriptor field explanation

Field name	Bit width	Description
DS	12	Descriptor size Restrictions: - SW: DS field maximum value is 2048. - SW: For other descriptors that LINKFIX, LEMPTY, EEMPTY, LINK and EOS, DS should not be set to 0.
INFO0	4	Information 0
ERR	1	Error
DSE	1	Data Size Error
AXIE	1	AXI Bus Error
DIE	1	Descriptor Interrupt Enable This (DESCR.DIE) field should not be set for LINK/LINKFIX descriptors in Rx chain.
DT	4	Descriptor Type, see section 35.5.1.2.2. Descriptor Types .
PTR	40	Pointer
INFO1	64	Information 1
TS	64	Timestamp

Note: General format descriptor field utilization is described per use-case later in the following sections.

35.5.1.2.2 Descriptor Types

The table below gives an overview of all descriptor types used by GWCA. The first column shows the descriptor type name and the second column the 4-bit number to be written to DESC.DT.

Table 35.6 Descriptor Type

Name	DT	Description
Empty data descriptors		
FEMPTY_IS	1	Frame Empty Incremental Start Frame data related descriptor without valid frame data. Used to set an incremental area.
FEMPTY_IC	2	Frame Empty Incremental Continue Frame data related descriptor without valid frame data. Used to continue using an incremental area.
FEMPTY_ND	3	Frame Empty No Data Storage Frame data related descriptor without valid frame data. Used to reject data.
FEMPTY	4	Frame Empty Frame complete data related descriptor without valid frame data. Used to save frame data.
FEMPTY_START	5	Frame Empty start Frame start data related descriptor without valid frame data. Used to save first part of a frame.
FEMPTY_MID	6	Frame Empty Middle Frame continues data related descriptor without valid frame data. Used to save middle part of a frame.
FEMPTY_END	7	Frame Empty End Frame end data related descriptor without valid frame data. Used to save end part of a frame.
Data descriptors		
FSINGLE	8	Frame Single Storage Element has valid frame data of a complete frame.
FSTART	9	Frame Start Storage Element has valid frame data. Frame starts with this descriptor and continues in next descriptor.
FMID	10	Frame Middle Storage Element has valid frame data. Frame has started with a previous descriptor and continues in next descriptor.
FEND	11	Frame End Storage Element has valid frame data. Frame has started with a previous descriptor and ends with this descriptor.
Chain control / HW/SW arbitration		
LINKFIX	0	Linkfix Control element pointing to next descriptor in chain.
LEEMPTY	12	Link Empty Used control element pointing to next descriptor in chain.
EEMPTY	13	EOS Empty Used control element pausing a transmit chain.
LINK	14	Link Control element pointing to next descriptor in chain.
EOS	15	End Of Set Control element pausing a transmit chain.

35.5.1.3 LINKFIX Table

35.5.1.3.1 LINKFIX Table Initialization

The usage of AXI address table requires the setting of a LINKFIX table in the CPU USER RAM at {GWDCBAC.DCBAU, GWDCBAC.DCBAL} location. This table will be used to set the first address of descriptor chains. It can be used any time to change a chain base address or to return to the previous base address.

The LINKXFIX table setting is described in [Figure 35.26](#) through an example. Note that SW and HW initialization are happening simultaneously.

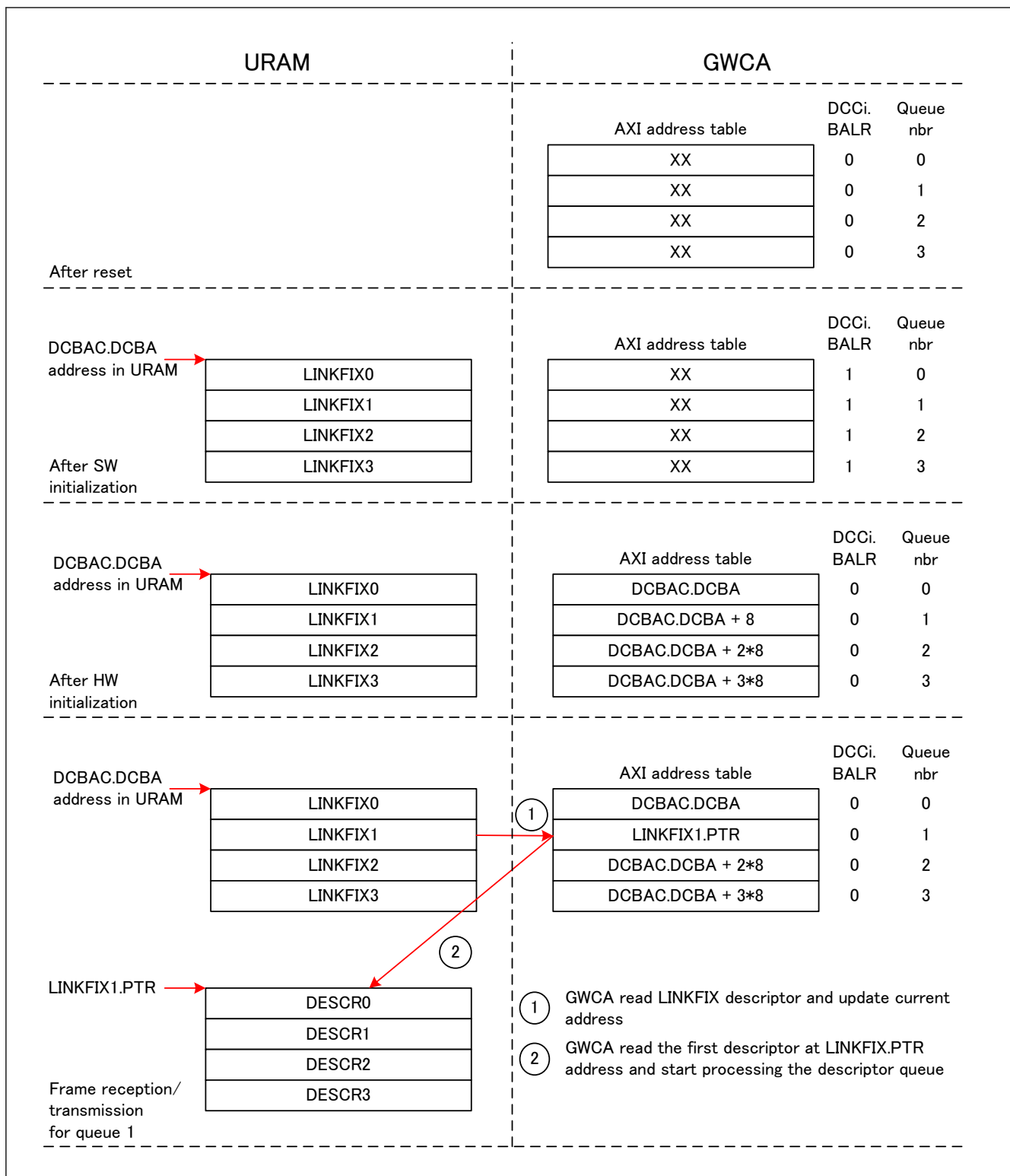


Figure 35.26 LINKXFIX table setting example

Restrictions:

- SW: A LEMPTY descriptor queues must be written for preventing them from starting.
- HW: Added descriptors in the LINKFIX table are 8 byte-descriptors which are independent of GWDCCi.EDE and GWDCCi.ETS register settings.

Note: LINKFIX and LINK descriptors are interchangeable. The only difference is that the LINK descriptor can be written back by HW.

35.5.1.3.2 LINKFIX Table Descriptors

For the LINKFIX table, a LINKFIX descriptor is used to start a descriptor queue, a LEMPTY descriptor is used to disable a TX queues (GWDCCi.DQT == 1) and a LEMPTY descriptor is used to disable a RX queues (GWDCCi.DQT == 0). In the LINKFIX table, only basic descriptors can be used (section 35.5.1.2.1. General Formats). The descriptor formats for LINKFIX table are describes in Figure 35.27 and Table 35.7. If a reception disabled queue is started despite being disabled in the LINKFIX table, the descriptor queue be considered as full (see register GWEIS2i explanation).

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	8'b0							
1	4'b0				4'b0			
2	4'd0				1'b0	1'b0	1'b0	1'b0
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								

LINKFIX for LINKFIX table

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	8'b0							
1	4'b0				4'b0			
2	4'd12				1'b0	1'b0	1'b0	1'b0
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								

LEMPY for LINKFIX table

Figure 35.27 LINKFIX table descriptor formats

Table 35.7 LINKFIX table descriptor field description

Field name	LINKFIX	LEMPY
DS	0	0
INFO0	0	0
ERR	0	0
DSE	0	0
AXIE	0	0
DIE	0	0
DT	0	12
PTR	PTR Pointer pointing to the first descriptor of the descriptor chain	0
INFO1 (GWDCCi.EDE == 1)	NA for LINKFIX table	NA for LINKFIX table
TS (GWDCCi.ETS == 1)	NA for LINKFIX table	NA for LINKFIX table

35.5.1.4 AXI Common Descriptor Utilization

AXI common descriptors are the descriptors which are not reserved to a special use-case and can be inserted in any descriptor chain at any time. It includes LINKFIX and LINK descriptors.

35.5.1.4.1 LINKFIX Descriptor Utilization

A LINKFIX descriptor is used to change a descriptor queue pointer in the GWCA to move it to another place and is described in [Figure 35.28](#). The LINKFIX descriptor is never written back to decrease the AXI load.

The LINKFIX descriptor format is described in [Figure 35.29](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.8](#).

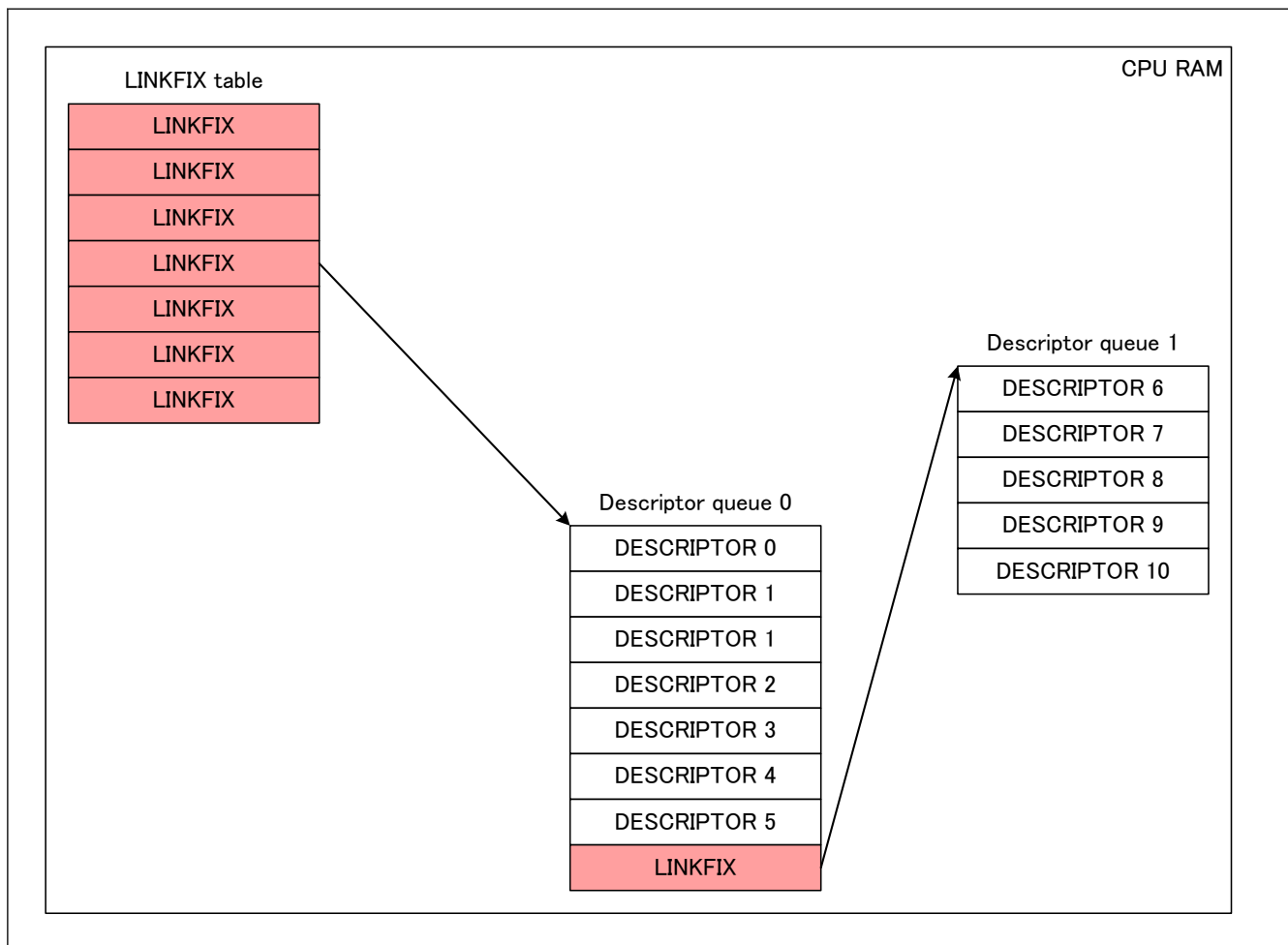


Figure 35.28 LINKFIX descriptor utilization

- Note:
- The LINKFIX table is described in [section 35.5.1.3. LINKFIX Table](#).
 - A LINKFIX descriptor can be placed anywhere in a descriptor queue.
 - LINKFIX and LINK descriptors are interchangeable. The only difference is that the LINK descriptor can be written back by HW.

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	8'b0							
1	4'b0				4'b0			
2	4'd0				1'b0	1'b0	1'b0	1'b0
3	PTR[39:32]							
4	PTR[31:0]							
5								
6								
7								

LINKFIX written by SW

Figure 35.29 LINKFIX utilization format

Table 35.8 LINKFIX utilization field description

Field name	Field description written by SW	Field description after HW write-back
DS	0	NA: LINKFIX descriptors are never written back
INFO0	0	NA: LINKFIX descriptors are never written back
ERR	0	NA: LINKFIX descriptors are never written back
DSE	0	NA: LINKFIX descriptors are never written back
AXIE	0	NA: LINKFIX descriptors are never written back
DIE	0	NA: LINKFIX descriptors are never written back
DT	0	NA: LINKFIX descriptors are never written back
PTR	PTR Pointer pointing to the new descriptor queue	NA: LINKFIX descriptors are never written back
INFO1 (GWDCCi.EDE == 1)	0	NA: LINKFIX descriptors are never written back
TS (GWDCCi.ETS == 1)	0	NA: LINKFIX descriptors are never written back

35.5.1.4.2 LINK Descriptor Utilization

A LINK descriptor has the same usage as a LINKFIX descriptor (see [section 35.5.1.4.1. LINKFIX Descriptor Utilization](#)). Contrary to the LINKFIX descriptor the LINK descriptor can be written back by HW. The LINK descriptor format is described in [Figure 35.30](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.9](#).

Note: LINKFIX and LINK descriptors are interchangeable. The only difference is that the LINK descriptor can be written back by HW.

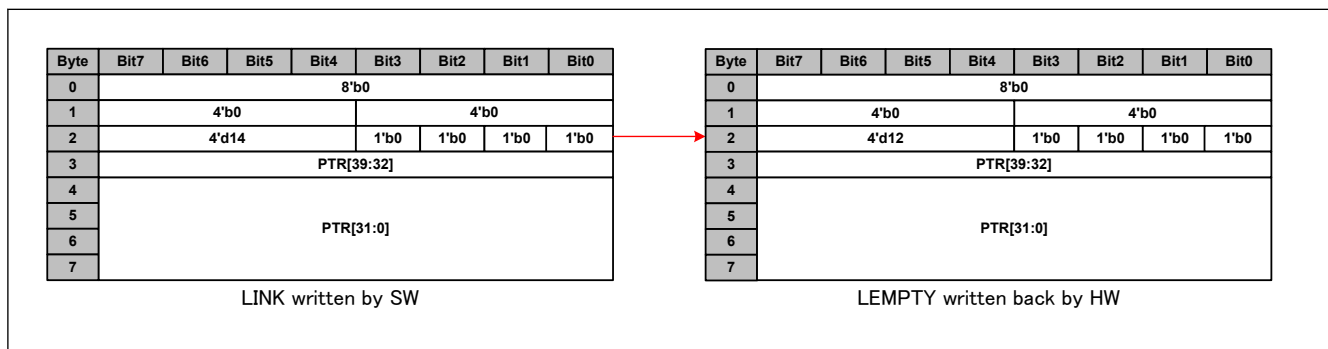


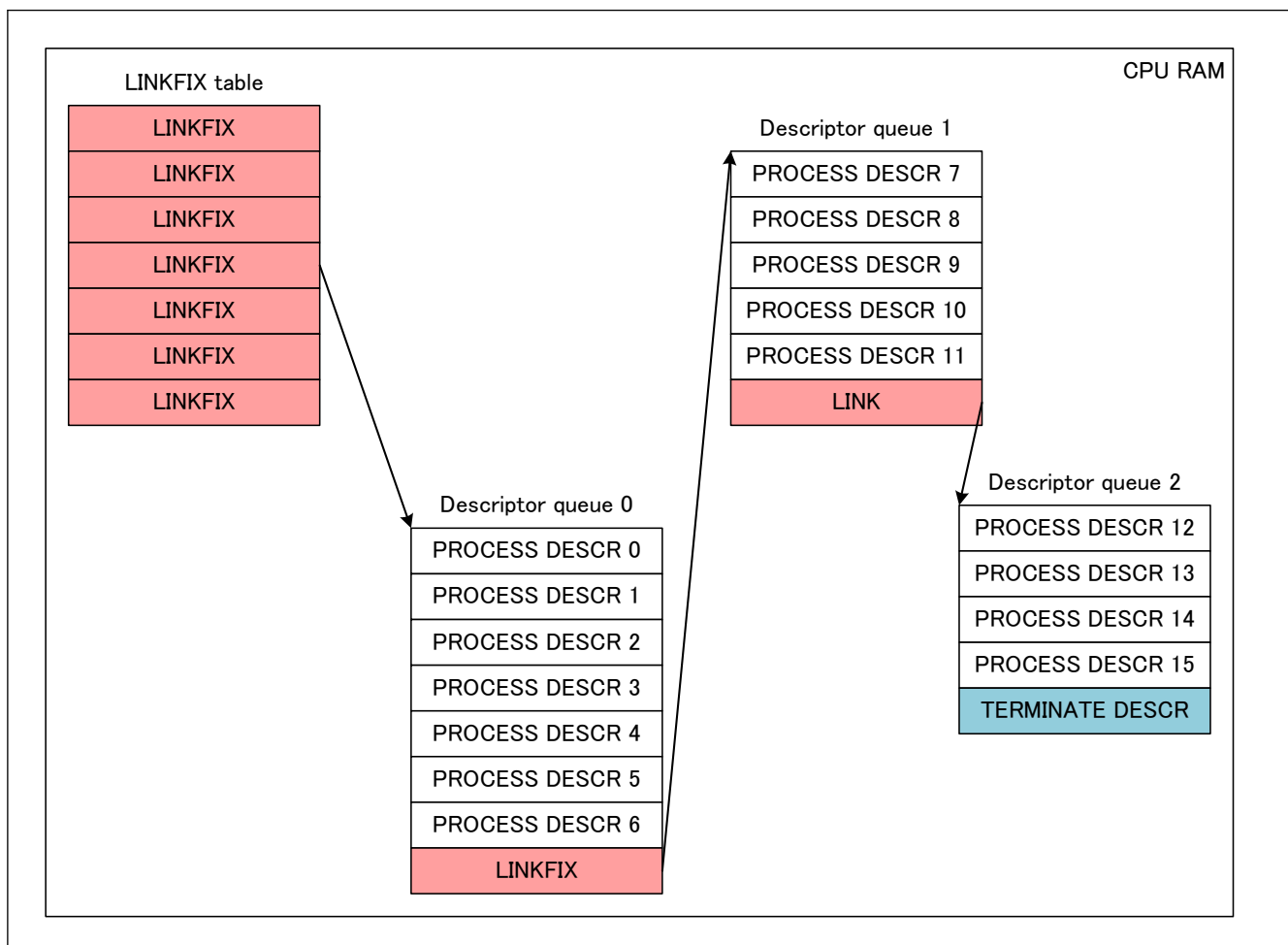
Figure 35.30 LINK utilization format

Table 35.9 LINK utilization field description

Field name	Field description written by SW	Field description after HW write-back
DS	0	0 (Value is copied from read descriptor)
INFO0	0	0 (Value is copied from read descriptor)
ERR	0	0 (Value is copied from read descriptor)
DSE	0	0 (Value is copied from read descriptor)
AXIE	0	0 (Value is copied from read descriptor)
DIE	0	0 (Value is copied from read descriptor)
DT	14	12 (Value is copied from read descriptor)
PTR	PTR (user setting) Pointer pointing to the new descriptor queue	PTR (Value is copied from read descriptor)
INFO1 (GWDCCi.EDE == 1)	0	NA: INFO1 not written back for LINK descriptors
TS (GWDCCi.ETS == 1)	0	NA: TS not written back for LINK descriptors

35.5.1.5 AXI Descriptor Queue Format

The AXI master accepts two kinds of descriptor queues: linear descriptor queues and cyclic descriptor queues. A linear descriptor queue is a queue which is ended by a terminate descriptor. A linear descriptor queue example is described in figure [Figure 35.31](#). A cyclic descriptor queue is a queue which is looping on itself. A cyclic descriptor queue example is described in figure [Figure 35.32](#).

**Figure 35.31 Linear descriptor queue example**

Restrictions:

- SW: To add data to a linear queue, the SW should write the data and process/terminate descriptor and overwrite the previous terminate descriptor with a LINK/LINKFIX descriptor pointing to the new descriptor chain.

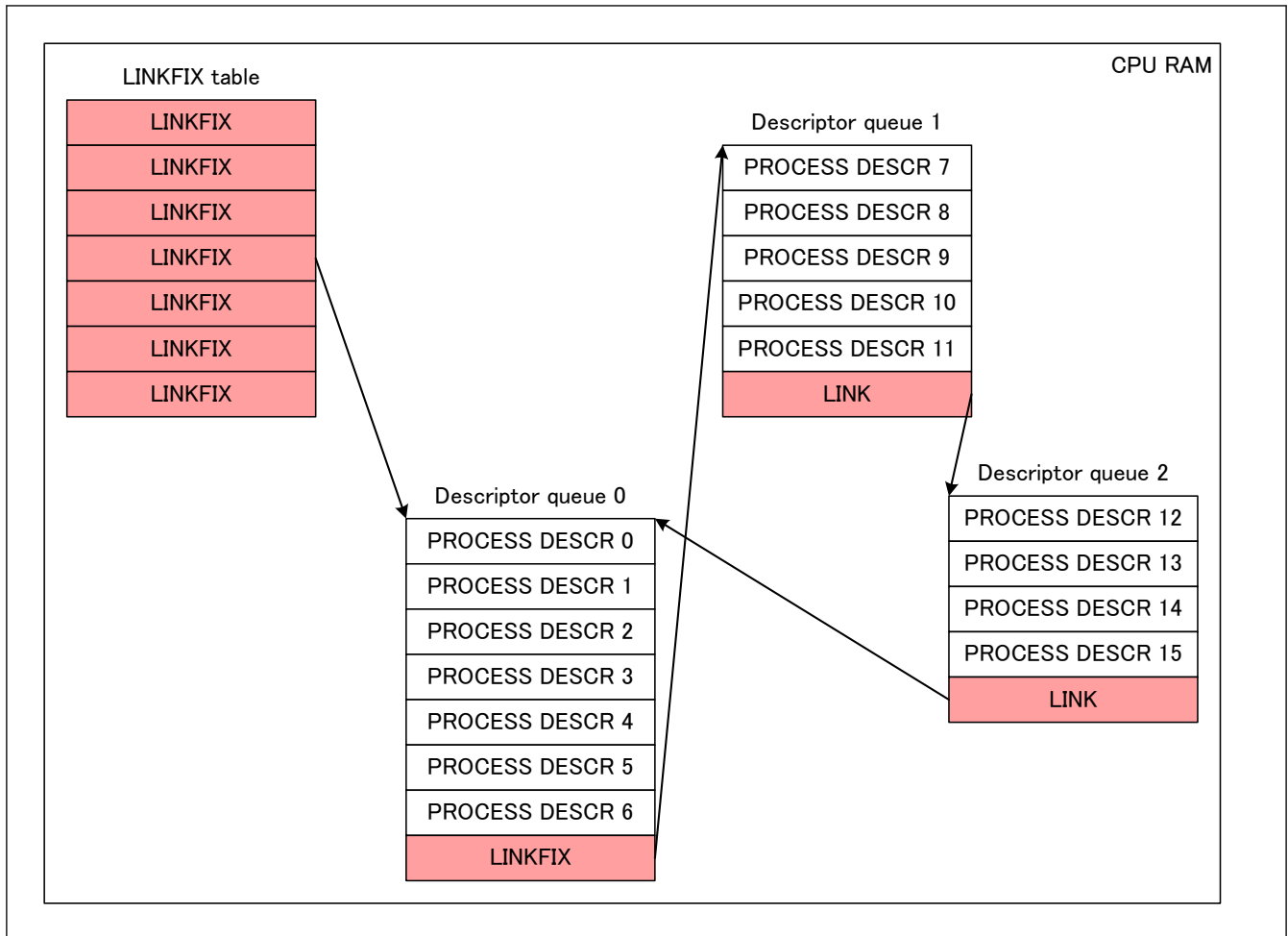


Figure 35.32 Cyclic descriptor queue example

- Note:
- The LINKFIX table is described in [section 35.5.1.3. LINKFIX Table](#).
 - The LINK, LINKFIX utilization is described in [section 35.5.1.4. AXI Common Descriptor Utilization](#).
 - A terminate descriptor (TERMINATE DESCR in [Figure 35.31](#)) correspond to a FSINGLE or a LEMPTY descriptor for an RX queue, a FEMPTY or a LEMPTY descriptor for a TX queue and a FSINGLE or a LEMPTY descriptor for a TS queue.
 - A process descriptor (PROCESS DESCR in [Figure 35.31](#) and [Figure 35.32](#)) correspond to a FEMPTY_IS, FEMPTY_IC, FEMPTY_ND, FEMPTY, FEMPTY_START, FEMPTY_MID or FEMPTY_END descriptor for an RX queue (see [section 35.5.3.5. AXI Master Interface](#)), to a FSINGLE, FSTART, FMID, FEND or EOS descriptor for a TX queue (see [section 35.5.2.2. AXI Master Interface](#)) and to a FEMPTY_ND for a TS queue (see [section 35.5.4.2. AXI Master Interface](#)).

35.5.1.6 AXI Address RAM Searching

Searching functions is used to read entries in the AXI address RAM. [Table 35.10](#) describes register used to read an entry in the TAS RAM. [Table 35.11](#) describes the read results.

Table 35.10 AXI Address read registers

Register name	Field name/corresponding field in MAC table	Field explanation
GWAARSS.AARA	Entry address Set the descriptor queue number from which AXU address should be read.	Not present in AXI address RAM, entry will be read from this address.

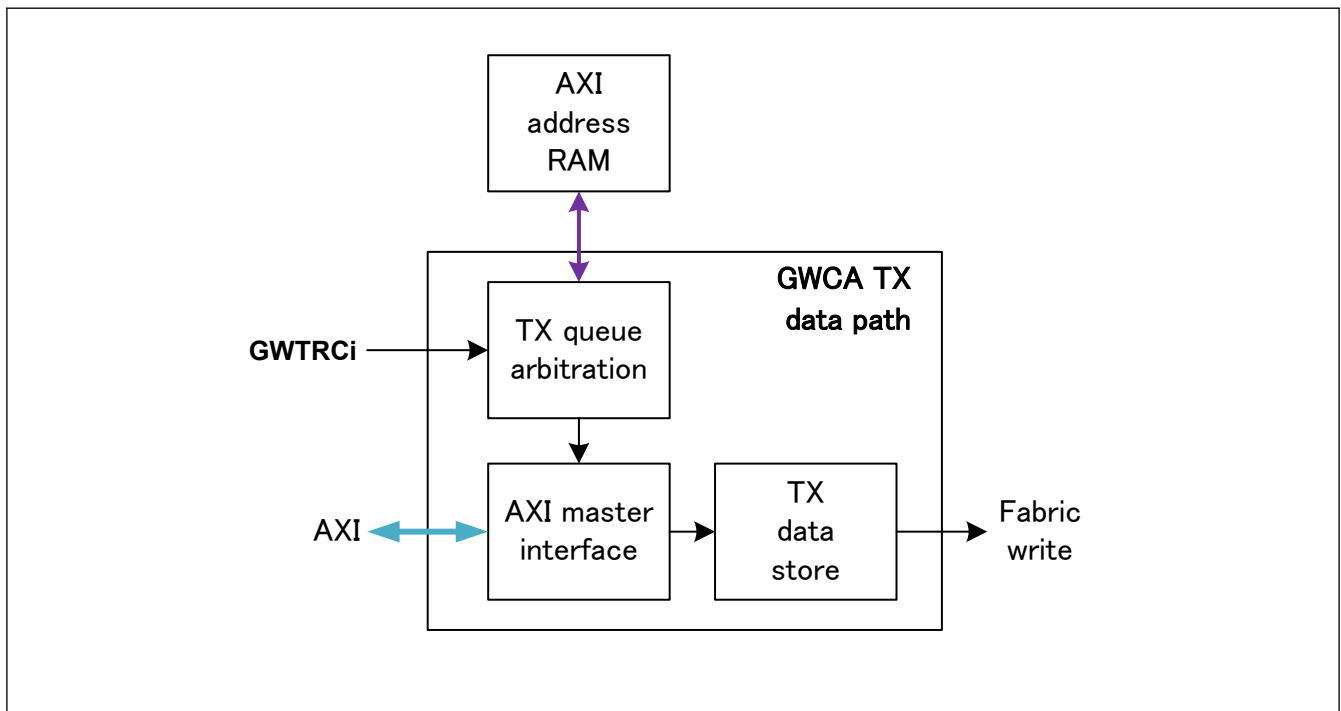
Table 35.11 AXI Address read result

Register name	Field name/corresponding field in MAC table	Field explanation
{GWAARSR0.ACARU, GWAARSR1.ACARL}	AXI address value	Address of the next descriptor to be processed for AXI descriptor queue number GWAARSS.AARA.

Note: {GWAARSR0.ACARU, GWAARSR1.ACARL} indicates the address of the next descriptor to be processed for AXI descriptor queue number but it does not mean that the previous descriptor process is completed. That's why this register only has debugging purpose and not HW/SW synchronization purposes.

35.5.2 Data Transmission

GWCA allows data transmission through the GWCA TX data path, The TX data path is described in [Figure 35.33](#).

**Figure 35.33 GWCA TX data path block diagram**

The TX data path is separated in three blocks:

- TX queue arbitration: This block arbitrates between TX queues when several queues are requesting for transmission.
- AXI master interface: This block handles the data/AXI descriptor exchange with the CPU.
- TX data store: This block extracts TAG information for frames, applies VLAN tagging and save the frames in the local RAM. This block can also release pointers while going out of OPERATION mode. The pointer release is here for switch hardware purpose and its description is not required for switch utilization so it will not be described.

35.5.2.1 TX Queue Arbitration

TX queue arbitration has four functions. Pausing certain queues when switch is about to overflow, limiting TX queue data rates, arbitrating between TX descriptor queue with a strict priority algorithm and arbitrating between same priority queues with a round-robin algorithm.

35.5.2.1.1 Per Priority Pause

Per priority pause aims at pausing descriptor queue transmission depending on their priority using PAUSE[2] values [COMA] and GWTPCp register. It avoids the AXI master to inject uncritical data in the switch when it is about to overflow.

Functions:

- GWTPCp sets the priorities that should be paused when PAUSE[2][p] is set.

35.5.2.1.2 Rate Limiters

Rate limiters are controlling the input flow of the switch. Using these limiters could prevent the switch from overflowing. GWCA has two types of rate limiters, a global rate limiter and 8 per-queue rate limiters. By setting the rate limiter configurations correctly, expected bandwidth with a tolerance value of less than $\pm 2\%$ can be achieved.

(1) Per-queue rate-limiter

A per-queue rate-limiter aims at limiting the data throughput per one descriptor queue using GWRLCi.RLE, GWRLCi.RLIV and GWRLULCi.RLUL registers. For per-port rate limiter/descriptor queue mapping information see [section 35.5.1.1. AXI Descriptor Queue Mapping](#).

Functions:

- GWRLCi.RLE register enables rate-limiter number i.
- GWRLCi.RLIV register sets the maximum throughput accepted by rate limiter number i and should be set respecting equation (1).
- GWRLULCi.RLUL register sets the maximum collision time accepted by rate limiter number i and should be set respecting equation (2).

$$1. \text{GWRLCi.RLIV} = 256 \times \text{ACLK_period[ns]} \times \text{maxBandwidth[Gbps]}$$

$$2. \text{GWRLULCi.RLUL} = \text{maximumBurst[bit]}$$

Where:

- *ACLK_period* is ACLK clock period.
- *maxBandwidth* is the expected maximum throughput accepted by rate limiter number i.
- *maximumBurst* is the maximum burst in bit accepted for corresponding queue.

(2) Global rate limiter

The global rate limiter aims at limiting all AXI data transmission throughput using GWGRLC.GRLE, GWGRLC.GRLIV and GWGRULC.GRLUL registers and can be monitored using GWGRLC.GRLULRS.

Functions:

- GWGRLC.GRLE register enables the global rate limiter.
- GWGRLC.GRLIV register sets the maximum throughput accepted by AXI data transmission and should be set respecting equation (1).
- GWGRULC.GRLUL register sets the maximum number of credits that the global rate limiter can stack and should be set respecting equation (2).
- GWGRLC.GRLULRS register shows if some credit has been lost because of a too high AXI latency or because of a too slow AXI.

$$1. \text{GWGRLC.GRLIV} = 256 \times \text{ACLK_period[ns]} \times \text{max_bandwidth[Gbps]}$$

$$2. \text{GWGRULC.GRLUL} = \text{ACLK_period[ns]} \times \text{max_bandwidth[Gbps]} \times 2 \times \text{maximum_axi_latency[cycle]}$$

Where:

- *ACLK_period* is ACLK clock period in ns.
- *max_bandwidth* is the expected maximum throughput accepted by GWCA.
- *maximum_axi_latency* is the AXI maximum latency in ACLK clock number.

Restrictions:

- SW: The sum of all GWRLCi.RLIV registers should always be smaller or equal to GWGRLC.GRLIV.

35.5.2.1.3 Strict Priority

Strict priority arbitrates between TX queue which are authorized by rate limiters to transmit using GWDCCi.DCP. The strict priority algorithm operates as the one in the data reception descriptor store. See section (1) [Strict priority \(SP\)](#) for more information.

Functions:

- GWDCCi.DCP register sets descriptor queue i priority. The highest priority is the biggest value.

35.5.2.1.4 Round Robin

Round robin arbitrates between the highest priority queues selected by the strict arbiter. The round robin operates as the one in the data reception descriptor store. See (2) [Round Robin \(RR\)](#) for more information.

35.5.2.2 AXI Master Interface

The AXI master interface handles the data/AXI descriptor exchange with the CPU. It receives the queue number of the descriptor queue number authorized to transmit by the TX queue arbitration, reads the corresponding data from the CPU RAM, and sends the data to TS data store. To do so, it uses GWTRCi.TSRj, GWDCCi.EDE, GWDCCi.ETS, and GWDCCi.SM, and can be monitored using GWAARSS, GWAARSR0, and GWAARSR1 interrupt registers.

Functions:

- GWTRCi.TSRj register is used to start transmission when a descriptor queue is ready.
- GWDCCi.EDE and GWDCCi.ETS registers set the descriptor format that will be used for descriptor queue number i. See [section 35.5.1.2.1. General Formats](#).
- GWDCCi.SM register sets how GWCA will write back descriptors. In this section all explanation will use GWDCCi.SM equal to 00. For GWDCCi.SM equal to 01 or 10 see the register explanation.
- For GWAARSS, GWAARSR0, and GWAARSR1 registers, see [section 35.5.1.6. AXI Address RAM Searching](#).

Restrictions:

- SW: Only basic descriptors and extended descriptors are authorized for data transmission. See [section 35.5.1.2.1. General Formats](#).

Data transmission can be done using linear or cyclic descriptor queues (see [section 35.5.1.5. AXI Descriptor Queue Format](#)). Data transmission process descriptors can be set two different ways, through the basic data transmission descriptor chain ([section 35.5.2.2.1. Basic Data Transmission](#)) and the time-controlled data transmission descriptor chain ([section 35.5.2.2.2. Time-controlled Data Transmission](#)).

35.5.2.2.1 Basic Data Transmission

Basic data transmission process descriptor utilization is described in [Figure 35.34](#). Basic data transmission process descriptor format is described in [Figure 35.35](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.12](#).

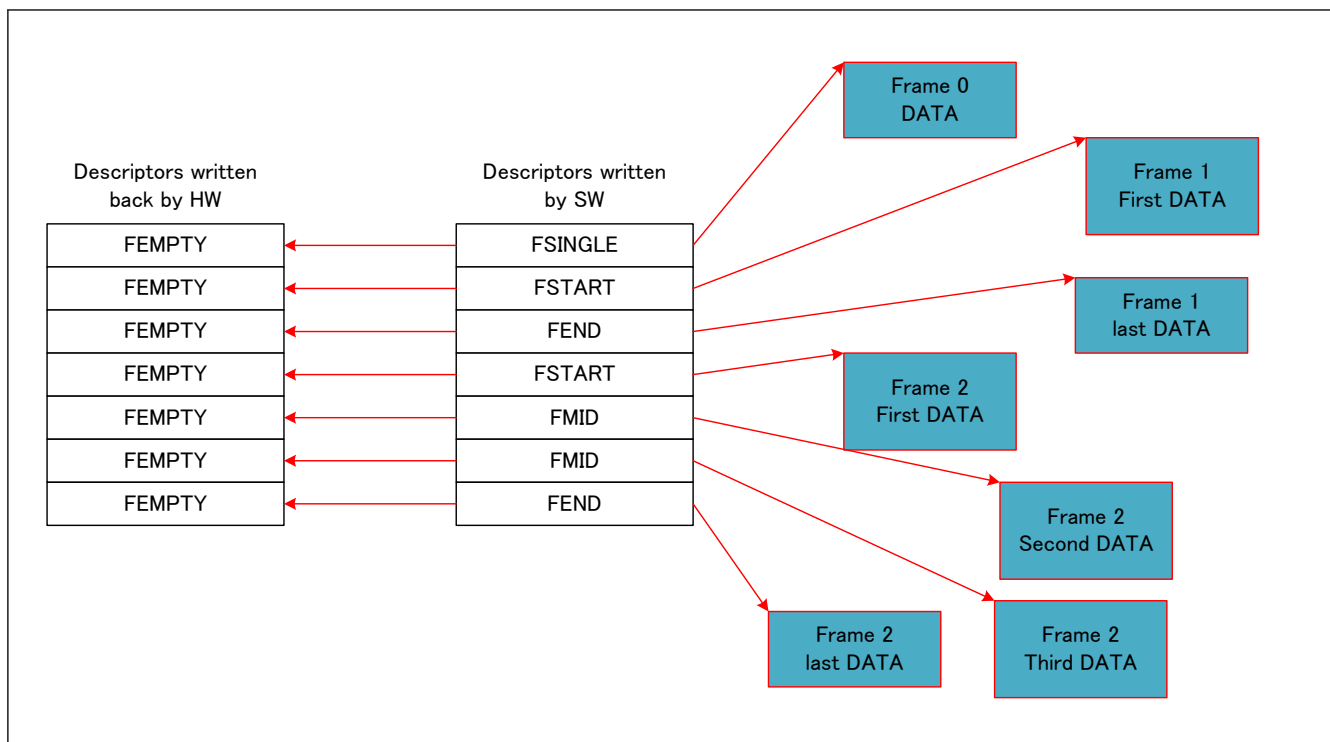


Figure 35.34 Data transmission process descriptor utilization

- Note:
- If needed, the SW can split a frame between several descriptors. The first descriptor should be a FSTART descriptor, the last descriptor should be a FEND descriptor and all other descriptors should be FMID descriptors.
 - When data is ready, CPU should set the corresponding GWTRCi.TSRj bit to start transfer. See GWTRCi.TSRj description.
 - GWTRCi.TSRj can be set anytime even if data is not available. The GWCA will in this case read the terminate descriptor and clear GWTRCi.TSRj bit.

Restrictions:

- SW: When a frame is split, the FSTART descriptor should be written after FMIDs and FEND descriptor for a given frame.

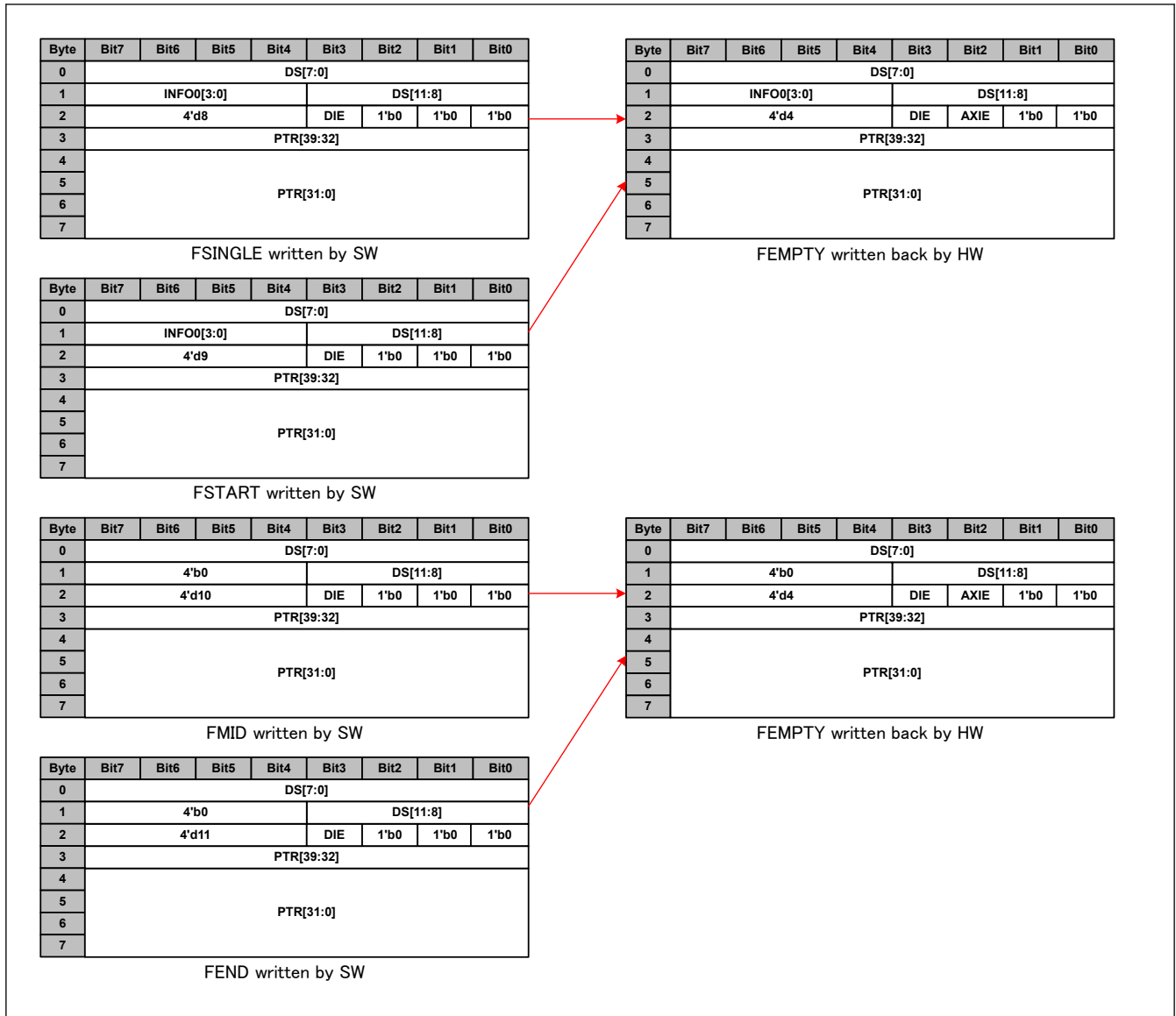


Figure 35.35 Data transmission process descriptor utilization format

Table 35.12 Data transmission process descriptor utilization field description (1 of 2)

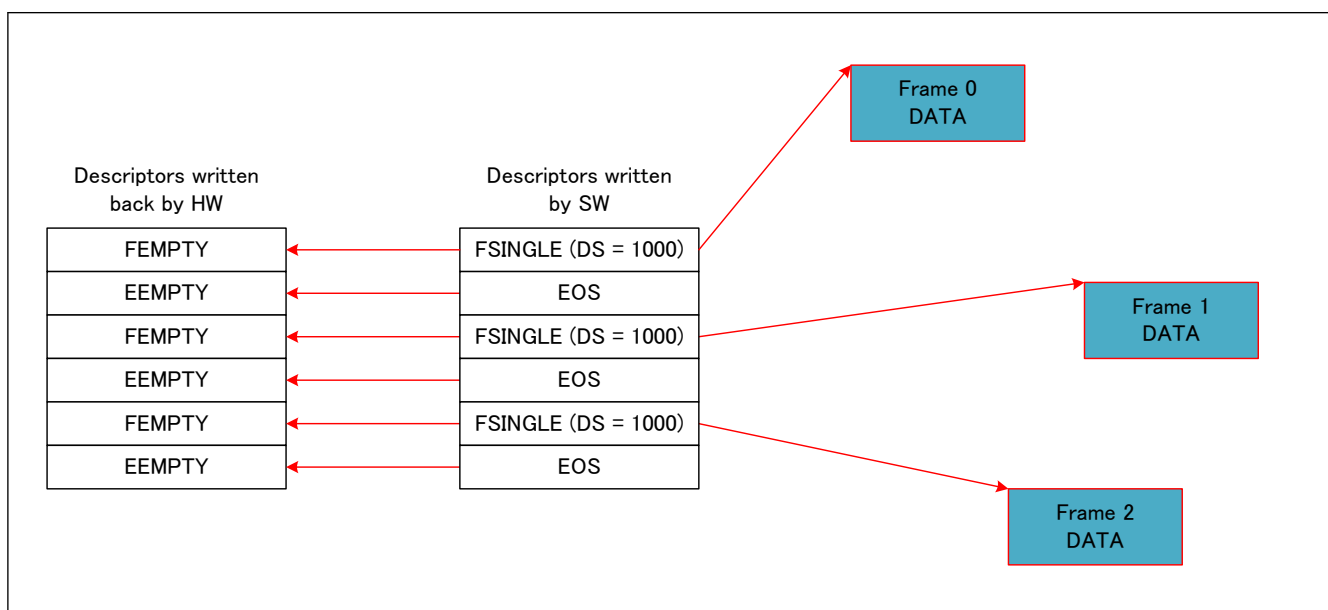
Field name	Field description written by SW	Field description after HW write-back
DS	Data what has been written to the URAM by SW	DS (Value is copied from read descriptor)
INFO0	For FSINGLE and FSTART see section 35.5.2.2.3. Data Transmission Common Descriptor Format 0 for FMID and FEND.	INFO0 (Value is copied from read descriptor)
ERR	0	0
DSE	0	0
AXIE	0	Set conditions: HW: An AXI error happened while reading a data (RRESP != 00).
DIE	DIE	DIE (Value is copied from read descriptor)
DT	8 for FSINGLE 9 for FSTART 10 for FMID 11 for FEND	4 for FEMPTY
PTR	Address where data is written in the URAM	PTR (Value is copied from read descriptor)

Table 35.12 Data transmission process descriptor utilization field description (2 of 2)

Field name	Field description written by SW	Field description after HW write-back
INFO1 (GWDCCi.EDE == 1)	For FSINGLE and FSTART see section 35.5.2.2.3. Data Transmission Common Descriptor Format 0 for FMID and FEND.	NA: INFO1 not written back for transmission
TS (GWDCCi.ETS == 1)	NA: No timestamp descriptor for transmission	NA: No timestamp descriptor for transmission

35.5.2.2.2 Time-controlled Data Transmission

Time-controlled data transmission process descriptor utilization is described in [Figure 35.36](#). Time-controlled data transmission process descriptor format is described in [section 35.4.2.11. Global Rate Limiter Setting Flow](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.13](#). FSINGLE, FSTART, FMID, FEND and FEMPTY formats are not described in this section, for their description see [Figure 35.35](#) and [Table 35.12](#).

**Figure 35.36 Data transmission process descriptor utilization**

- Note:
- If needed, the SW can split a frame between several descriptors. The first descriptor should be a FSTART descriptor, the last descriptor should be a FEND descriptor and all other descriptors should be FMID descriptors.
 - When data is ready, CPU should set the corresponding GWTRCi.TSRj bit to start transfer. See GWTRCi.TSRj description.
 - Reading an EOS descriptor will have per effect to clear GWTRCi.TSRj bit. So, if the CPU set GWTRCi.TSRj bit at a cyclic timing, CPU can control the data throughput (If TSR is already set when the CPU tries to set it the next time, only one frame will be sent, so a decrease in throughput can happen).
 - GWTRCi.TSRj bit can be set automatically using gPTP timer [gPTP]. See [section 35.5.5. gPTP Triggered Transmission](#).
 - GWTRCi.TSRj can be set anytime even if data is not available. The GWCA will in this case read the terminate descriptor and clear GWTRCi.TSRj bit.

Restrictions:

- SW: When a frame is split, the FSTART descriptor should be written after FMIDs and FEND descriptor for a given frame.

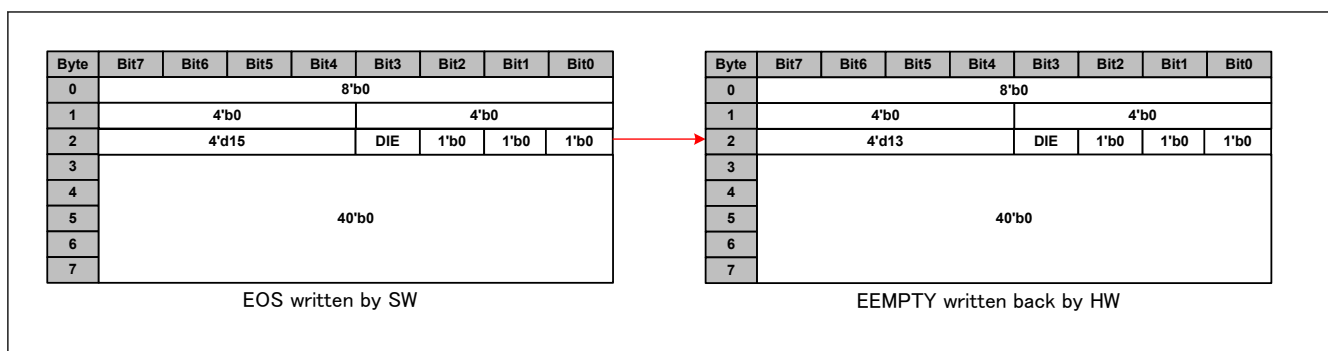


Figure 35.37 Time-controlled data transmission process descriptor utilization format

Table 35.13 Time-controlled data transmission process descriptor utilization field description

Field name	Field description written by SW	Field description after HW write-back
DS	0	DS (Value is copied from read descriptor)
INFO0	0	INFO0 (Value is copied from read descriptor)
ERR	0	0
DSE	0	0
AXIE	0	0
DIE	DIE	DIE (Value is copied from read descriptor)
DT	15	13
PTR	0	PTR (Value is copied from read descriptor)
INFO1 (GWDCCi.EDE == 1)	0	NA: INFO1 not written back for EOS descriptors
TS (GWDCCi.ETS == 1)	NA: No timestamp descriptor for transmission	NA: No timestamp descriptor for transmission

35.5.2.2.3 Data Transmission Common Descriptor Format

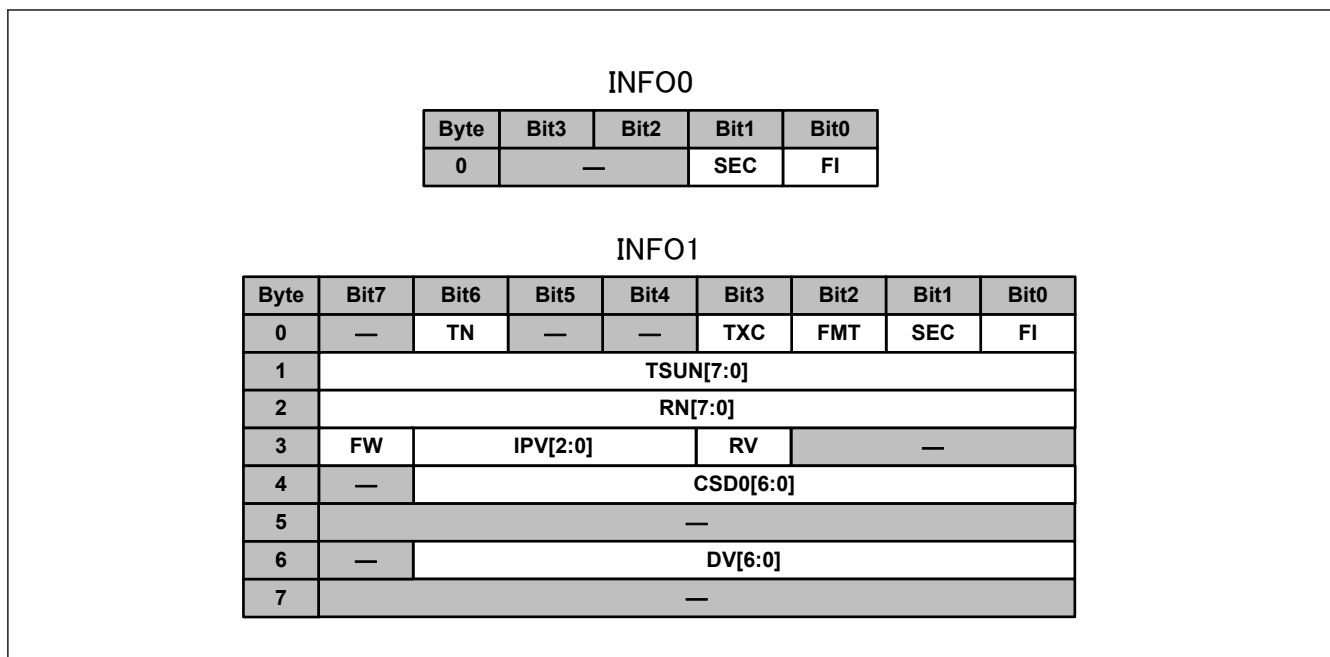
In this section are described the fields which are common for any data transmission (INFO0 and INFO1). There are two types of formats, the transmission direct descriptor and the transmission ethernet descriptor formats. The transmission direct descriptor could be used, for example, for non-routable protocol forwarding (network control) or for CPU-CPU communication and the transmission ethernet descriptor could be used for all normal ethernet transmissions. All the fields contained in AXI descriptors, if quoted, will be written ADESCR.{Field name}.

(1) Transmission direct descriptor

Transmission direct descriptor is used by a CPU to send frames by deciding their target directly (Forwarding engine processing will be by-passed [FWD]). Transmission direct descriptor INFO formats are described in [Figure 35.38](#) and [Table 35.14](#).

Restrictions:

- SW: The Transmission direct descriptor can only be and extended descriptor, see [section 35.5.1.2.1. General Formats](#).

**Figure 35.38** Transmission direct descriptor INFO formats**Table 35.14** Transmission direct descriptor INFO field descriptions

Field name	Bit width	Description
FI	1	FCS in.
SEC	1	Secure descriptor
FMT	1	Descriptor format Restrictions: • SW: Fixed to 1 for Direct descriptor.
TXC	1	TX Timestamp capture [RMAC]. Used to set corresponding bit in RMAC TX interface.
TN	1	Timer utilized for capture [RMAC]. Used to set corresponding bit in RMAC TX interface.
TSUN	8	Timestamp unique number [RMAC]. Used to set corresponding bit in RMAC TX interface.
RV	1	Routing valid. See section 35.5.3.2. L2/L3 Update , to ethernet agent L2/3 update [ETHA] and to forwarding engine L3 routing table [FWD].
RN	5	Routing valid. See section 35.5.3.2. L2/L3 Update , to ethernet agent L2/3 update [ETHA] and to forwarding engine L3 routing table [FWD].
IPV	3	Internal priority value Target priority of the frame.
FW	1	The FCS contained in the frame is wrong (It will be corrected in the switch).
CSD0	6	CPU sub destination for GWCA0
DV	4	Destination vector [MFWD]
—	—	Reserved area Restrictions: • SW: these fields should be set to 0.

(2) Transmission ethernet descriptor

Transmission ethernet descriptor is used by a CPU to send frames that will be forwarded by the Forwarding engine [FWD]. Transmission ethernet descriptor INFO formats are described in [Figure 35.39](#) and [Table 35.15](#).

Restrictions:

- SW: Because FW field (see [\(1\) Transmission direct descriptor](#)) is not present in an Ethernet descriptor. Any transmit frame should have either a correct FCS or no FCS.

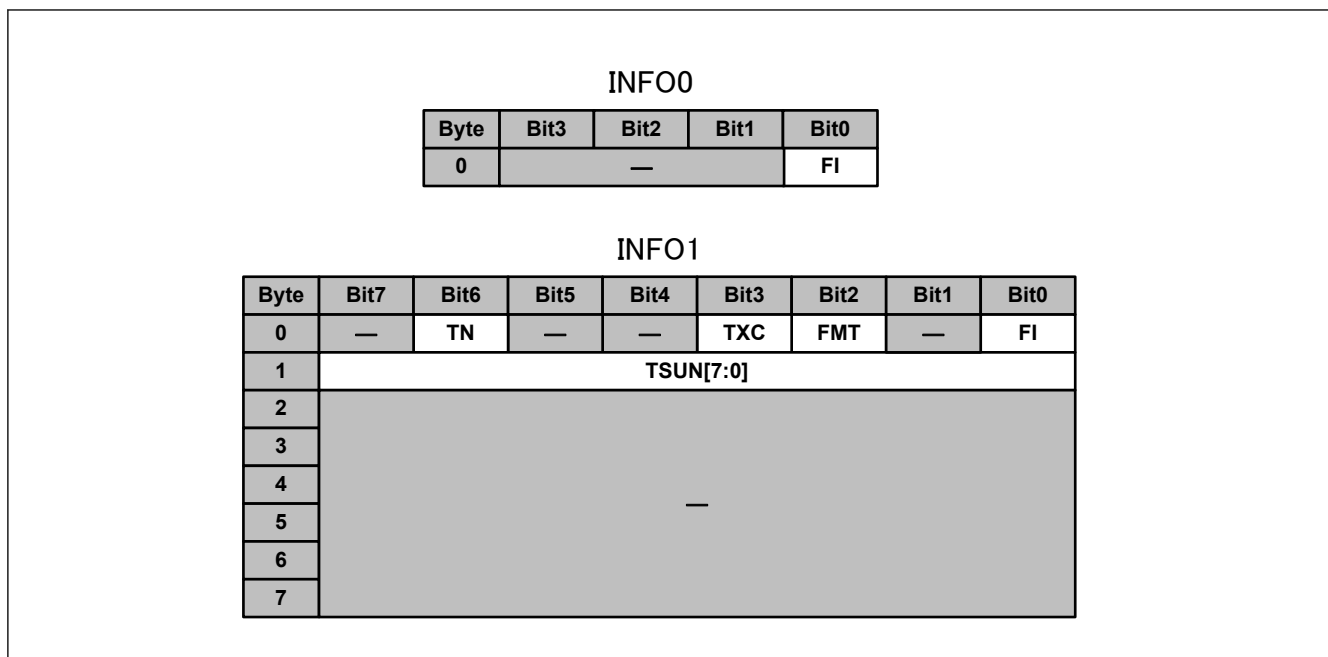


Figure 35.39 Transmission ethernet descriptor INFO formats

Table 35.15 Transmission ethernet descriptor INFO field descriptions

Field name	Bit width	Description
FI	1	FCS in.
FMT	1	Descriptor format Restrictions: SW: Fixed to 0 for Ethernet descriptors.
TXC	1	TX Timestamp capture [RMAC]. Used to set corresponding bit in RMAC TX interface.
TN	1	Timer utilized for capture [RMAC]. Used to set corresponding bit in RMAC TX interface.
TSUN	8	Timestamp unique number [RMAC]. Used to set corresponding bit in RMAC TX interface.
—	—	Reserved Restrictions: SW: these fields should be set to 0.

35.5.2.3 TX Data Store

TX data store has follow functions, it extracts TAG information for frames, applies VLAN tagging, and save the frames in the local RAM. These functions are applied to all frames independently from their descriptor format.

35.5.2.3.1 TAG Information Extraction

TAG information extraction is done in HW without setting intervention. The extracted format can be filtered using GWTTFC register. TAG information extraction is described in [Figure 35.40](#) along with GWTTFC register bit explanation for TAG filtering. The extracted fields are SN, CoS-TCI, C-TCI and S-TCI.

Data														TAG type	Filtered by GWTTFC.
XX	XX	Type	MAC source						MAC Destination					No TAG	NT
XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX		
00	00	C1	F1	MAC source						MAC Destination				R-TAG	RT
XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	Type	SN			
CoS-TCI	00	81	MAC source						MAC Destination					CoS-TAG	CST
XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	Type			
CoS-TCI	00	81	MAC source						MAC Destination					CoSR-TAG	CSRT
XX	XX	XX	XX	XX	XX	XX	Type	SN	00	00	C1	F1			
C-TCI	00	81	MAC source						MAC Destination					C-TAG	CT
XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	Type			
C-TCI	00	81	MAC source						MAC Destination					CR-TAG	CRT
XX	XX	XX	XX	XX	XX	XX	Type	SN	00	00	C1	F1			
S-TCI	A8	88	MAC source						MAC Destination					SC-TAG	SCT
XX	XX	XX	XX	XX	XX	XX	XX	XX	Type	CCs-TCI	00	81			
S-TCI	A8	88	MAC source						MAC Destination					SCR-TAG	SCRT
XX	XX	XX	XX	Type	SN	00	00	C1	F1	CCs-TCI	00	81			
Any other format														Unknown TAG	UT

Figure 35.40 TAG information extraction

- Note:
- Only colored part in are considered for TAG information extraction.
 - HF1C1 corresponds to R-TAG TPID and SN correspond to R-TAG sequence number [802.1CB].
 - H8100 corresponds to C-TAG TPID, C-TCI corresponds to a C-TAG TCI with a non-null VID, CoS-TCI corresponds to a C-TAG TCI with a null VID and CCs-TCI corresponds to C-TCI or CoS-TCI [802.1Q].
 - H88A8 corresponds to S-TAG TPID and S-TCI corresponds to an S-TAG TCI [802.1Q].
 - Type corresponds to any Ethernet type value which is not HF1C1, H88A8 or H8100.
 - TPID values can be configured in Forwarding Engine [FWD]. Values for TPIDs can be set in forwarding engine (in FWTTTC0 and FWTTTC1 registers).

Restrictions:

- HW: Because 32 bytes are needed for TAG extraction, GWCA will reject transmit frames smaller than 32 bytes and set GWEIS0.UsMFSES interrupt register.
- SW: In R-TAG, the reserved part should always be set to 0. If the reserved part is not set to 0 and corresponding frame contains an FCS, the frame could be forwarded with an error.

35.5.2.3.2 VLAN Tagging

VLAN tagging replaces or overwrites present TAGs in a frame using the format given by TAG information extraction (section 35.5.2.3.1. TAG Information Extraction), switch VLAN mode (FWGC.SVM register in MFWD) and, GWVCC and GWVTC registers. After VLAN tagging, any frame has three TAGs, one R-TAG, one C-TAG and one S-TAG which will be directly saved in the TAG RAM [MFAB].

Functions:

- GWVCC.VIM register sets the TX data path in Port based VLAN (all incoming frames are sent to the forwarding engine with the same VLAN ID). GWVCC.VIM setting is ignored when the switch in is No VLAN mode (FWGC.SVM register in forwarding engine set to 00) [MFWD].
- GWVTC register is used to set the HW TAGs. There are three possible HW TAGs. The HW C-TAG made from GWVTC.CTV as VID, GWVTC.CTP as PCP and GWVTC.CTD as DEI. The HW S-TAG made from GWVTC.STV as VID, GWVTC.STP as PCP and GWVTC.STD as DEI. The HW CoS-TAG made from GWVTC.CTV as VID, the incoming frame PCP as PCP and the incoming frame DEI as DEI.

TAGs after VLAN tagging are described in [Figure 35.41](#), [Figure 35.42](#) and [Figure 35.43](#) depending on Switch VLAN mode (FWGC.SVM register in MFWD) and the type of frame given by TAG information extraction ([section 35.5.2.3.1. TAG Information Extraction](#)). In the figures, the data extracted from the incoming frames are in blue and the added HW TAGs are in red.

TAG type	R-TAG	C-TAG	S-TAG
No TAG	0	0	0
R-TAG	SN	0	0
CoS-TAG	0	CoS-TCI	0
CoSR-TAG	SN	CoS-TCI	0
C-TAG	0	C-TCI	0
CR-TAG	SN	C-TCI	0
SC-TAG	0	C-TCI	S-TCI
SCR-TAG	SN	C-TCI	S-TCI
Unknown TAG	0	0	0

Figure 35.41 TAGs after VLAN tagging: Switch in No-VLAN mode [MFWD]

TAG type	Incoming VLAN mode (GWVCC.VIM == 1'b0)			Port based VLAN mode (GWVCC.VIM == 1'b1)		
	R-TAG	C-TAG	S-TAG	R-TAG	C-TAG	S-TAG
No TAG	0	HW C-TAG	0	0	HW C-TAG	0
R-TAG	SN	HW C-TAG	0	SN	HW C-TAG	0
CoS-TAG	0	HW CoS-TAG	0	0	HW CoS-TAG	0
CoSR-TAG	SN	HW CoS-TAG	0	SN	HW CoS-TAG	0
C-TAG	0	C-TCI	0	0	HW C-TAG	0
CR-TAG	SN	C-TCI	0	SN	HW C-TAG	0
SC-TAG	0	C-TCI	S-TCI	0	HW C-TAG	0
SCR-TAG	SN	C-TCI	S-TCI	SN	HW C-TAG	0
Unknown TAG	0	HW C-TAG	0	0	HW C-TAG	0

Figure 35.42 TAGs after VLAN tagging: Switch in C-TAG mode [MFWD]

TAG type	Incoming VLAN mode (GWVCC.VIM == 1'b0)			Port based VLAN mode (GWVCC.VIM == 1'b1)		
	R-TAG	C-TAG	S-TAG	R-TAG	C-TAG	S-TAG
No TAG	0	HW C-TAG	HW S-TAG	0	HW C-TAG	HW S-TAG
R-TAG	SN	HW C-TAG	HW S-TAG	SN	HW C-TAG	HW S-TAG
CoS-TAG	0	HW CoS-TAG	HW S-TAG	0	HW CoS-TAG	HW S-TAG
CoSR-TAG	SN	HW CoS-TAG	HW S-TAG	SN	HW CoS-TAG	HW S-TAG
C-TAG	0	C-TCI	HW S-TAG	0	C-TCI	HW S-TAG
CR-TAG	SN	C-TCI	HW S-TAG	SN	C-TCI	HW S-TAG
SC-TAG	0	C-TCI	S-TCI	0	C-TCI	HW S-TAG
SCR-TAG	SN	C-TCI	S-TCI	SN	C-TCI	HW S-TAG
Unknown TAG	0	HW C-TAG	HW S-TAG	0	HW C-TAG	HW S-TAG

Figure 35.43 TAGs after VLAN tagging: Switch in SC-TAG mode [MFWD]

35.5.2.3.3 Frame Saving in Local RAM

For frame saving in the local RAM, see Fabric specification document [MFAB], section “Data representation in RAMs”. Only the frame descriptor mentioned in Fabric specification document [MFAB] will be described in this section.

There are two types of Local RAM descriptors, direct local descriptors and the ethernet local descriptors. They are respectively created from direct descriptors and ethernet descriptors, see [section 35.5.2.2.3. Data Transmission Common Descriptor Format](#).

All the fields in these descriptors, if quoted, will be written LDESCR.{Field name}.

(1) Direct local descriptor

Direct local descriptor format is described in [Figure 35.44](#) and [Table 35.16](#).

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	—	TN	CRT	IET	TXC	FMT	SEC	FI
1	TSUN[7:0]							
2	SAEF[7:0]							
3	—				RTGI	VCTRL[2:0]		
4	RN[7:0]							
5	FW	IPV[2:0]			RV	—		
6	TPL[15:0]							
7								
8	—	CSD0[6:0]						
9	—	CSD1[6:0]						
10	—	DV[6:0]						
11	—							
12								
13								
14								
15								

Figure 35.44 Direct local descriptor format

Table 35.16 Direct local descriptor field description

Field name	Bit width	Description	Value for GWCA
FI	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
SEC	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
FMT	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
TXC	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
IET	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
CRT	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
TN	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
TSUN	8	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
SAEF	8	Source agent error flags	See (3) SAEF format .
VCTRL	3	VLAN control	VCTRL[2] values: - Set to GWVCC.VIM. - VCTRL[1:0] values 00: For No TAG, R-TAG and Unknown TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 01: For C-TAG and CR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 10: For SC-TAG and SCR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 11: For CoS-TAG and CoSR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction.
RTGI	1	R-TAG in	Values: 0: For No TAG, CoS-TAG, C-TAG, SC-TAG and Unknown TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 1: For R-TAG, CoSR-TAG, CR-TAG and SCR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction.
RV	1	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
RN	5	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
IPV	3	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
FW	1	See Table 35.14 .	Copied from transmission direct descriptor except when set by checksum handling function. See (1) Transmission direct descriptor for transmission direct descriptor and see 8.2.3.4 .
TPL	16	Total payload length	Payload length of frame. It only includes the data saved in the data RAM [MFAB].
CSD0	6	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
DV	4	See Table 35.14 .	Copied from transmission direct descriptor. See (1) Transmission direct descriptor .
—	—	Reserved area	Set to 0

(2) Ethernet local descriptor

Ethernet local descriptor format is described in [Figure 35.45](#) and [Table 35.17](#).

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	—	TN	CRT	IET	TXC	FMT	—	FI
1	TSUN[7:0]							
2	SAEF[7:0]							
3	—				RTGI	VCTRL[2:0]		
4	—							
5								
6	TPL[15:0]							
7								
8	TSNS[29:0]							
9								
10								
11	TSD	TSV	TSS[31:0]					
12								
13								
14								
15								

Figure 35.45 Ethernet local descriptor format

Table 35.17 Ethernet local descriptor field description (1 of 2)

Field name	Bit width	Description	Value for GWCA
FI	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
FMT	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
TXC	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
IET	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
CRT	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
TN	1	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
TSUN	8	See Table 35.15 .	Copied from transmission ethernet descriptor. See (2) Transmission ethernet descriptor .
SAEF	8	Source agent error flags	See (3) SAEF format .
VCTRL	3	VLAN control	VCTRL[2] values: - Set to GWVCC.VIM. - VCTRL[1:0] values 00: For No TAG, R-TAG and Unknown TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 01: For C-TAG and CR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 10: For SC-TAG and SCR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 11: For CoS-TAG and CoSR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction.

Table 35.17 Ethernet local descriptor field description (2 of 2)

Field name	Bit width	Description	Value for GWCA
RTGI	1	R-TAG in	Values: 0: For No TAG, CoS-TAG, C-TAG, SC-TAG and Unknown TAG frames, see section 35.5.2.3.1. TAG Information Extraction. 1: For R-TAG, CoSR-TAG, CR-TAG and SCR-TAG frames, see section 35.5.2.3.1. TAG Information Extraction.
TPL	16	Total payload length	Payload length of frame. It only includes the data saved in the data RAM [MFAB].
TSV	1	Timestamp valid	Set to 0. Unused in GWCA.
TSD	1	Timestamp default	Set to 0. Unused in GWCA.
TSNS	30	Timestamp nanosecond [gPTP]	Set to 0. Unused in GWCA.
TSS	32	Timestamp second [gPTP] Restrictions: HW: The 16-upper bits of the gPTP second part are truncated.	Set to 0. Unused in GWCA.
—	—	Reserved area	Set to 0

(3) SAEF format

Source agent error flag format is described in [Figure 35.46](#) and [Table 35.18](#).

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	—	DNE	—	SEQE	AXE	—	—	TFE

Figure 35.46 Source agent error flag format**Table 35.18 Source agent error flag field description**

Field name	Bit width	Description
TFE	1	TAG Filtering Error Set conditions: HW: When a frame does not fit the required TAG ingress format. See GWTTFC register explanation.
AXE	1	AXI Error Set conditions: HW: When an AXIE error happened during a split frame, the frame is sent incomplete.
SEQE	1	Sequence Error Set conditions: HW: When a FSINGLE, FSTART, EOS or unknown descriptor is received during a split frame, the frame is sent incomplete.
DNE	1	Descriptor Number Error HW: When a number GWMDNC.TXDMN+1 of descriptor has been already read to fetch a frame during a split frame and the frame is not finished. The frame is sent incomplete.
—	—	Reserved Set to 0

35.5.3 Data Reception

GWCA allows data reception through the GWCA RX data path, The RX data path is described in [section 35.4.2.11. Global Rate Limiter Setting Flow](#).

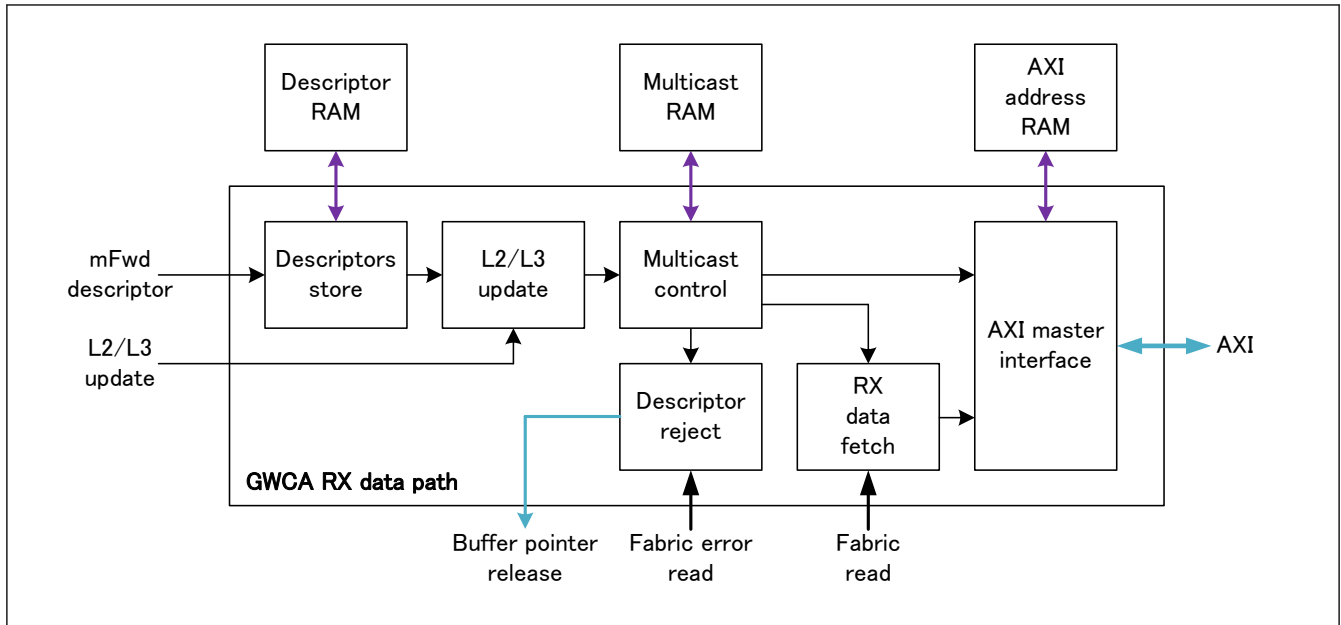


Figure 35.47 GWCA RX data path block diagram

The RX data path is separated in seven blocks:

- **Descriptor store:** This block aims at storing the descriptor received from the MFWD in the Descriptor RAM and controls the arbitration between descriptors.
- **L2/L3 update:** This block aims at fetching the L2/L3 update rules from the Forwarding Engine to update the frame while sending it.
- **Frame size check:** This block aims at checking the frame size and security level and decide to forward or discard frames.
- **Multicast control:** This block aims at checking the frame size and security level and decide to forward or discard frames and at copying the descriptors to be able to send the data at several CPU sub-destinations.
- **Descriptor reject:** This block aims at releasing the pointer of the frames rejected by Frame size control block. This block is here for switch hardware purpose and its description is not required for switch utilization so it will not be described.
- **RX data fetch:** This block fetches frames data from the local RAM, update frames depending on the information obtained by L2/L3 update module, on VLAN control information and the R-TAG information, can remove frame FCSs, and release frame pointers. The pointer release is here for switch hardware purpose and its description is not required for switch utilization so it will not be described.
- **AXI master interface:** This block handles the data/AXI descriptor exchange with the CPU.

35.5.3.1 Descriptor Store

Descriptor store has two functions. Storing the descriptors coming from the descriptor bus MFWD in the descriptor RAM and arbitrating descriptor read from the descriptor RAM to decide in which order the descriptors will be sent to the CPU.

35.5.3.1.1 Descriptor Storage

Descriptor storage controls the descriptor storage using `GWIRC.IPVRi`, `GWRDQSC.RDQSL`, `GWRDQC.RDQD`, `GWRDQC.RDQP` and `GWRDQDCq.DQD` registers and can be monitored using `GWRDQMq.DNQ` and `GWRDQMLMq.DMLQ` registers, `GWEIS1.DQOES` and `GWEIS1.DQSES` interrupt registers. [Figure 35.48](#) describes an example for Descriptor storage setting.

Functions:

- `GWIRC.IPVRi` register is used to map descriptor received from forwarding engine with a given `FDESCR.IPV` [MFWD] value to a determined descriptor queue. For example, by setting `GWIRC.IPVR6` to 1 all the descriptors received with `FDESCR.IPV` equal to 6 will be stored in descriptor queue number 1.

- GWRDQSC.RDQSLn is used to set a queue's security level. If a queue is secure (GWRDQSC.RDQSLn is set) and a non-secure descriptor is received (FDESCR.SEC [MFWD] is not set) for this queue, the descriptor will not be stored and GWEIS1.DQSESi flag will be set.
- GWRDQC.RDQP register is used to pause descriptor queues. For paused descriptor queues, the descriptor transmission to CPU will stop but the descriptor storage in the descriptor RAM will continue.
- GWRDQC.RDQD register is used to disable descriptor queues. For disabled descriptor queues and no descriptor will be stored in the Descriptor RAM.
- GWRDQDCq.DQD registers are used to set the maximum number of descriptors that can be stored in each descriptor queue. If a descriptor is received for descriptor q while queue q is full (GWRDQDCq.DQD == GWRDQMq.DNQ), the descriptor will not be stored and GWEIS1.DQOESi flag will be set.
- GWRDQMq.DNQ registers are used to monitor the current number of descriptors in each descriptor queue.
- GWRDQMLMq.DMLQ registers are used to monitor the maximum number of descriptors that has been held in each queue since the previous reset or the previous time corresponding register has been read.
- GWEIS1.DQOES register signals that descriptors have been discarded because of a descriptor queue overflow.
- GWEIS1.DQSES register signals that descriptors have been discarded because of a descriptor queue security error.

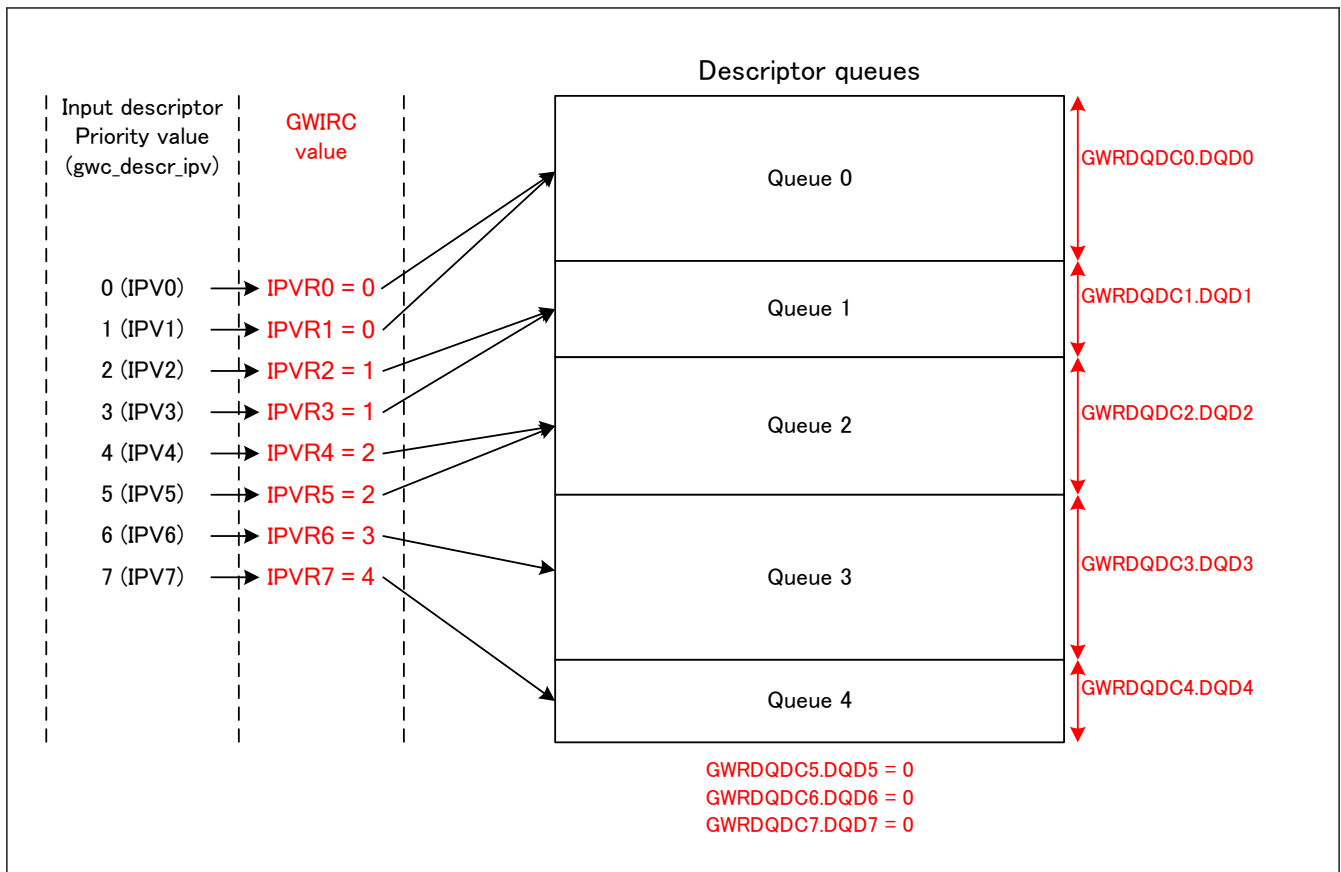


Figure 35.48 Descriptor storage setting example

35.5.3.1.2 Descriptor Arbitration

Descriptor arbitration controls the descriptor reading order for reception using GWRDQAC register. It has 4 modes. The Strict Priority, the Round Robin, the Weight Round Robin and the Hybrid mode. This function is used to ensure QoS when the AXI access allows a throughput which is too small to allow all the data to be stored in the user RAM.

(1) Strict priority (SP)

To set the descriptor arbitration in strict priority mode, all the GWRDQAC.RDQAq (q = 0 to 7) registers should be set to 0. In this case a strict priority algorithm will be applied for descriptor arbitration. The strict priority algorithm is described through an example in [Figure 35.49](#).

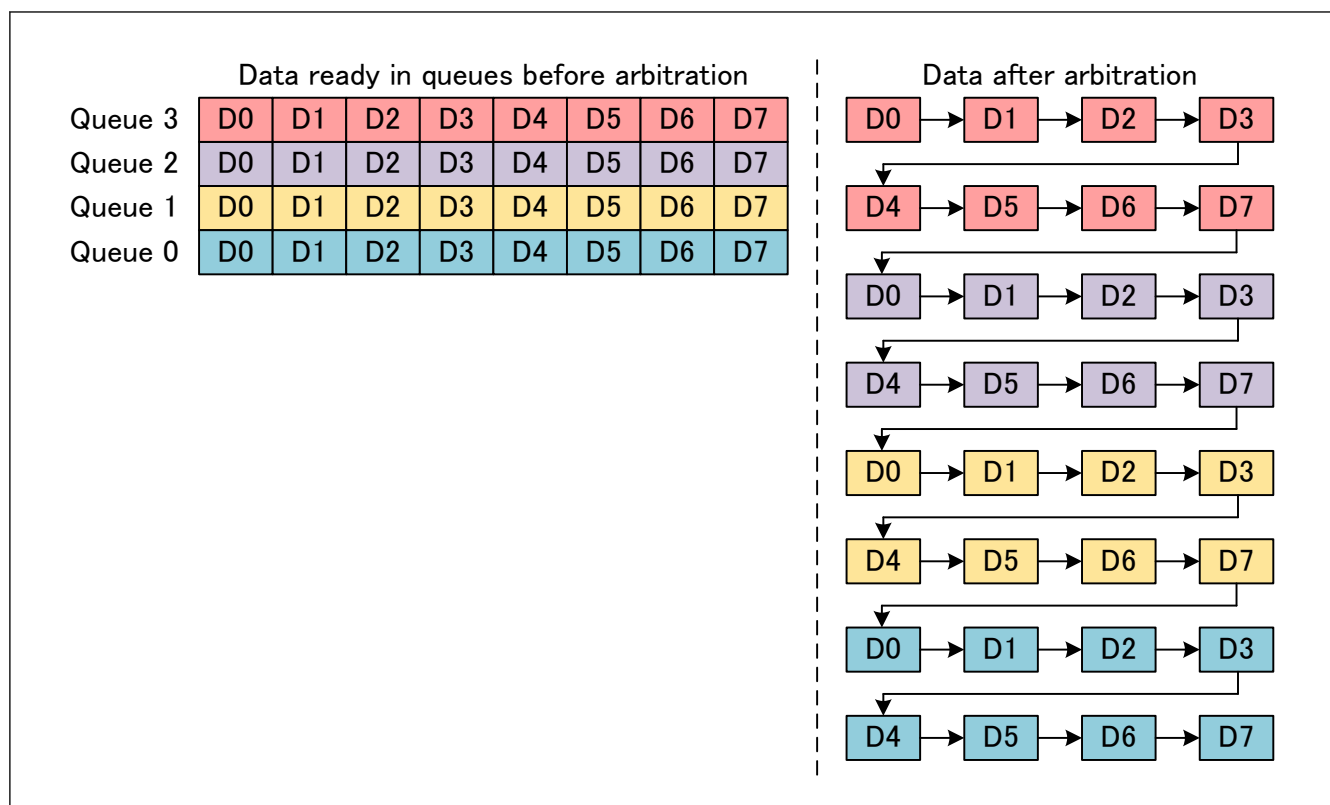


Figure 35.49 Strict priority arbitration example

(2) Round Robin (RR)

To set the descriptor arbitration in round robin mode, all the GWRDQAC.RDQAq (q = 0 to 7) registers should be set to 1. In this case a round robin algorithm will be applied for descriptor arbitration. The round robin algorithm is described through an example in [Figure 35.50](#).

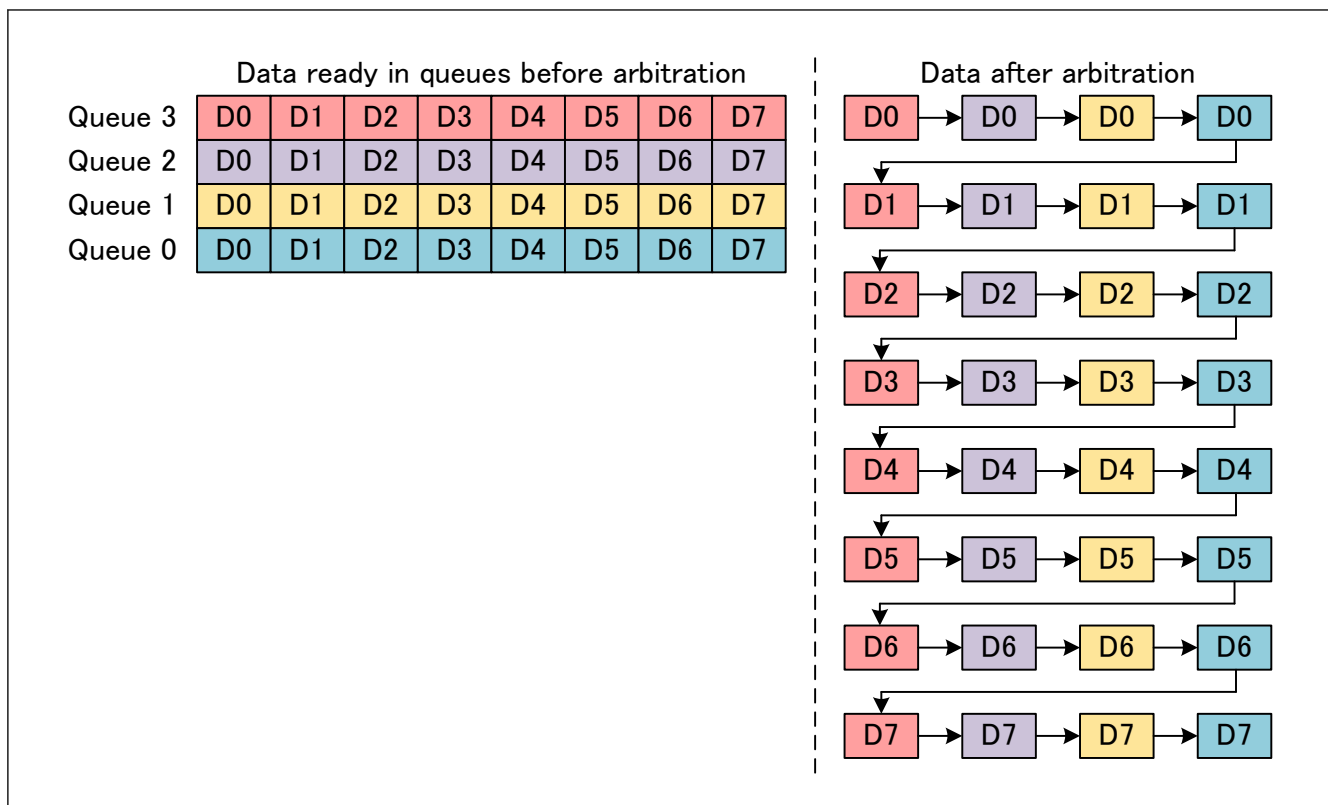


Figure 35.50 Round robin arbitration example

(3) Weight Round Robin (WRR)

To set the descriptor arbitration in weight round robin mode, all the GWRDQAC.RDQA_q ($q = 0$ to 7) registers should be set to a value different than 0 with at least one queue set to a value different than 1. In this case a weight round robin algorithm will be applied for descriptor arbitration. The weight round robin algorithm is described through an example in [Figure 35.51](#) (RDQA0 = 2, RDQA1 = 1, RDQA2 = 3 and RDQA3 = 4). In this arbitration GWRDQAC.RDQA_q register represents the weight of queue q . the heavier a queue is the more chance it will have to transmit data.

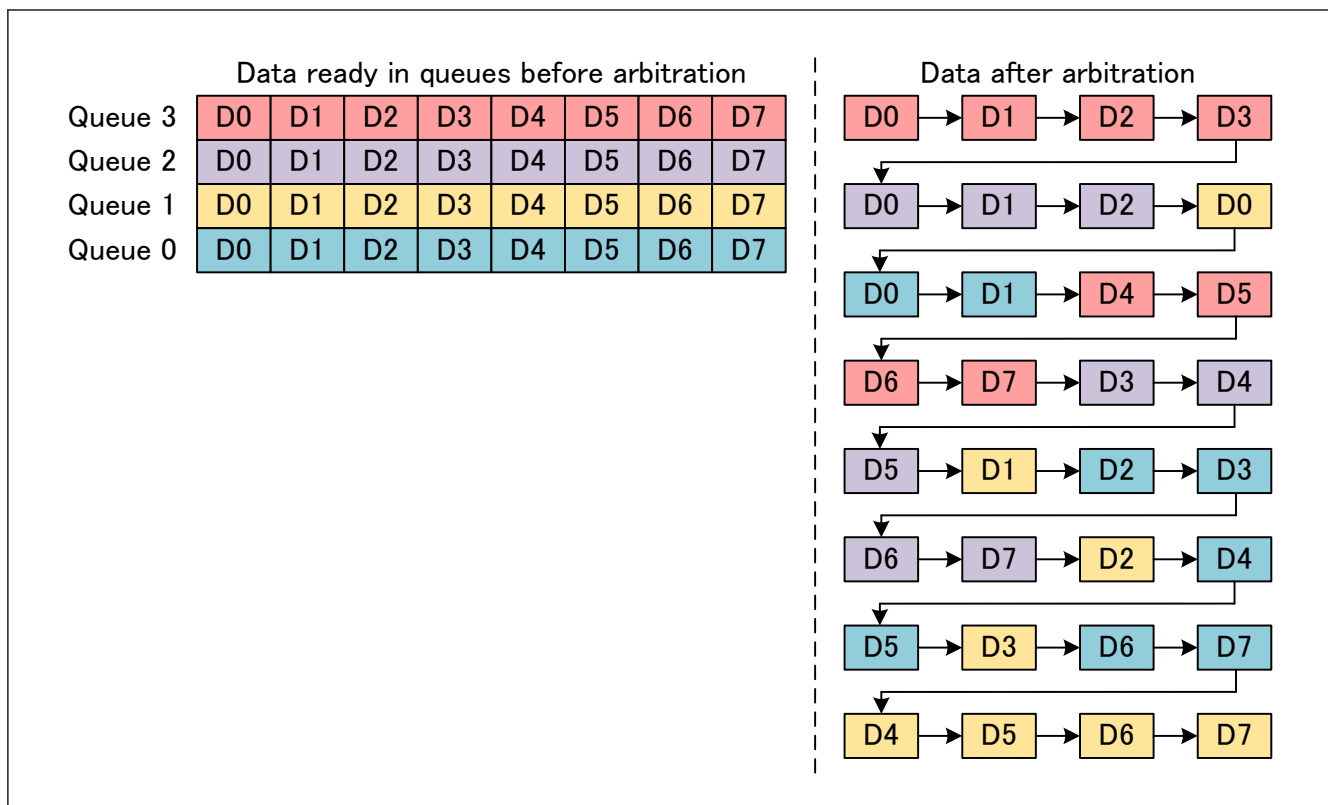


Figure 35.51 Weight round robin arbitration example

(4) Hybrid arbitration

It is allowed to use at the same time SP and RR or SP and WRR. This mode is called the hybrid arbitration mode. To set the descriptor arbitration in hybrid arbitration mode, some of the GWRDQAC.RDQA_q ($q = 0$ to 7) registers should be set to 0 and some others to a value different than 0 (For restrictions see GWRDQAC.RDQA_q register explanation). The hybrid arbitration algorithm is described through an example in [Figure 35.52](#) (RDQA0 = 0, RDQA1 = 1, RDQA2 = 2 and RDQA3 = 0).

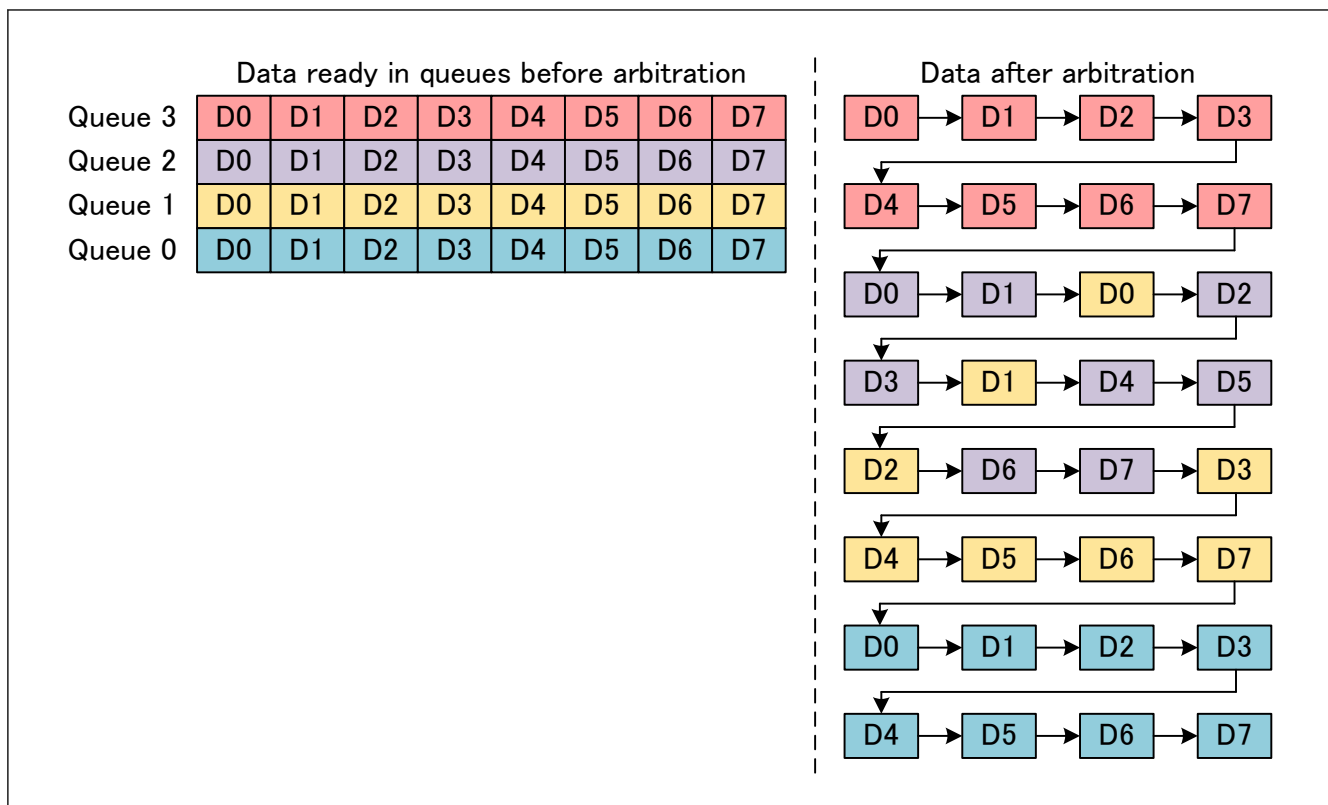


Figure 35.52 Hybrid arbitration example

35.5.3.2 L2/L3 Update

L2/L3 update aims at fetching the update rules thanks to L2/L3 update bus [MFWD].

The L2/L3 update rules will be fetched for any descriptor received with FDESCR.RV set [MFWD]. Rule number FDESCR.RN [MFWD] will be used for L2/L3 update.

Functions:

- For rule fetching setting details see Forwarding Engine specification document [MFWD].
- For frame update information see RX data fetch block explanation in [section 35.5.3.4. RX Data Fetch](#).

35.5.3.3 Multicast Control

Multicast function aims at copying the same frame to several AXI descriptor queue. It allows different CPU to process same frames at the same time without interfering with each other. If nothing is set, all the descriptor queues are initialized as unicast queues.

Multicast control has four functions. Checking the frame size, security level and queue type (TX or RX), copying the descriptor coming from L2/L3 update using the multicast table, modifying the multicast table using multicast learning and reading the multicast table using multicast searching.

35.5.3.3.1 Frame Check

Frame check aims at checking if a frame has the right properties to be received by CPU using GWRMFSCq, GWDCi.SL, GWDCi.DQT registers and can be monitored using GWEIS0.FSES, GWEIS4.DSSES, GWEIS4.DSSEIOS, GWEIS4.DSSECN, GWEIS5.DCTES, GWEIS5.DCTEIOS, GWEIS5.DCTECN interrupt registers.

Functions:

- GWRMFSCq registers are used to set the maximum frame size accepted for each descriptor queue. Any frame received for queue q with a size bigger than GWRMFSCq.MFS will be discarded and GWEIS0.FSES flag will be set. The frame size considered is the size after VLAN update and with FCS. In this case, multicast will not happen and the frame will totally be discarded.

- GWDCCi.SL registers are used to set the security level of a CPU sub destination. Any frame received unsecure (FDESCR.SEC [MFWD] is not set) for a secure CPU sub destination (GWDCCi.SL is set) discarded and GWEIS4.DSSES flag will be set (during multicast, for all CPU sub destination for which GWDCCi.SL is set frame will be discarded and for all CPU sub destination for which GWDCCi.SL is not set, frame will be received).
- GWDCCi.DQT is used to control is the received frame is going to a reception descriptor queue. Any frame received for a transmission descriptor queue will be discarded and GWEIS5.DCTES flag will be set. In case of multicast, only the targeted transmission queue will see their data lost.
- GWEIS0.FSES interrupt register signals that a descriptor has been rejected because the corresponding frame where oversized.
- GWEIS4.DSSES, GWEIS4.DSSEIOS and GWEIS4.DSSECN interrupt registers signal that a descriptor has been rejected because of an AXI descriptor queue security error.
- GWEIS5.DCTES, GWEIS5.DCTEIOS, GWEIS5.DCTECN interrupt registers signal that a descriptor has been rejected because of a descriptor chain type error.

35.5.3.3.2 Descriptor Copy

Descriptor copy aims at multicasting descriptor to send them to the AXI master interface and the RX data fetch using the multicast table.

The multicast table is divided in two fields, MUL.MN[2:0] which is the multicast number and MUL.MNRCN[5:0] which is the next descriptor chain and is used to link AXI descriptor chains together. AXI descriptor chains linked together form a multicast chain to which a frame will be multicast.

Because the multicast table is based on links all the multicast patterns are not possible.

The multicast table utilization is described in [Figure 35.53](#) through an example.

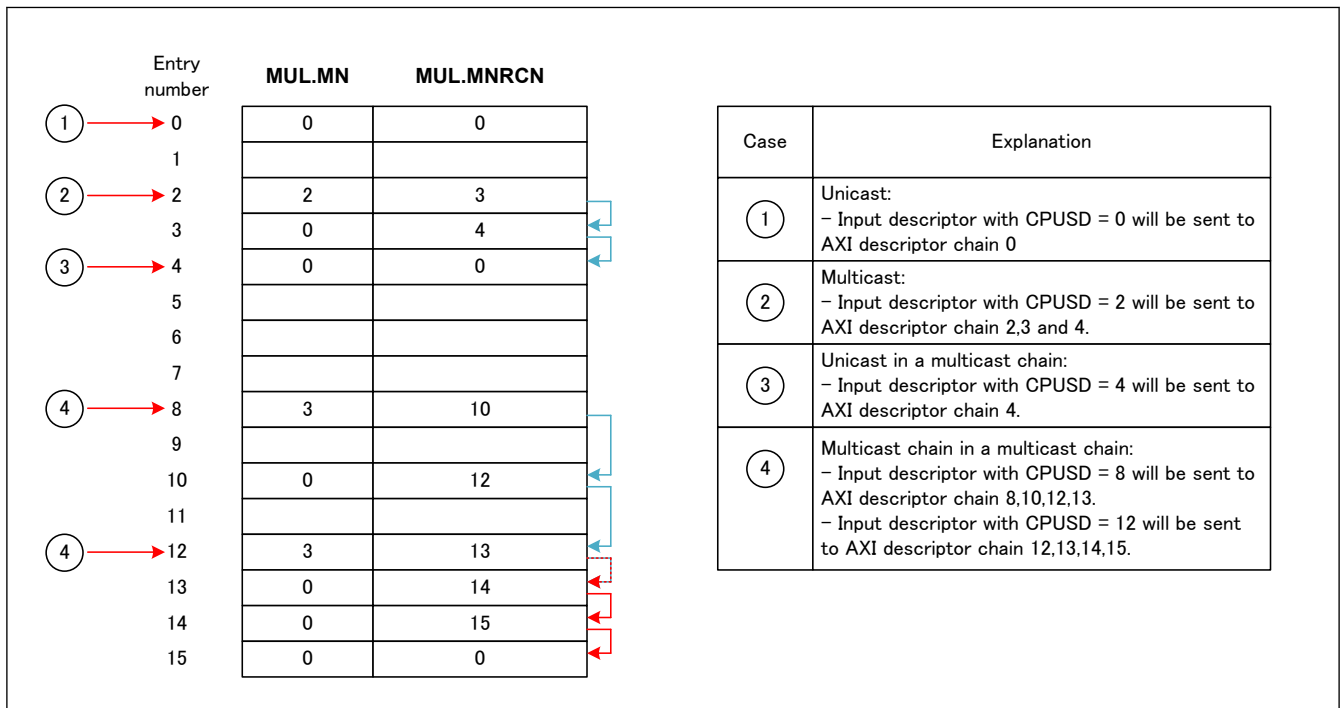


Figure 35.53 Multicast table utilization example

35.5.3.3.3 Multicast Learning

Multicast learning aims at updating entries in the multicast table using GWMSTLS and GWMSTLR registers. [Table 35.19](#) describes mapping between learning registers and fields in multicast table. [Table 35.20](#) describes the learn results, their mapping with the multicast table and how to handle them. It is possible to multicast a frame to 8 AXI descriptor chains.

Functions:

- GWMSTLS register is used to set the information needed to overwrite an entry.

- GWMSTLR register is used to check if the previously set entry has been overwritten.
- To set entries in the multicast table, see software flow in [section 35.4.2.7. Multicast Table Setting Flow](#).
- If GWAC.AMP = 1 (the AXI transaction is pending), learning will not complete. Release the AXI master pose.

Table 35.19 Learn registers/Multicast table mapping registers

Register name	Field name in L3 table
GWMSTLS.MSENL	Not present in multicast table, used to indicate which entry should be overwritten.
GWMSTLS.MNL	MUL.MN, see section 35.5.3.3.2. Descriptor Copy
GWMSTLS.MNRCNL	MUL.MNRCN, see section 35.5.3.3.2. Descriptor Copy

Table 35.20 Learn result

Register name	Field name	Field explanation
GWMSTLR.MTLF	Learning fail	Learning fail will happen in one of the following conditions: • When an APB tries to learn a multicast entry and the multicast table is not ready (GWMTIRM.MTR is not set)

(1) Static learning

When a system aims at using the multicast table in a static way (set in CONFIG mode only) there is no restriction regarding the entry format, the entry link pointers and the learning order of entries.

(2) Dynamic learning

When a system aims at using the multicast table in a dynamic way (set in OPERATION mode), the restrictions described in [Figure 35.54](#) should be respected.

Entry number	MUL.MN	MUL.MNRCN	Restrictions
0			SW: Except the first entry of a multicast chain, no entry can have a multicast number different than 0
1			SW: An entry can only be contained in one multicast chain.
2	2	3	SW: Setting a new multicast chain should always start from the last entry the chain. In this example, learning should follow this order: entry 4, entry 3 and then entry 2.
3	0	4	
4	0	0	SW: Suppression a multicast chain happens by only writing 0 to MUL.MN in the first entry of the corresponding multicast chain. In this example, written 0 to MUL.MN in entry 2 will disable the multicast chain.
5			SW: Re-configuration a multicast chain is only possible by suppressing and setting the corresponding multicast chain.
6			
7			

Figure 35.54 Multicast table dynamic utilization restrictions

35.5.3.3.4 Multicast Searching

Multicast searching aims at reading entries in the multicast table using GWMSTSS and GWMSTSR registers. [Table 35.21](#) describes mapping between learning registers and fields in multicast table. [Table 35.22](#)

Functions:

- GWMSTSS register is used to set the entry that should be read.
- GWMSTSR register is used to read the read result.
- To read an entry in the multicast table, see software flow in [section 35.4.2.9. AXI RAM Reset Flow](#).

Table 35.21 Search registers/Multicast table mapping registers

Register name	Field name in L3 table
GWMSTSS.MSENS	Not present in multicast table, used to indicate which entry should be read.

Table 35.22 Search result

Register name	Field name/ corresponding field in multicast table	Field explanation
GWMSTSR.MNR	MUL.MN	Multicast table read value
GWMSTSR.MNRCNR	MUL.MNRCN	Multicast table read value

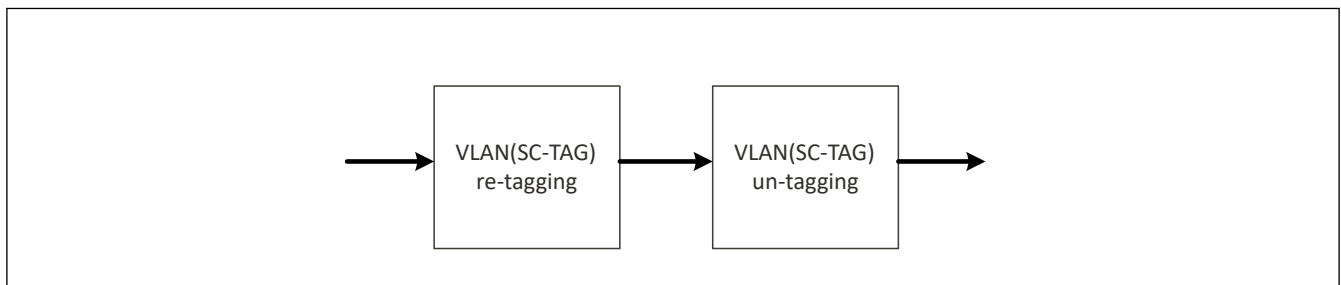
35.5.3.4 RX Data Fetch

RX data fetch has the following functions. Fetching the data from the Local RAM, untagging/ VLANs depending on VLAN settings, inserting an R-TAG, updating the data depending on L2/L3 update information fetched in L2/L3 update, handling FCS, sending the data the AXI master interface and releasing the buffer pointer of the read frames.

The data fetching from Local RAM, the data sending to the AXI master interface and the pointer release are here for switch hardware purpose and its description is not required for switch utilization so it will not be described.

35.5.3.4.1 VLAN (SC-TAG)

Fetching the data from the Local RAM, updating the data depending on “L23 update” and “TAG function registers”. Updating VLANs data path is described in [Figure 35.55](#).

**Figure 35.55 VLAN data path**

(1) VLAN(SC-TAG) re-tagging

Layer 2 update can happen for the following fields:

- Destination MAC address updated to L23U.MDA when L23U.MDAU set.
- Source MAC address is updated to {GWMAC0.MAU, GWMAC1.MAL} when L23U.MSAU set.
- C-TAG VID is updated to L23U.CVID when L23U.CVIDU set.
- C-TAG PCP is updated to L23U.CPCP when L23U.CPCPU set.
- C-TAG VID is updated to L23U.CDEI when L23U.CDEIU set.
- S-TAG VID is updated to L23U.SVID when L23U.SVIDU set.
- S-TAG PCP is updated to L23U.SPCP when L23U.SPCPU set.
- S-TAG VID is updated to L23U.SDEI when L23U.SDEIU set.

Restrictions:

- HW: To update the C-TAG/S-TAG fields, the C-TAG/S-TAG should be present in the frame after VLAN untagging ((2) [VLAN\(SC-TAG\) untagging](#)). If not preset, the update will not happen (e.g. L2 update does not change the frame format and does not permit TAG insertion).

(2) VLAN(SC-TAG) untagging

VLAN update aims at modifying VLAN frame format depending on switch VLAN mode (FWGC.SVM register in MFWD), the VLAN control information given by the forwarding engine descriptor (FDESCR.VCTRL [FWD]) and GWVCC.VEM register.

Functions:

- GWVCC.VEM register sets the VLAN reception mode.

The frames format after VLAN untagging are described in [Figure 35.56](#), [Figure 35.57](#) and [Figure 35.58](#) Switch VLAN mode (FWGC.SVM register in MFWD) and VLAN control information. The conversion between the frame stored in the Local RAM and the frame format after VLAN untagging is described in [Figure 35.59](#).

GWVCC.VEM					
VCTRL	3'b000	3'b001	3'b010	3'b011	3'b100
0	No TAG	No TAG	No TAG	No TAG	No TAG
1	No TAG	C-TAG	C-TAG	C-TAG	C-TAG
2	No TAG	C-TAG	C-TAG	SC-TAG	SC-TAG
3	No TAG	CoS-TAG	CoS-TAG	CoS-TAG	CoS-TAG
4	NA	NA	NA	NA	NA
5	NA	NA	NA	NA	NA
6	NA	NA	NA	NA	NA
7	NA	NA	NA	NA	NA

Figure 35.56 Frame format after VLAN untagging: Switch in No-VLAN mode [MFWD]

GWVCC.VEM					
VCTRL	3'b000	3'b001	3'b010	3'b011	3'b100
0	No TAG	No TAG	C-TAG	No TAG	C-TAG
1	No TAG	C-TAG	C-TAG	C-TAG	C-TAG
2	No TAG	C-TAG	C-TAG	SC-TAG	SC-TAG
3	No TAG	CoS-TAG	C-TAG	CoS-TAG	C-TAG
4	No TAG	No TAG	C-TAG	No TAG	C-TAG
5	No TAG	No TAG	C-TAG	No TAG	C-TAG
6	No TAG	No TAG	C-TAG	No TAG	C-TAG
7	No TAG	CoS-TAG	C-TAG	CoS-TAG	C-TAG

Figure 35.57 Frame format after VLAN untagging: Switch in C-VLAN mode [MFWD]

GWVCC.VEM					
VCTRL	3'b000	3'b001	3'b010	3'b011	3'b100
0	No TAG	No TAG	C-TAG	No TAG	SC-TAG
1	No TAG	C-TAG	C-TAG	C-TAG	SC-TAG
2	No TAG	C-TAG	C-TAG	SC-TAG	SC-TAG
3	No TAG	CoS-TAG	C-TAG	CoS-TAG	SC-TAG
4	No TAG	No TAG	C-TAG	No TAG	SC-TAG
5	No TAG	C-TAG	C-TAG	C-TAG	SC-TAG
6	No TAG	C-TAG	C-TAG	C-TAG	SC-TAG
7	No TAG	CoS-TAG	C-TAG	CoS-TAG	SC-TAG

Figure 35.58 Frame format after VLAN untagging: Switch in SC-VLAN mode [MFWD]

Frame storage in Local RAM																	
TAG RAM			Data RAM														
SN	C-TCI	S-TCI	XX	XX	Type	MAC Destination						MAC Destination					
			XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX

Frame after VLAN un-tagging																	
No TAG	XX	XX	Type	MAC source						MAC Destination							
	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	
C-TAG	C-TCI	00	81	MAC source						MAC Destination							
	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	XX	Type			
SC-TAG	S-TCI	A8	88	MAC Destination						MAC Destination							
	XX	XX	XX	XX	XX	XX	XX	XX	XX	Type	C-TCI	00	81				

Figure 35.59 Local RAM frame to frame format after VLAN untagging conversion

- Note:
- The Local RAM frame to frame format after VLAN untagging conversion for CoS-TAG is not described because it is the same as C-TAG except that the VID in the C-TCI will be fixed to 0.
 - The R-TAG insertion is not described here because part of the L2/L3 update function, see [section 35.5.3.4.2. R-TAG Insertion](#).
 - H8100 corresponds to C-TAG TPID, C-TCI corresponds to a C-TAG TCI with a non-null VID, CoS-TCI corresponds to a C-TAG TCI with a null VID and CCs-TCI corresponds to C-TCI or CoS-TCI [802.1Q].
 - H88A8 corresponds to S-TAG TPID and S-TCI corresponds to an S-TAG TCI [802.1Q].
 - Type corresponds to any Ethernet type value which is not HF1C1, H88A8 or H8100.
 - TPID values can be changed in Forwarding Engine [FWD]. Values for TPIDs can be set in forwarding engine (in FWTTTC0 and FWTTTC1 registers).

35.5.3.4.2 R-TAG Insertion

The R-TAG insertion happens for the two following cases:

- For frames received with an R-TAG (LDESCR.RTGI was set during reception, see [section 35.5.2.3.3. Frame Saving in Local RAM](#), so, in descriptor received for forwarding, FDESCR.RTGI was also set) and for which routing is not requested (FDESCR.RV was not set in received descriptor [MFWD]) or for which routing is requested but the R-TAG

stripping is not request by routing (L23U.RTU [MFWD] was not set to 10 in rule number FDESCR.RN [MFWD] during routing rule fetching by L2/L3 update block, see [section 35.5.3.2. L2/L3 Update](#)).

- For frames received without an R-TAG and for which routing and the R-TAG insertion is requested (L23U.RTU [MFWD] was not set to 01 in rule number FDESCR.RN [MFWD] during routing rule fetching by L2/L3 update block, see [section 35.5.3.2. L2/L3 Update](#)).

In both cases, the R-TAG sequence number (SN in [Figure 35.60](#)) will be set to the sequence number FDESCR.SEQN [MFWD] received in received descriptor.

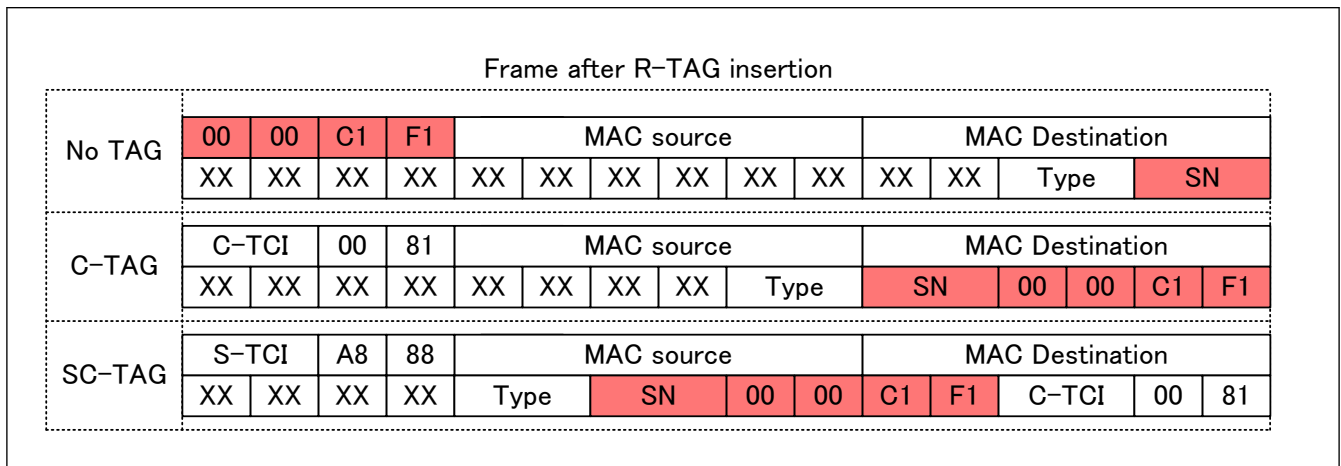


Figure 35.60 R-TAG inserted frame format

- Note:
- HF1C1 corresponds to R-TAG TPID and SN correspond to R-TAG sequence number[802.1CB].
 - H8100 corresponds to C-TAG TPID, C-TCI corresponds to a C-TAG TCI with a non-null VID, CoS-TCI corresponds to a C-TAG TCI with a null VID and CCs-TCI corresponds to C-TCI or CoS-TCI [802.1Q].
 - H88A8 corresponds to S-TAG TPID and S-TCI corresponds to an S-TAG TCI [802.1Q].
 - Type corresponds to any Ethernet type value which is not HF1C1, H88A8 or H8100.
 - TPID values can be changed in Forwarding Engine [FWD]. Values for TPIDs can be set in forwarding engine (in FWTTTC0 and FWTTTC1 registers).

35.5.3.4.3 L2 /L3 Update

L2/L3 updates is used to the layer 2 and the layer 3 of frames. L2/L3 update happens for frames for which routing is requested (received descriptor with FDESCR.RV set [FWD]) and depending on the update information fetched by the L2/L3 update block (see [section 35.5.3.2. L2/L3 Update](#)).

(1) L2 update

See [\(1\) VLAN\(SC-TAG\) re-tagging](#).

(2) L3 update

Layer 3 updates happens when L23U.TTLU was set. L3 update will happen in two steps, the frame decoding and the L3 update.

The decoder decodes the type field (see [section 35.5.2.3.1. TAG Information Extraction](#)) of the Layer 3 update frame and separate frames in the following formats:

- IPv4: When type field is equal to H0800.
- IPv6: When type field is equal to H86DD.
- Unknown: When type field is other than H0800 and H86DD.

Depending on the decoding result, the L3 update will happen has following:

- For IPv4: TimeToLive will be decremented by 1 if it is other than 0 and HeaderChecksum will be corrected using a relative equation (If the checksum were wrong in the input frame, the checksum will also be wrong for the output L3 updated frame).

- For IPv6: HopLimit will be decremented by 1 if it is other than 0.
- For Unknown: L3 update will be ignored.

35.5.3.4.4 FCS Handling

Ethernet frame FCS can be forwarded to the CPU using GWRGC.RCPT register.

Functions:

- GWRGC.RCPT is used to pass frame FCSs to the CPU.

However, following conditions should be met for GWCA to forward a frame FCS to the CPU:

- For Ethernet agent ETHA, RMAC should not have removed the FCS [RMAC] at the input port.
- For GWCA, the FCS should be present in the ingress frame and valid (FI set and FW not set in input descriptor, see [section 35.5.2.2.3. Data Transmission Common Descriptor Format](#)).
- The VLANs should be the same at the egress than at the ingress port.
- The frame should not be a routed frame (received descriptor FDESCR.RV field not set MFWD).

Note:

- If one of the above conditions is not met by a frame, the FCS will be removed from it even if GWRGC.RCPT register is set to 1.
- If a frame is received by CPU with an FCS, FI flag will be set in the reception descriptor. If not, it will not be set.

35.5.3.5 AXI Master Interface

The AXI master interface handles the data/AXI descriptor exchange with the CPU. It receives the updated data and the information related to the data for RX data fetch block and stores them in the CPU RAM. To do so, the AXI master interface uses GDCCi.EDE, GDCCi.ETS, GDCCi.SM, and can be monitored using GWAARSS, GWAARSR0 and GWAARSR1, GWIDAUASi, GWIDASMi, GWIDASAMi0, GWIDASAMi1, GWIDACAMi0, GWIDACAMi1 registers and GWEIS2i.DFEST, GWEIS3.IAOESi, GWEIS4.DSES, GWEIS4.DSEIOS, GWEIS4.DSECN interrupt registers.

Functions:

- GDCCi.EDE and GDCCi.ETS registers set the descriptor format that are used for descriptor queue number i. See [section 35.5.1.2.1. General Formats](#).
- GDCCi.SM register sets how GWCA writes back descriptors. In this section, all explanation will use GDCCi.SM equal to 00b. For GDCCi.SM equal to 01b or 10b, see the register explanation.
- For GWAARSS, GWAARSR0 and GWAARSR1 registers, see [section 35.5.1.6. AXI Address RAM Searching](#).
- GWIDAUASi, GWIDASMi, GWIDASAMi0, GWIDASAMi1, GWIDACAMi0, GWIDACAMi1 registers and GWEIS3.IAOESi interrupt register are used for incremental area monitoring, see [section 35.5.3.5.3. Single Page Incremental Data Reception](#).
- GWEIS2i.DFEST interrupt register signals that frames have been lost because a full descriptor queue.
- GWEIS4.DSES, GWEIS4.DSEIOS, GWEIS4.DSECN interrupt registers signal that frames have been received with an unexpected size, see [section 35.5.3.5.2. Size-controlled Data Reception](#).

Data reception can be done using linear or cyclic descriptor queues (see [section 35.5.1.5. AXI Descriptor Queue Format](#)). Data reception process descriptors can be set six different ways, through the basic data reception descriptor chain ([section 35.5.3.5.1. Basic Data Reception](#)), the Size-controlled data reception descriptor chain ([section 35.5.3.5.2. Size-controlled Data Reception](#)), the Single page incremental data reception descriptor chain ([section 35.5.3.5.3. Single Page Incremental Data Reception](#)), the Interrupt based multi-page incremental data reception descriptor chain ([section 35.5.3.5.4. Interrupt Based on Multi-page Incremental Data Reception](#)), the Header removal incremental data reception descriptor chain ([section 35.5.3.5.5. Header Removal Incremental Data Reception](#)).

35.5.3.5.1 Basic Data Reception

Basic data reception process descriptor utilization is described in [Figure 35.61](#). Basic data reception process descriptor format is described in [Figure 35.62](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.23](#).

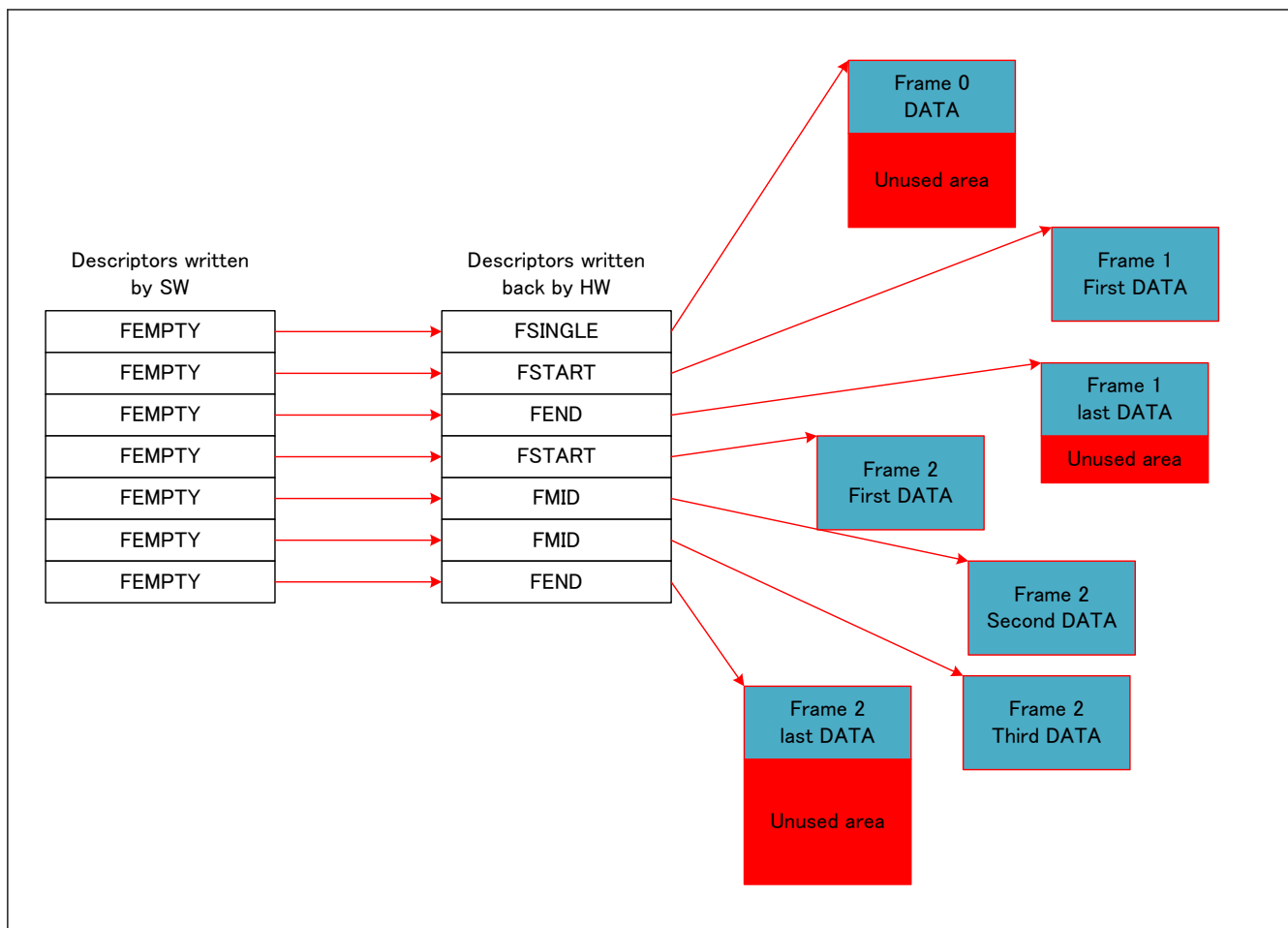


Figure 35.61 Basic data reception process descriptor utilization

- Note:
- If a frame is too big to fit in an area reserved by one descriptor, it will be automatically split. The first descriptor will be FSTART, the last descriptor will be FEND and all other descriptors will be FMID.
 - When a frame is split the hardware will write back the FSTART descriptor last. Even if the queue is set in keep DT mode, this order is kept.

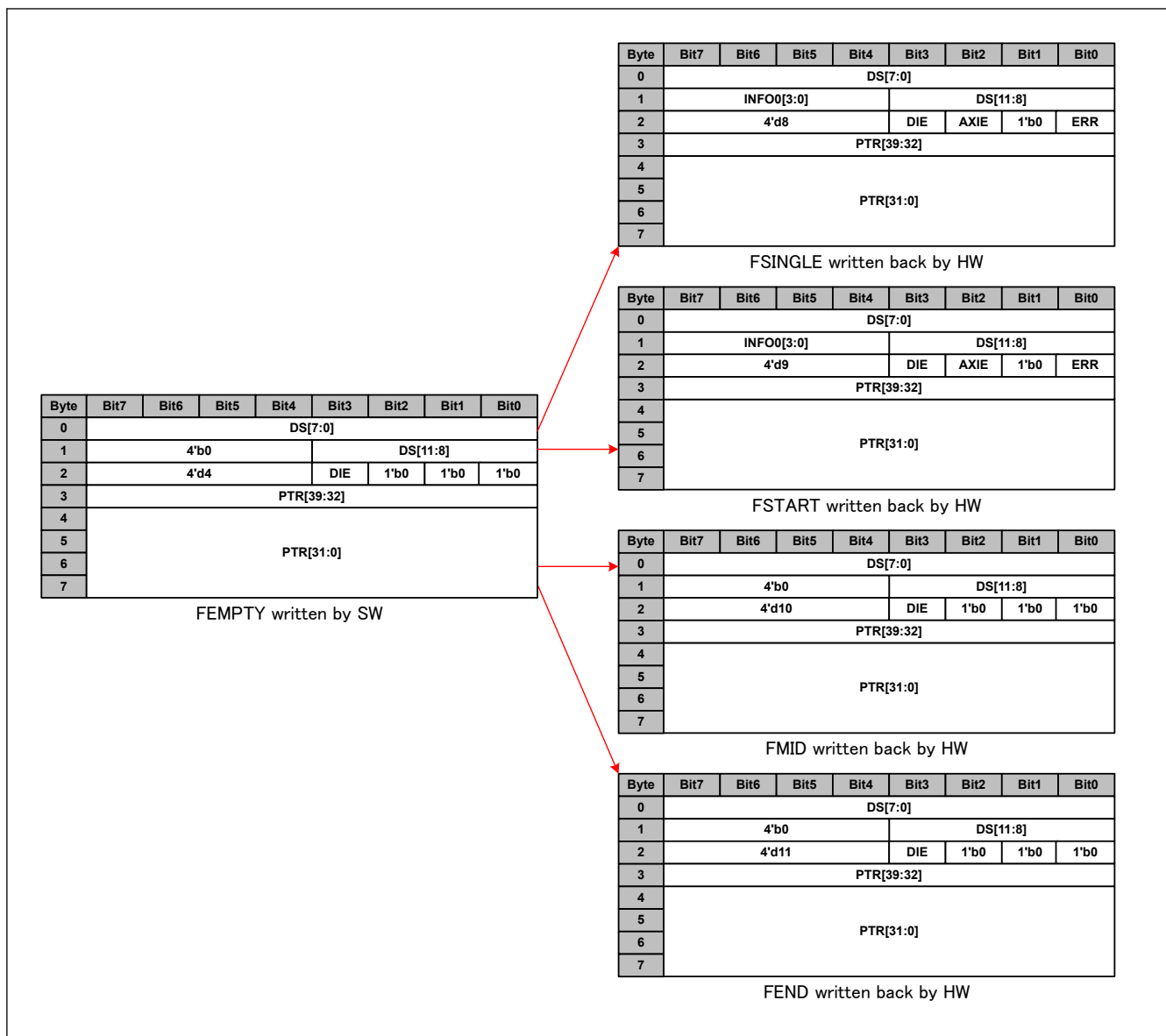


Figure 35.62 Basic data reception process descriptor utilization format

Table 35.23 Basic data reception process descriptor utilization field description (1 of 2)

Field name	Field description before write-back	Field description after write-back
DS	Size of available URAM memory associated to the pointer for data storage	Actual size that has been written by the AXIBMI in the URAM.
INFO0	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
ERR	0	Set conditions: - HW: A descriptor queue full error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back. - HW: A descriptor number error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame will not be written) and a FSINGLE or a FSTART descriptor written back.
DSE	0	0

Table 35.23 Basic data reception process descriptor utilization field description (2 of 2)

Field name	Field description before write-back	Field description after write-back
AXIE	0	Set conditions: - HW: An AXI error happened while writing a data (BRESP != 00) and a FSINGLE or a FSTART descriptor written back. - HW: An AXI error happened while reading a descriptor during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back.
DIE	DIE	DIE (Value is copied from read descriptor).
DT	4 for FEMPTY	8 for FSINGLE 9 for FSTART 10 for FMID 11 for FEND
PTR	Address where data should be written in the URAM	PTR (Value is copied from read descriptor).
INFO1 (GWDCCi.E DEi == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
TS (GWDCCi.E TSi == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .

35.5.3.5.2 Size-controlled Data Reception

Size-controlled data reception aims at controlling the frame size at reception while purposely splitting the header and the payload of a frame. By size-controlled data reception the SW ensure that descriptor order is maintained even if a too short or too long frame is received. Size-controlled data reception process descriptor utilization is described in [Figure 35.63](#). Size-controlled data reception process descriptor format is described [Figure 35.64](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) in and [Table 35.24](#). Size-controlled data reception can be monitored using GWEIS4.DSES, GWEIS4.DSEIOS, GWEIS4.DSECN interrupt registers.

Function:

- GWEIS4.DSES, GWEIS4.DSEIOS, and GWEIS4.DSECN interrupt registers signal a frame received with an unexpected size.

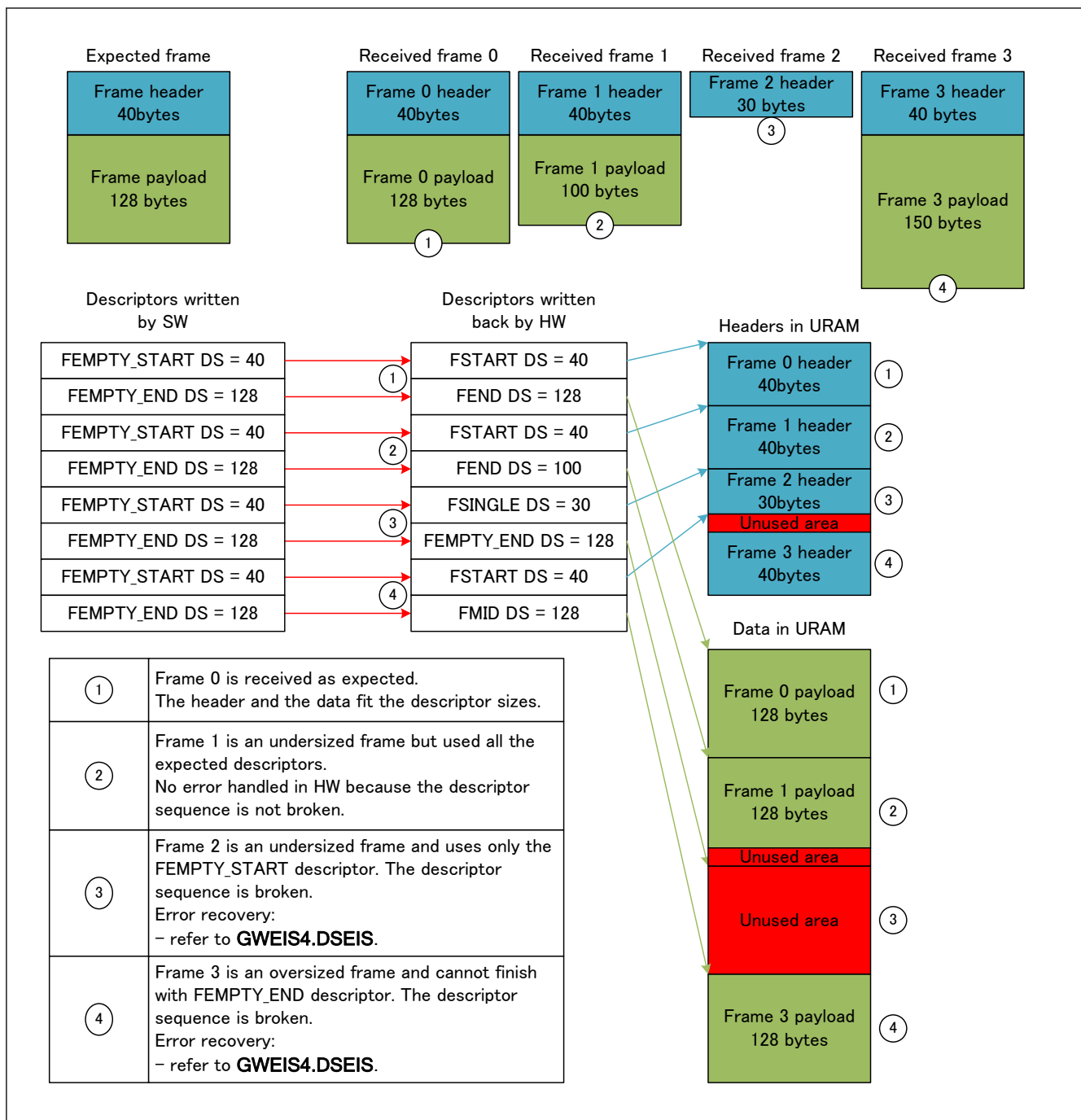


Figure 35.63 Size-controlled data reception process descriptor utilization

- Note:
- When a frame is split the hardware will write back the FSTART descriptor last. Even if the queue is set in keep DT mode, this order is kept.
 - A frame can be split in more than two descriptors by adding FEMPTY_MID descriptors between FEMPTY_START and FEMPTY_END descriptors.

Restrictions:

- SW: The process descriptor queue should always repeat the pattern of the same descriptors from the first descriptor to the descriptor before the termination descriptor. This pattern should start with an FEMPTY_START descriptor, finish by a FEMPTY_END descriptor and in between be only composed by FEMPTY_MID descriptors. If this is violated (for example, FEMPTY_START is detected in the middle), the frame will be discarded/rejected without notification.

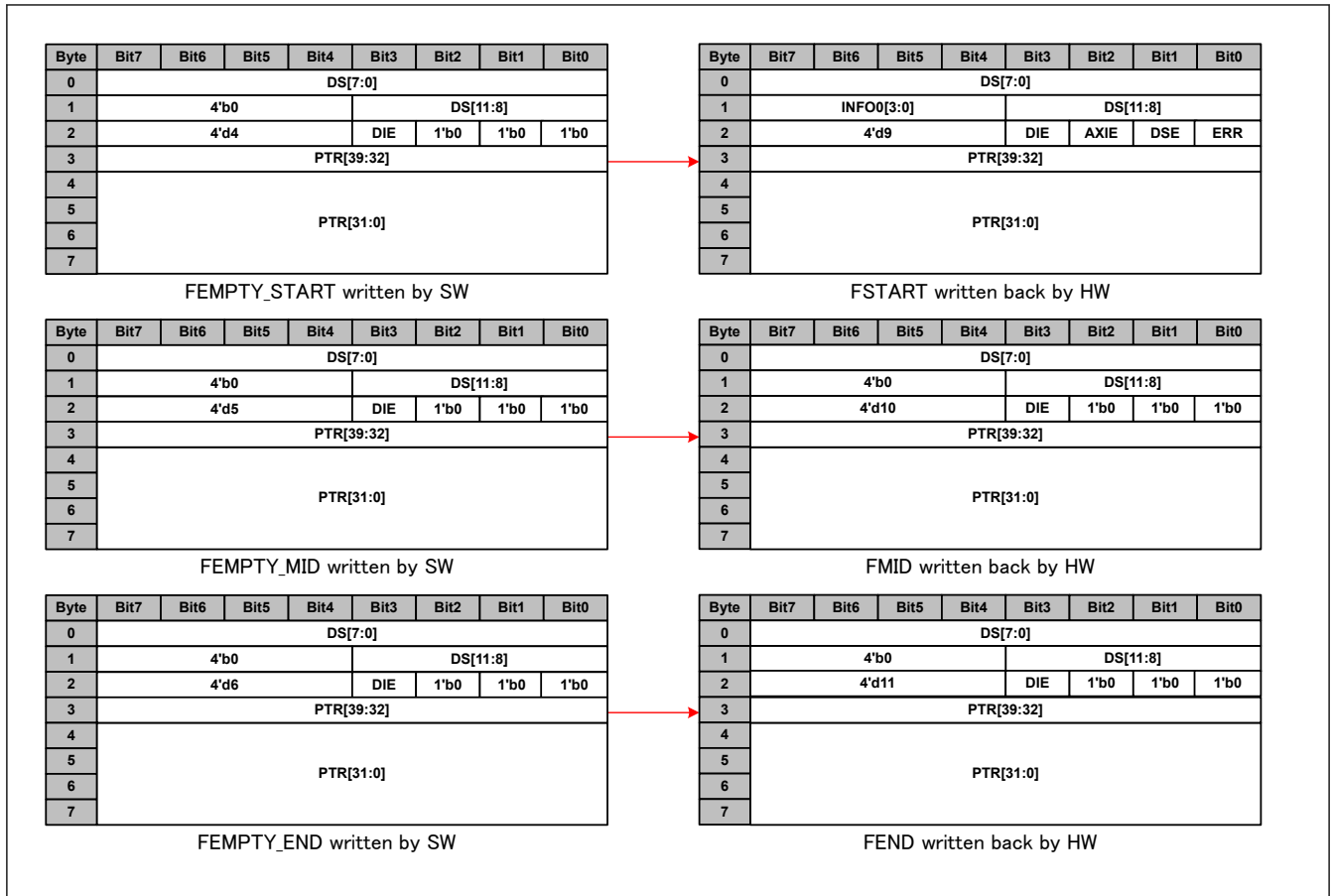


Figure 35.64 Size-controlled data reception process descriptor utilization format

Table 35.24 Size-controlled data reception process descriptor utilization field description (1 of 2)

Field name	Field description before write-back	Field description after write-back
DS	Size of available URAM memory associated to the pointer for data storage	Actual size that has been written by the AXIBMI in the URAM.
INFO0	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
ERR	0	Set condition: - HW: A descriptor queue full error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back. (This case should not happen due to the process descriptor order) - HW: A descriptor number error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame will not be written) and a FSINGLE or a FSTART descriptor written back.
DSE	0	Set conditions: - HW: The frame has been finished with a FEMPTY_START or a FEMPTY_MID descriptor (undersized frame) - HW: The frame is too big to finish with a FEMPTY_END descriptor (oversized frame)
AXIE	0	Set conditions: - HW: An AXI error happened while writing a data (BRESP != 00) and a FSINGLE or a FSTART descriptor written back. - HW: An AXI error happened while reading a descriptor during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back.
DIE	DIE	DIE (Value is copied from read descriptor).

Table 35.24 Size-controlled data reception process descriptor utilization field description (2 of 2)

Field name	Field description before write-back	Field description after write-back
DT	5 for FEMPTY_START 6 for FEMPTY_MID 7 for FEMPTY_END	9 for FSTART 10 for FMID 11 for FEND
PTR	Address where data should be written in the URAM	PTR (Value is copied from read descriptor).
INFO1 (GWDCCi.E DEi == 1)	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
TS (GWDCCi.E TSi == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .

35.5.3.5.3 Single Page Incremental Data Reception

Single page incremental data reception aims at saving the received frames back to back in a given memory area. By single page incremental data reception, the SW avoid wasting CPU power to stack data together. Single page incremental data reception process descriptor utilization is described in [Figure 35.65](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)). Single page incremental data reception process descriptor format is described [Figure 35.66](#), [Table 35.25](#), [Figure 35.67](#) and [Table 35.26](#). Single page incremental data reception HW/SW synchronization should be done by GWIDAUASi register and can be monitored by GWIDASMi, GWIDASAMi0, GWIDASAMi1, GWIDACAMi0, GWIDACAMi1 registers and GWEIS3.IAOESi interrupt register.

Functions:

- GWIDAUASi register shows the number of bytes that has been written in the incremental area. After reading the data from the CPU RAM, SW should write to this register the quantity of data that has been read. The CPU data handling can be done, either based on the written back descriptor in the corresponding descriptor queue, or on the byte value contained in the increment area than can be read in GWIDAUASi register.
- GWIDASMi register is used to monitor the total size of the incremental area.
- {GWIDASAMi0.IDASAU, GWIDASAMi1.IDASAL} register array is used to monitor the start address of the incremental area.
- {GWIDACAMi0.IDACAU, GWIDACAMi1.IDACAL} register array is used to monitor the next used address in the incremental area.
- GWEIS3.IAOESi interrupt register signals data loss due to incremental area overflow.

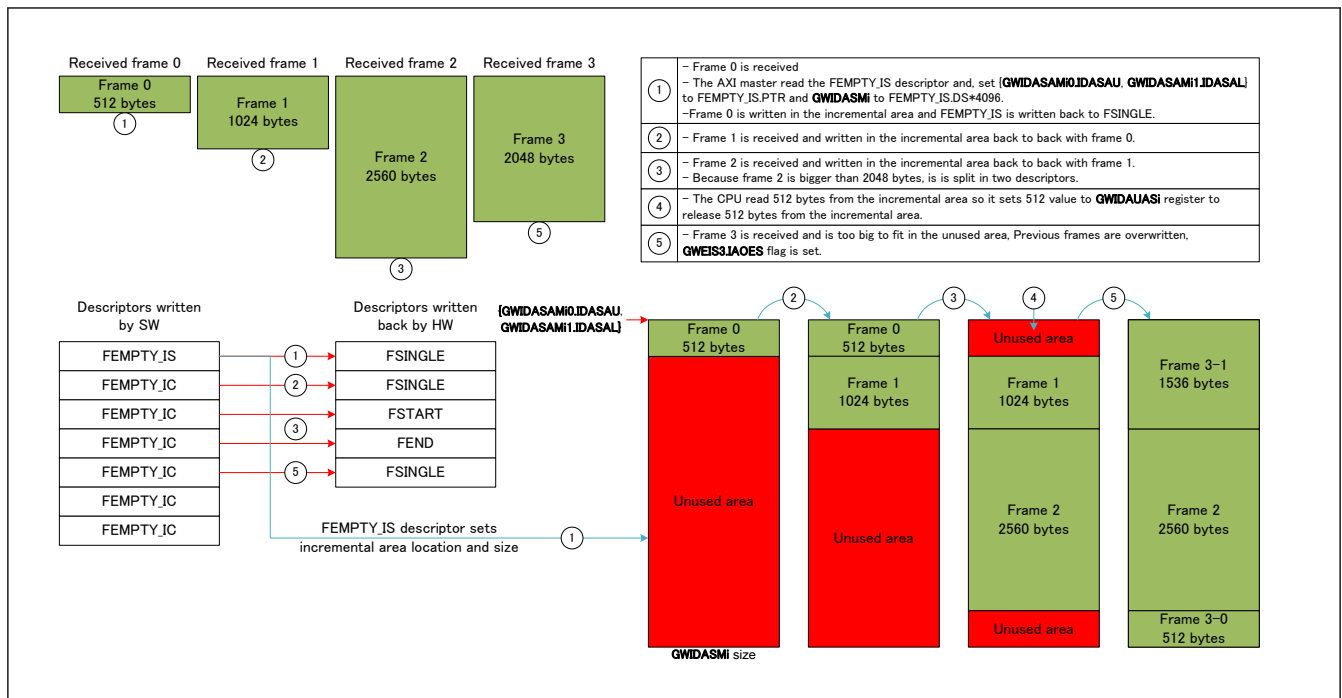


Figure 35.65 Single page incremental data reception process descriptor utilization

Restrictions:

- SW: This reception type can only be used for the first 4 RX chains, see [section 35.5.1.1. AXI Descriptor Queue Mapping](#).
- Note:
- If a frame is too big to fit in an area reserved by one descriptor, it will be automatically split. The first descriptor will be FSTART, the last descriptor will be FEND and all other descriptors will be FMID.
 - When a frame is split the hardware will write back the FSTART descriptor last. Even if the queue is set in keep DT mode, this order is kept.
 - When GWCA reset has been applied (GWMS.OPS set to 00), all the incremental registers (i.e. incremental page start address, current address, page size and used page size) were cleared. Therefore, when the GWCA has moved back again to OPERATION mode (GWMS.OPS set to 11) and a frame has been received by GWCA incremental chain, it is not able to write frame using FEMPTY_IC descriptor. Descriptor of AXI current address have to be (set up) a FEMPTY IS.

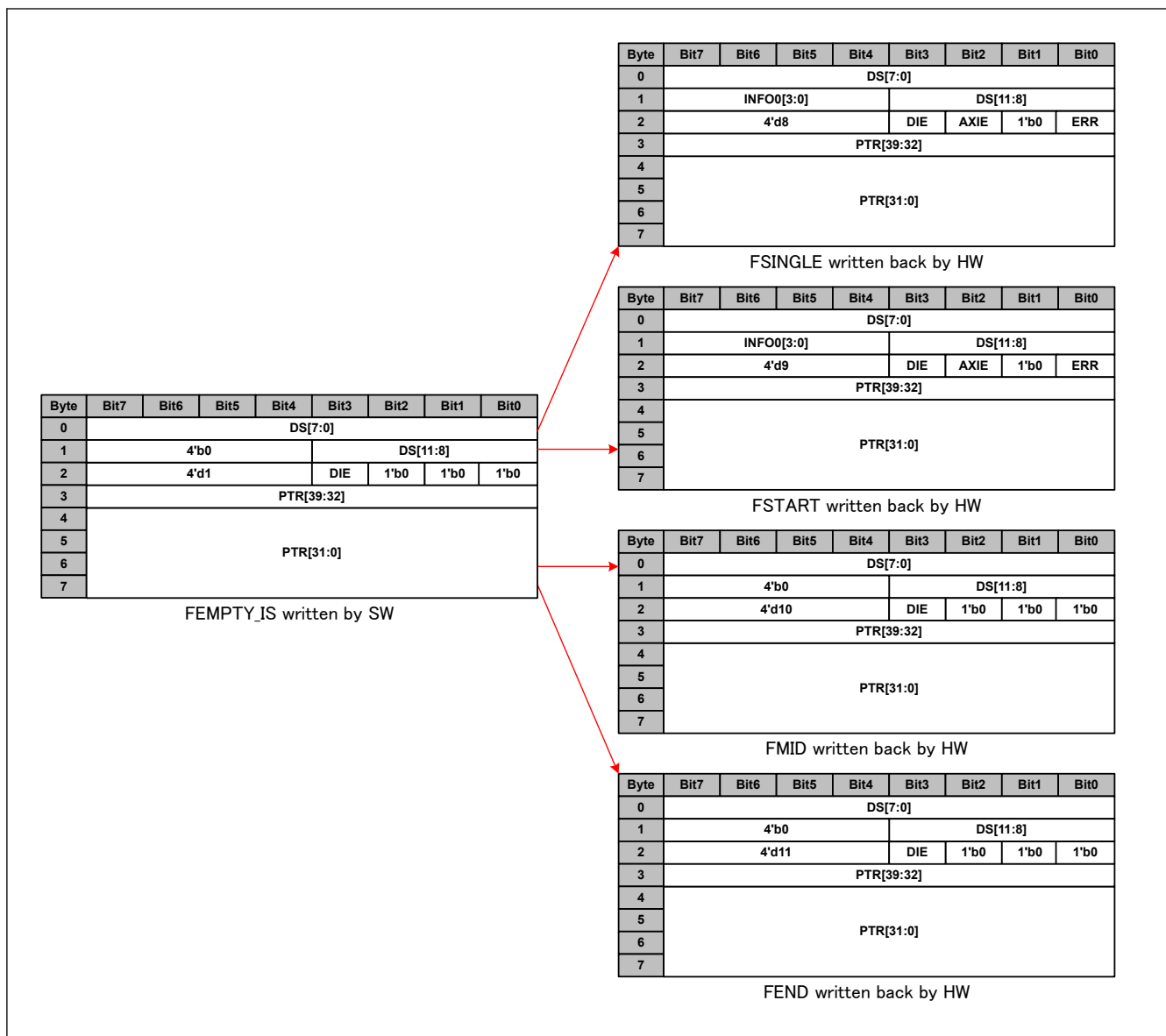


Figure 35.66 Single page incremental data reception FEMPTY_IS process descriptor utilization format

Table 35.25 Single page incremental data reception FEMPTY_IS process descriptor utilization field description (1 of 2)

Field name	Field description before write-back	Field description after write-back
DS	Size of the incremental area in 4 KB unity.	Size of the data stored in the URAM associated tot the descriptor.
INFO0	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
ERR	0	Set conditions: - HW: A descriptor queue full error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back. - HW: A descriptor number error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame will not be written) and a FSINGLE or a FSTART descriptor written back.
DSE	0	0

Table 35.25 Single page incremental data reception FEMPTY_IS process descriptor utilization field description (2 of 2)

Field name	Field description before write-back	Field description after write-back
AXIE	0	Set conditions: • HW: An AXI error happened while writing a data (BRESP != 00) and a FSINGLE or a FSTART descriptor written back. • HW: An AXI error happened while reading a descriptor during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back.
DIE	DIE	DIE (Value is copied from read descriptor).
DT	1 for FEMPTY_IS	8 for FSINGLE 9 for FSTART 10 for FMID 11 for FEND
PTR	Address where the incremental area starts in the URAM.	PTR (Value is copied from read descriptor).
INFO1 (GWDCCi.EDE == 1)	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
TS (GWDCCi.ETS == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .

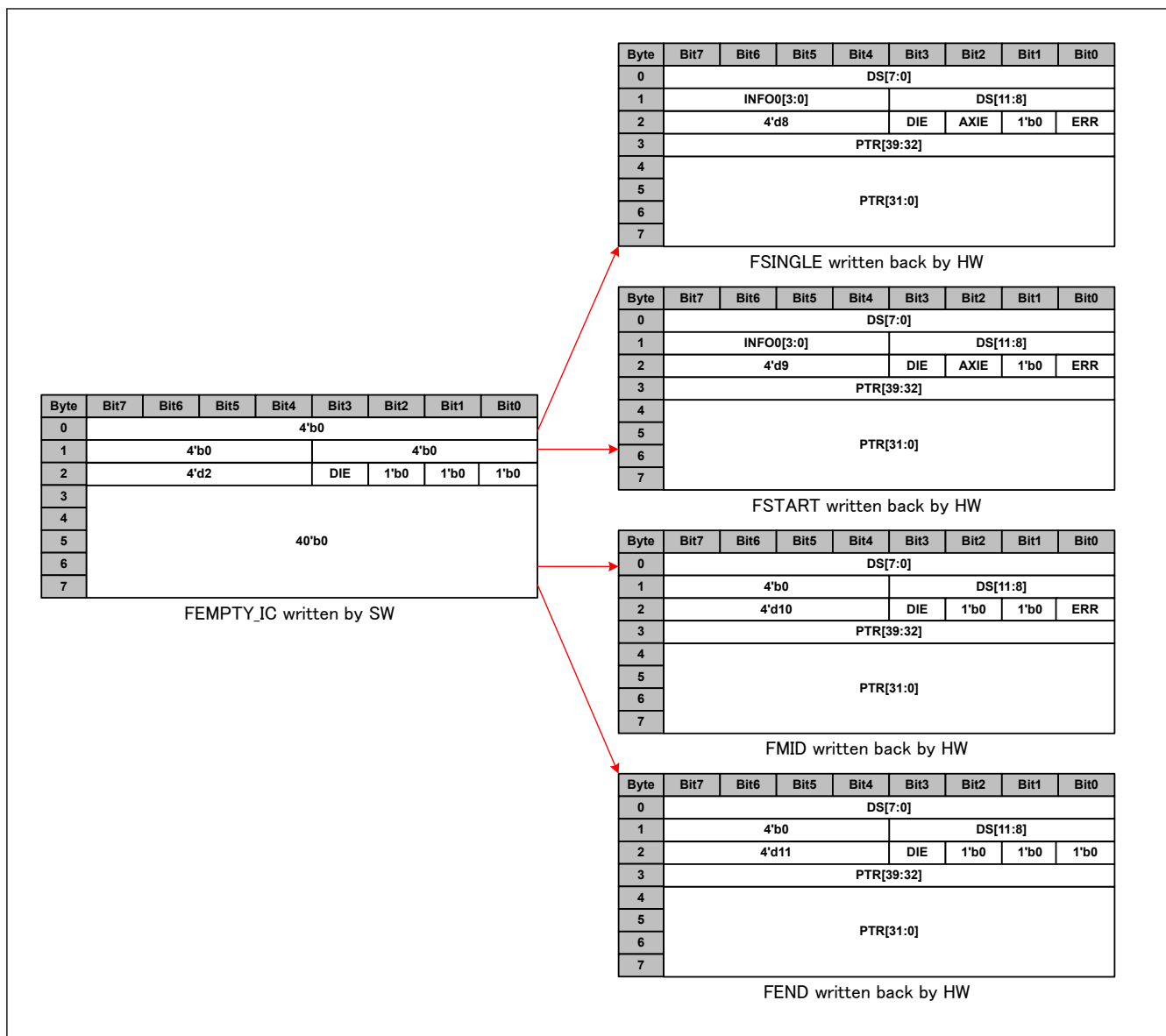


Figure 35.67 Single page incremental data reception FEMPTY_IC process descriptor utilization format

Table 35.26 Single page incremental data reception FEMPTY_IC process descriptor utilization field description (1 of 2)

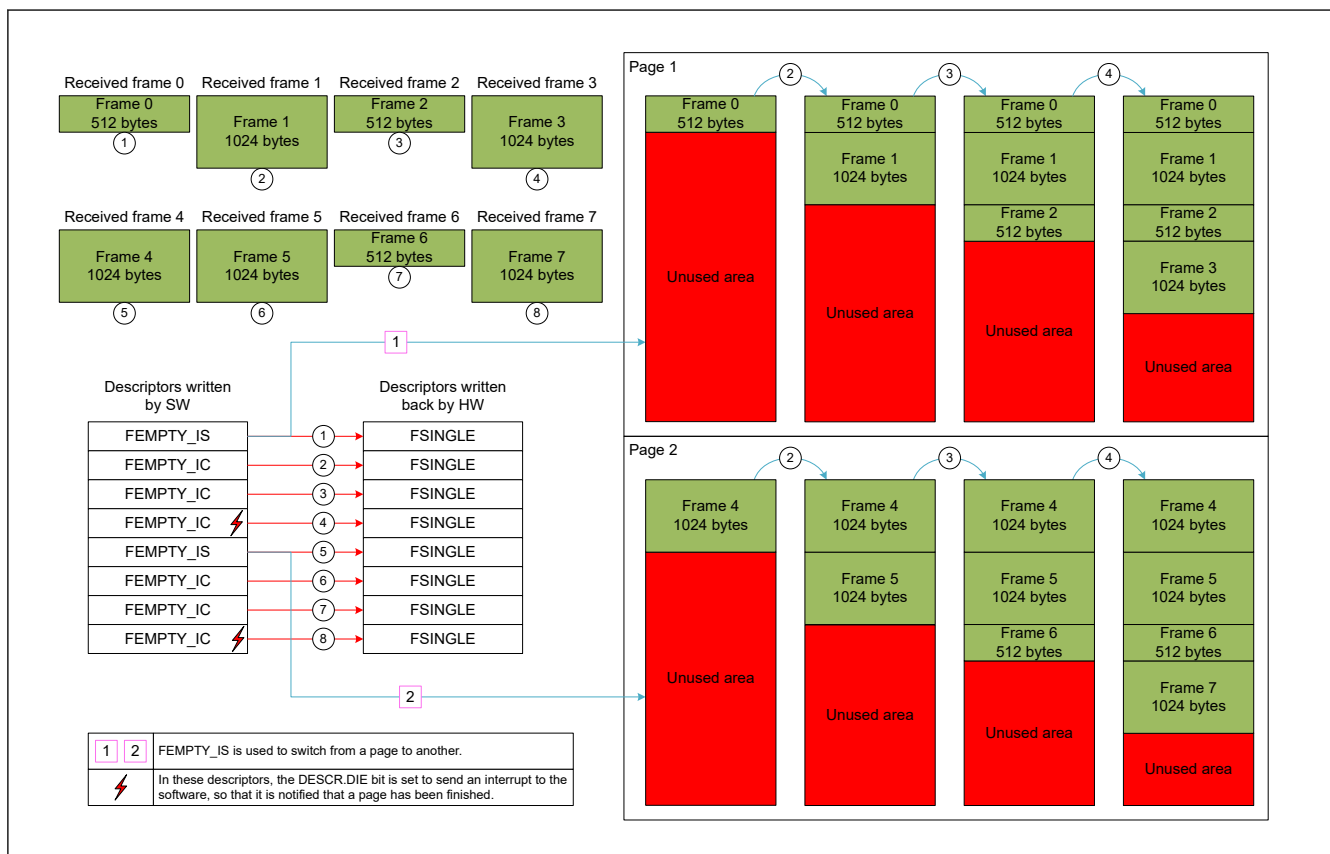
Field name	Field description before write-back	Field description after write-back
DS	0	Size of the data stored in the URAM associated tot the descriptor.
INFO0	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
DSE	0	0
AXIE	0	Set conditions: - HW: An AXI error happened while writing a data (BRESP != 00) and a FSINGLE or a FSTART descriptor written back. - HW: An AXI error happened while reading a descriptor for a split frame and the descriptor was not the first.
DIE	DIE	DIE (Value is copied from read descriptor).

Table 35.26 Single page incremental data reception FEMPTY_IC process descriptor utilization field description (2 of 2)

Field name	Field description before write-back	Field description after write-back
DT	2 for FEMPTY_IC	8 for FSINGLE 9 for FSTART 10 for FMID 11 for FEND
PTR	0	Address where the data corresponding to the descriptor has been written in the URAM
INFO1 (GWDCCi.EDE == 1)	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
TS (GWDCCi.ETS == 1)	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .

35.5.3.5.4 Interrupt Based on Multi-page Incremental Data Reception

An interrupt based on multi-page incremental data reception aims at saving the received frames back to back in a given memory area. By an interrupt based on multi-page incremental data reception, the SW avoids wasting CPU power to stack data together. An interrupt based on multi-page incremental data reception process descriptor utilization is described in [Figure 35.68](#). An interrupt based on multi-page incremental data reception process descriptor format is the same as single page incremental data reception. For details, see [section 35.5.3.5.3. Single Page Incremental Data Reception](#).

**Figure 35.68 Interrupt based on multi-page incremental data reception process descriptor utilization**

Restrictions:

- SW: It should be ensured that, recognizing the expected data and/or controlling page sizes and the number of descriptors for a page, a page cannot overflow.
- SW: This reception type can only be used for the first 4 RX chains.

- Note:
- If a frame is too big to fit in an area reserved by one descriptor, it will be automatically split. The first descriptor will be FSTART, the last descriptor will be FEND, and all other descriptors will be FMID.
 - When a frame is split, the hardware will write back the FSTART descriptor last. Even if the queue is set in keeping DT mode, this order is kept.
 - An interrupt based on multi-page incremental data reception allows any number of pages to be used.
 - When GWCA reset has been applied (GWMS.OPS is set to 00), all the incremental registers (i.e. incremental page start address, current address, page size, and used page size) were cleared. Therefore, when the GWCA has moved back again to OPERATION mode (GWMS.OPS is set to 11) and a frame has been received by GWCA incremental chain, it is not able to write a frame using FEMPTY_IC descriptor. A descriptor of AXI current address has to be (set up) a FEMPTY_IS.

35.5.3.5.5 Header Removal Incremental Data Reception

Header removal incremental data reception aims at suppressing header from incremental data. It is an update of single page incremental data reception ([section 35.5.3.5.3. Single Page Incremental Data Reception](#)) and interrupt based multi-page incremental data reception ([section 35.5.3.5.4. Interrupt Based on Multi-page Incremental Data Reception](#)) by adding a FEMPTY_ND between each incremental descriptor. Header removal incremental data reception process descriptor utilization is described in [Figure 35.69](#). Header removal process descriptor format is described in [Figure 35.70](#) (for basic descriptor, [section 35.5.1.2.1. General Formats](#)) and [Table 35.27](#).

Restrictions:

- SW: A FEMPTY_IS/FEMPTY_IC descriptor can only contain 2048 bytes. It should be taken in account when setting the descriptor in the descriptor queue because the HW does not ensure that the descriptor sequence is correct.

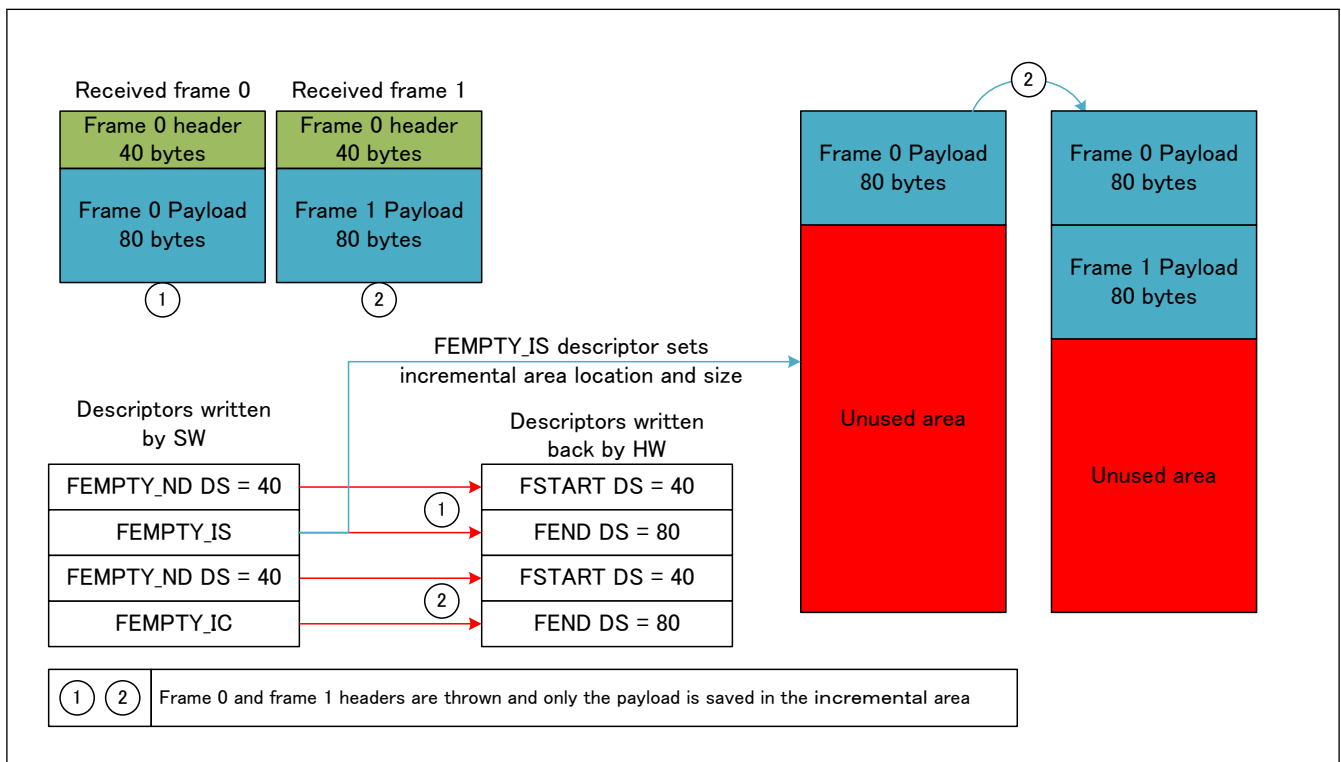


Figure 35.69 Header removal incremental data reception process descriptor utilization

- Note:
- When using Size controlled data reception, to get rid of the header part of a frame and still keep the descriptor order, it is possible to start every frame with a FEMPTY_START descriptor always pointing toward the same memory address (by setting always the same pointer).
 - When GWCA reset has been applied (GWMS.OPS set to 00), all the incremental registers (i.e incremental page start address, current address, page size and used page size) were cleared. Therefore, when the GWCA has moved back again to OPERATION mode (GWMS.OPS set to 11) and a frame has been received by GWCA incremental chain, it is not able to write frame using FEMPTY_IC descriptor. Descriptor of AXI current address have to be (set up) a FEMPTY_IS.

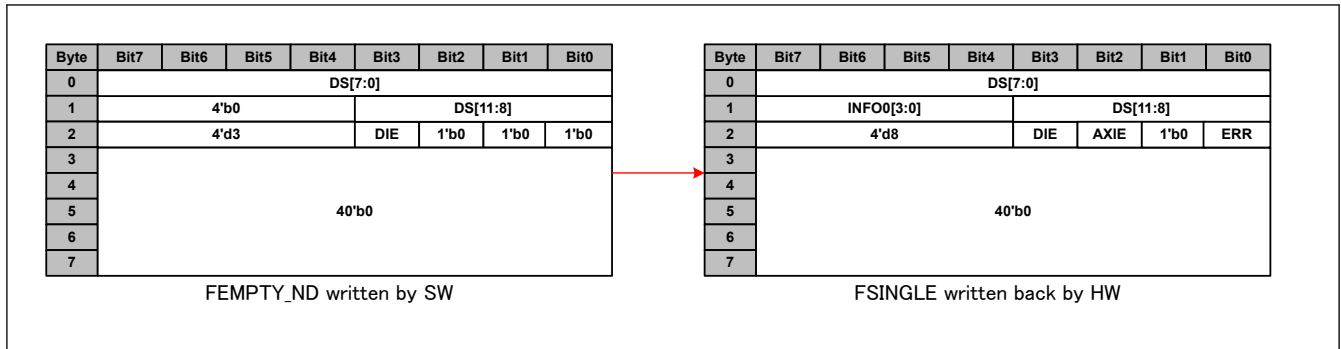


Figure 35.70 Header removal utilization format

Table 35.27 Header removal process descriptor utilization field description

Field name	Field description before write-back	Field description after write-back
DS	Size to be rejected	Actual size that has been rejected (it should be the same as DS before write-back).
INFO0	0	0 for FEND and FMID. For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
ERR	0	Set conditions: - HW: A descriptor queue full error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back. - HW: A descriptor number error happened while reading an AXI address during a split frame (The first descriptor has been processed normally but the last part of the frame will not be written) and a FSINGLE or a FSTART descriptor written back.
DSE	0	0
AXIE	0	Set conditions: - HW: An AXI error happened while writing a data (BRESP != 00) and a FSINGLE or a FSTART descriptor written back. - HW: An AXI error happened while reading a descriptor during a split frame (The first descriptor has been processed normally but the last part of the frame has no descriptor to be written) and a FSINGLE or a FSTART descriptor written back.
DIE	DIE	DIE (Value is copied from read descriptor).
DT	3 for FEMPTY_ND	8 for FSINGLE 9 for FSTART 10 for FMID 11 for FEND Restrictions: SW: Because the HW turns a FEMPTY_ND descriptor in a FSINGLE, FSTART, FMID or FEND descriptor, by reading the descriptor it is impossible to know that it contains no data. Consequently, SW should remember where the FEMPTY_ND descriptors have been written.
PTR	0	0
INFO1 (GWDCCi.EDE == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .
TS (GWDCCi.ETS == 1)	0	0 for FEND and FMID, For FSTART and FSINGLE, see section 35.5.3.5.6. Data Reception Common Descriptor Format .

35.5.3.5.6 Data Reception Common Descriptor Format

In this section are described the fields which are common for any data transmission (INFO0, INFO1 and TS). There are two types of formats, the reception direct descriptor and the reception ethernet descriptor formats. They are respectively created from direct local descriptors and ethernet local descriptors, see [section 35.5.2.3.3. Frame Saving in Local RAM](#). All the fields contained in AXI descriptors, if quoted, will be written ADESCR.{Field name}.

(1) Reception direct descriptor

Reception direct descriptor corresponds to frames sent by a CPU and by deciding their target directly (Forwarding engine processing has been by-passed [FWD]). Reception direct descriptor INFO and TS formats are described in [Figure 35.71](#) and [Table 35.28](#).

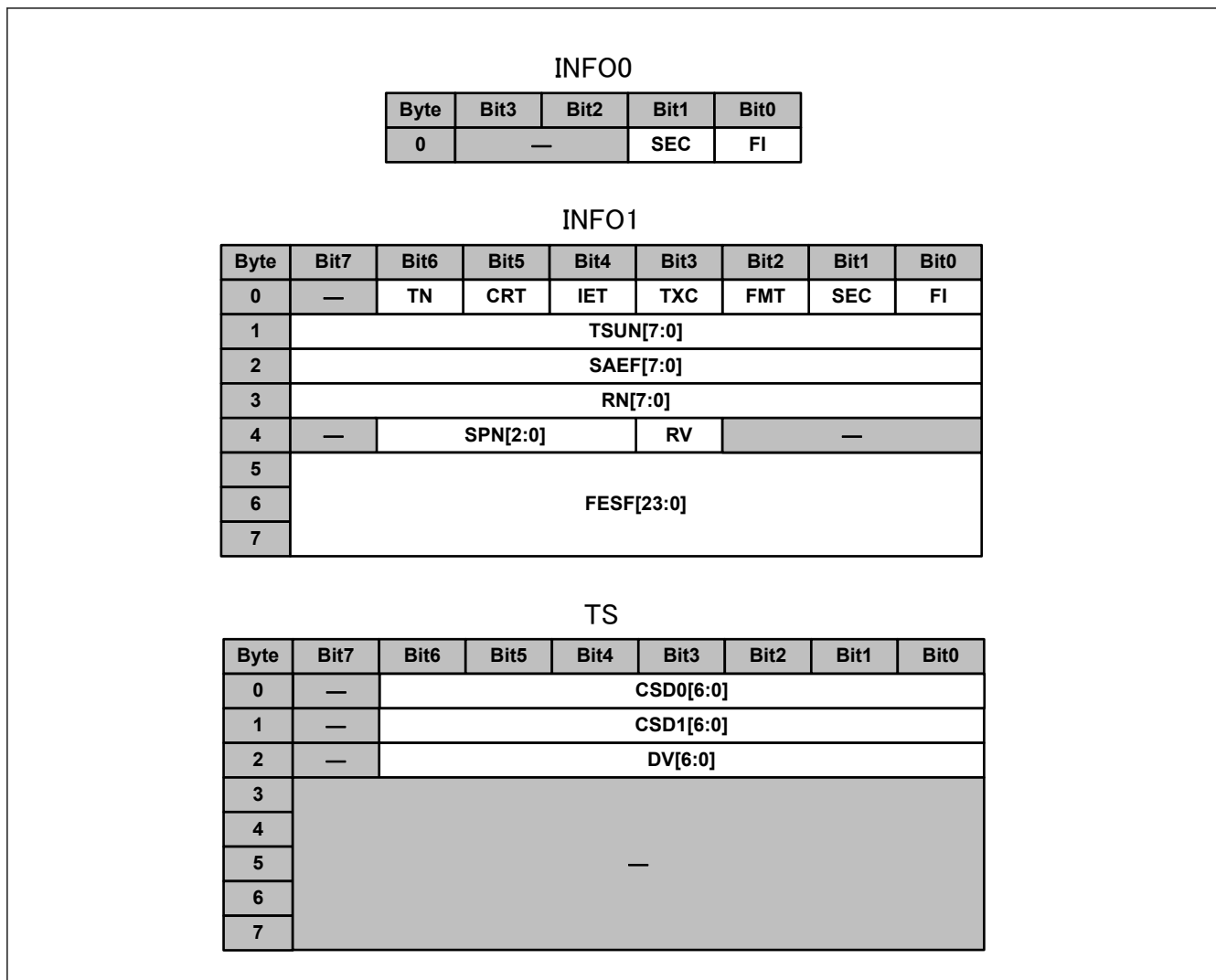


Figure 35.71 Reception direct descriptor INFO and TS formats

Table 35.28 Reception direct descriptor INFO and TS field descriptions (1 of 2)

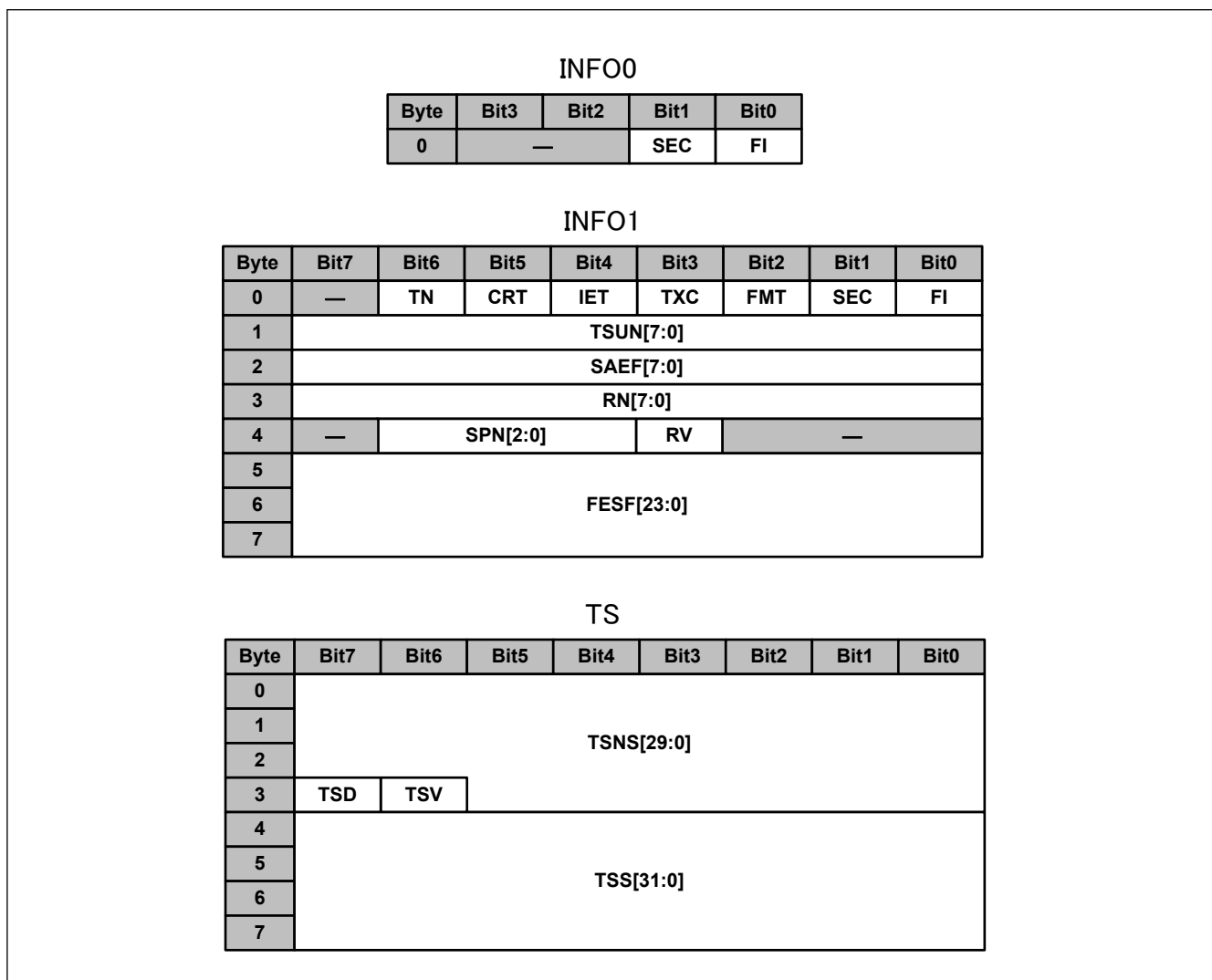
Field name	Bit width	Description	Value for GWCA
FI	1	See Table 35.14 .	See section 8.3.4.5.
SEC	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
FMT	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
TXC	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
IET	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
CRT	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
TN	1	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
TSUN	8	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
SAEF	8	See Table 35.16 .	Copied from direct local descriptor. See (1) Direct local descriptor .
RV	1	See Table 35.14 .	Copied from received descriptor FDESCR.RV field [MFWD]

Table 35.28 Reception direct descriptor INFO and TS field descriptions (2 of 2)

Field name	Bit width	Description	Value for GWCA
RN	5	See Table 35.14 .	Copied from received descriptor FDESCR.RN field [MFWD]
SPN	2	Source port number	Port number from which the frame entered the switch [MFAB] Copied from received descriptor FDESCR.SPN field [MFWD]
FESF	24	Forwarding engine status flags	Copied from received descriptor FDESCR.MINFO field [MFWD]
CSD0	6	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
DV	4	See Table 35.14 .	Copied from direct local descriptor. See (1) Direct local descriptor .
—	—	Reserved area	Set to 0

(2) Reception ethernet descriptor

Reception ethernet descriptor corresponds to frames that has been by the MFWD. Reception ethernet descriptor INFO and TS formats are described in [Figure 35.72](#) and [Table 35.29](#).

**Figure 35.72 Reception ethernet descriptor format INFO and TS formats****Table 35.29 Reception ethernet descriptor format INFO and TS field descriptions (1 of 2)**

Field name	Bit width	Description	Value for GWCA
FI	1	See Table 35.15 .	See section 8.3.4.5.

Table 35.29 Reception ethernet descriptor format INFO and TS field descriptions (2 of 2)

Field name	Bit width	Description	Value for GWCA
SEC	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
FMT	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TXC	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
IET	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
CRT	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TN	1	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TSUN	8	See Table 35.15 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
SAEF	8	See Table 35.17 .	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
RV	1	See Table 35.15 .	Copied from received descriptor FDESCR.RV field [MFWD]
RN	5	See Table 35.15 .	Copied from received descriptor FDESCR.RN field [MFWD]
SPN	2	Source port number	Port number from which the frame entered the switch [MFAB] Copied from received descriptor FDESCR.SPN field [MFWD]
FESF	24	Forwarding engine status flags	Copied from received descriptor FDESCR.MINFO field [MFWD]
TSV	1	Timestamp valid	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TSD	1	Timestamp default	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TSNS	30	Timestamp nanosecond [gPTP] PCH header timestamp [29:0]	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
TSS	32	Timestamp second [gPTP] PCH header timestamp [31:30] Restrictions: HW: The 16-upper bits of the gPTP second part are truncated.	Copied from ethernet local descriptor from [GWCA Data transmission] or [ETHA Data reception]
—	—	Reserved area	Set to 0

35.5.4 Timestamp Reception [gPTP]

GWCA allows Timestamp reception through the GWCA TS path, The TS path is described in [Figure 35.73](#).

Timestamps received through this function are only Ethernet agent TX timestamps. Ethernet agent RX timestamps are stored in the local RAM along with their corresponding data and can only be received by CPU through RX data path with the corresponding data.

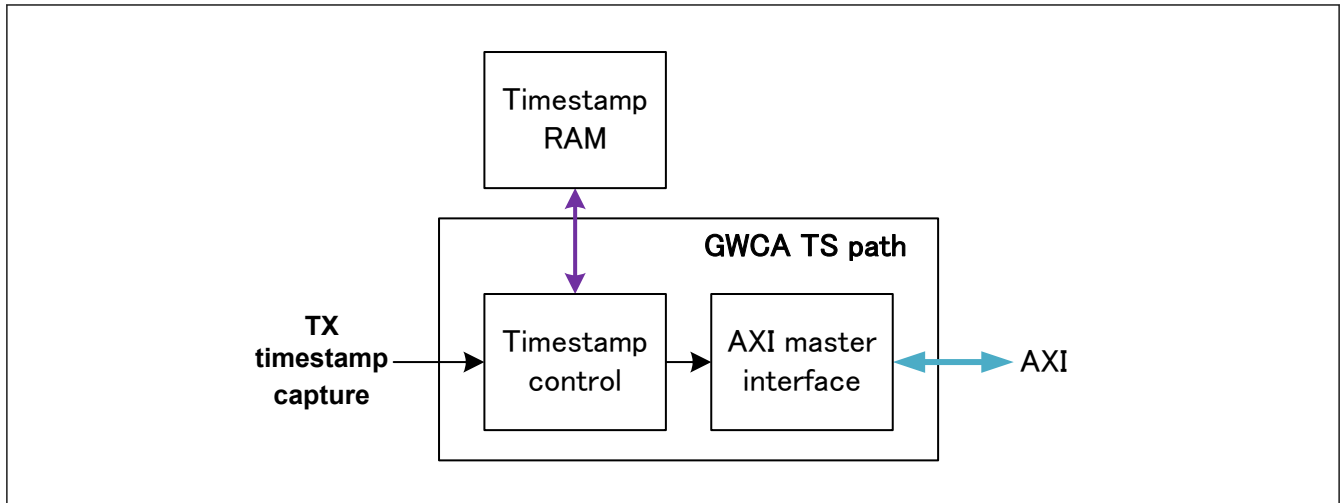


Figure 35.73 GWCA TS path block diagram

The TS path is separated in two blocks:

- **Timestamp control:** This block aims at receiving the timestamps from TX Timestamp Capture interface [RMAC], storing them in the Timestamp RAM and sending them to the AXI master interface.
- **AXI master interface:** This block handles the timestamp/AXI descriptor exchange with the CPU.

35.5.4.1 Timestamp Control

Timestamp control controls the descriptor storage/fetching to/from the Timestamp RAM using GWTSDCCs.TE register and can be monitored using GWTSNM and GWTSMNM registers and GWEIS0.TSOVFES interrupt registers.

Functions:

- GWTSDCCs.TE is used to enable timestamp reception for timer *s*. If enabled, all timestamps received from TX Timestamp Capture interfaces [RMAC] for timer *s* will be stored in the timestamp RAM and later given to the AXI master interface.
- GWTSNM register is used to monitor the current number of timestamps present in the timestamp RAM.
- GWTSMNM register is used to monitor the maximum number of timestamps that has been held in the timestamp RAM since the previous reset or the previous time corresponding register has been read.
- GWEIS0.TSOVFES interrupt register is used to flag a timestamp loss because of Timestamp RAM overflow.

35.5.4.2 AXI Master Interface

The AXI master interface handles the timestamp/AXI descriptor exchange with the CPU. It will receive the timestamps and the information related to the timestamps and store them in the CPU RAM. To do so, it uses GWTDCACs0, WTDCACs1, and GWTSDCCs.DCS registers and it can be monitored using GWTSDIS.TSDIS and GWEIS0.TDFES interrupt registers.

Functions:

- GWTDCACs0 and GWTDCACs1 registers form the AXI address {GWTDCACs0.TSCCAU, GWTDCACs1.TSCCAL} which correspond to the current address being processed by the GWCA for TS descriptor queue *s*. It can be used to set the first address of the descriptor queue during GWCA initialization, move an under-use TS descriptor queue to another location in the CPU RAM or monitor the currently processed descriptor address.
- GWTSDCCs.DCS register is used to map timer *s* timestamps to a TS descriptor queue number.
- GWTSDIS.TSDIS interrupt register is used to flag some timestamps are ready in the CPU RAM.
- GWEIS0.TDFES interrupt register is used to flag a timestamp loss because the timestamp descriptor queue is full.

Timestamp reception can be done using linear or cyclic descriptor queues (see [section 35.5.1.5. AXI Descriptor Queue Format](#)). Timestamp reception process descriptor utilization is described in [Figure 35.74](#). Timestamp reception process descriptor format is described in [Figure 35.75](#) and [Table 35.30](#).

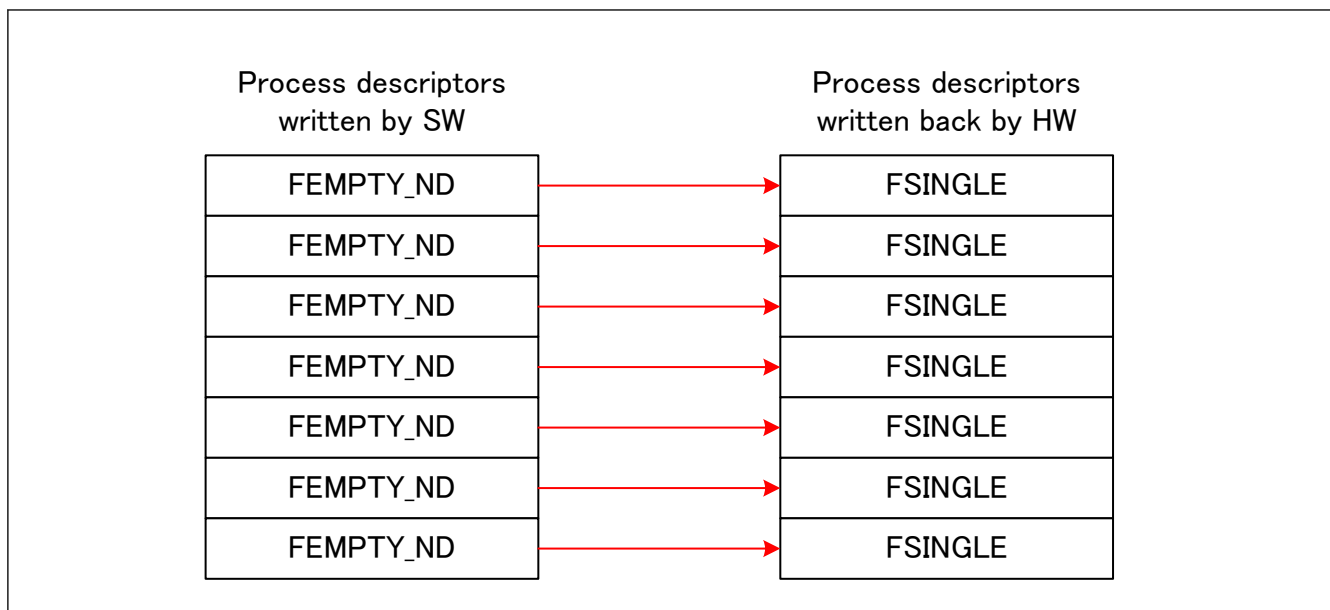


Figure 35.74 TS reception process descriptor utilization

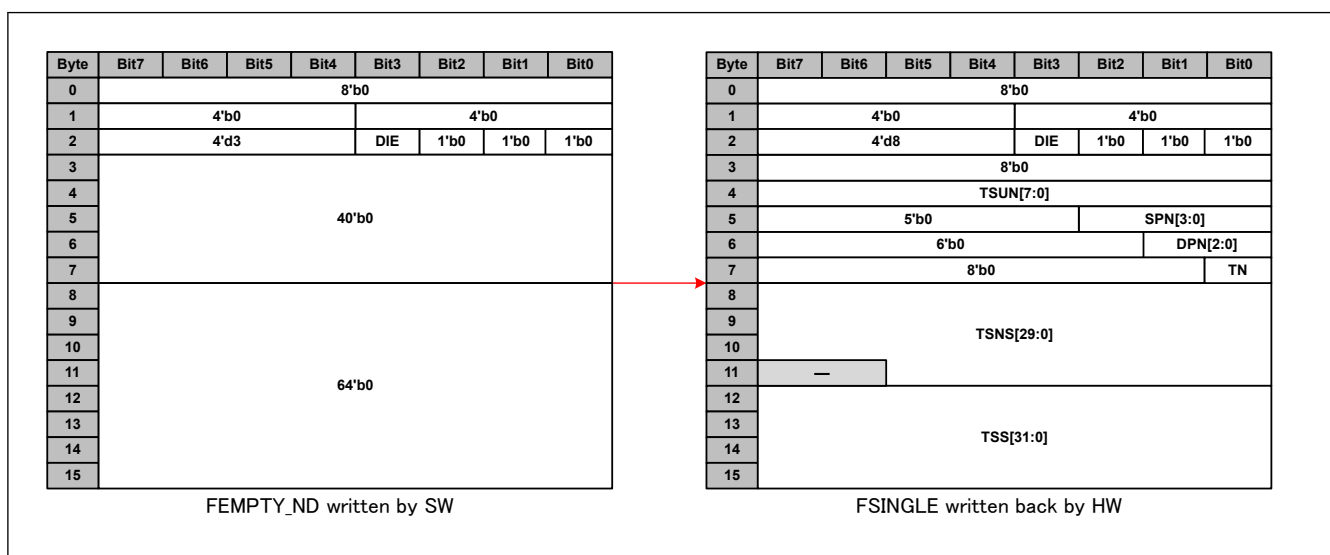


Figure 35.75 TS reception process descriptor utilization format

Table 35.30 TS process descriptor utilization field description (1 of 2)

Field name	Field description written by SW	Field description after HW write-back
DS	0	0
INFO0	0	0
ERR	0	0
DSE	0	0
AXIE	0	0
DIE	DIE	DIE (Value is copied from read descriptor).
DT	3 for FEMPTY_ND	8 for FSINGLE
PTR	0	Used as timestamp related information field See Table 35.31
INFO1	NA: Info1 is never present for TS reception	NA: Info1 is never present for TS reception

Table 35.30 TS process descriptor utilization field description (2 of 2)

Field name	Field description written by SW	Field description after HW write-back
TS	0	See Table 35.32

Table 35.31 TS reception process descriptor PTR field explanation

Field name	Bit width	Description
TSUN	8	Timestamp unique number [RMAC]
SPN	2	Port number from which the timestamp corresponding frame entered the switch [MFAB]
DPN	1	Port number by which the timestamp has been taken
TN	1	Timer Number

Table 35.32 TS reception process descriptor TS field explanation

Field name	Bit width	Description
TSNS	30	Timestamp nanosecond [gPTP]
TSS	32	Timestamp second [gPTP] Restrictions: HW: The 16-upper bits of the gPTP second part are truncated.

35.5.5 gPTP Triggered Transmission

It is possible to use gPTP timer to trig a periodic transmission start using time-controlled data transmission (see [Table 35.13](#)). To do so, CYCLIC_COMP[i][c] (Where i is 0 for GWCA0) functional interface is linked to INTER_TX[8+c] (see [Figure 35.21](#)).

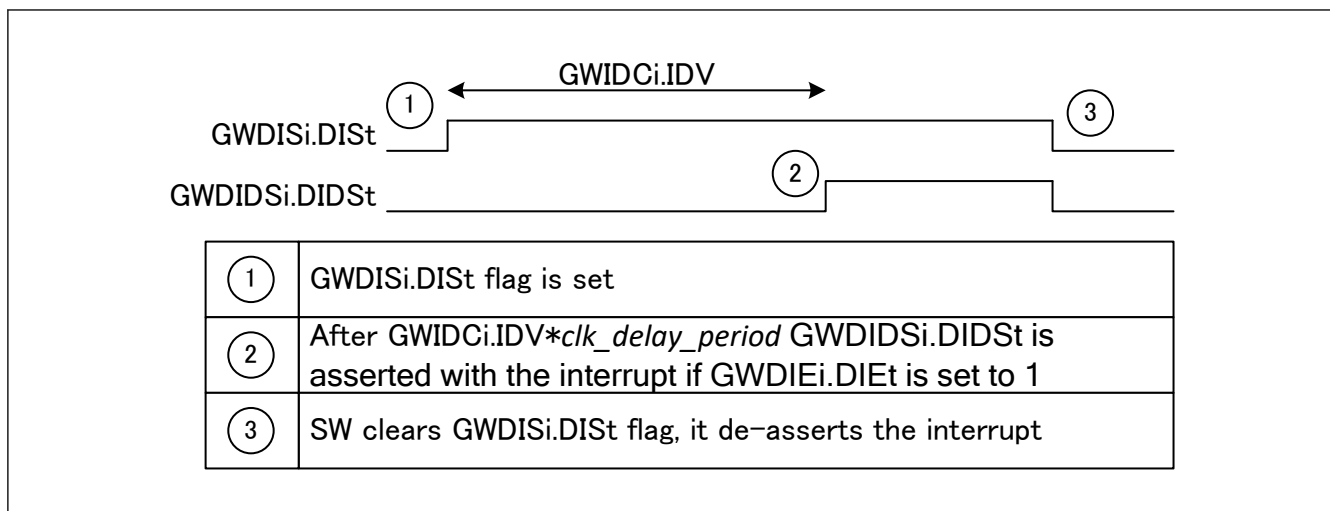
For 8 parameter explanation, see gPTP specification [gPTP].

35.5.6 Interrupt Delay Function

Interrupt delay function aims at delaying data interrupts (GWDISi.DISi) to accumulate data during a certain among of time before requesting software to process the data. As a result, software load is diminished, and data latency is still guaranteed. To operate, interrupt delay function uses GWIDPC and GWIDCi registers and can be monitored using GWDIDSi interrupt register. Interrupt delay function is described in [Figure 35.76](#).

Functions:

- GWIDPC register sets an internal clock of period *clk_delay_period* used as base clock for interrupt delay function.
- GWIDCi register sets the delay time for descriptor queue i in *clk_delay_period*.
- GWDIDSi.DIDSt register is the delayed GWDISi.DISi interrupt register and triggers the interrupt gwc_gwdis_int[t].
- For interrupt handling, software can decide to read GWDIDSi register and to process only interrupts that have been set until their corresponding delay time is archived (allows a certain amount of data surely came before processing) or can decide to read GWDISi register and to process all the interrupts even the ones which did not reach their delay time (allows a certain amount of data surely came before processing at least for one queue can diminish the interrupt load).

**Figure 35.76** Interrupt delay function

Restrictions:

- HW: The maximum delay achievable by the interrupt delay function is $\text{clk_period} \times 4194304$. For example, it will be around 10 ms for a clock clk with a frequency of 400 MHz.

35.6 Precautions

- AXI address bus width
The AXI address bus width degenerated from 40 bits to 32 bits by products connection. Please use AXI descriptor address with only 32 bits. Please fix to all 0 for upper 8 bits for AXI address. (For example, cannot use GWDCBAC0.)